SQL for Analysts

A practical introduction to querying, transforming, and understanding relational data

---

Chapter 1: The Relational World — Databases, Tables, and Your First Query

> ***SQL for Analysts***
>
> *A practical introduction to querying, transforming, and understanding relational data*
>
> *© Patrick Dolinger, NSCC Institute of Technology*
>
> *Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)*
>
> *You are free to share and adapt this material for any purpose, provided appropriate credit is given.*

Chapter Overview

This chapter is your entry point into the world of relational databases and SQL. If you have never written a line of SQL before, welcome — this is the right place to start. If you have some prior exposure, the early sections will feel familiar and the later ones will add precision to concepts you may have used intuitively.

By the end of this chapter you will have connected to a live database, navigated its structure, understood what a relational database actually is and why it works the way it does, and written your first SELECT statements. More importantly, you will understand *why* each piece of syntax exists — not just what to type, but what you are asking for when you type it.

By the end of this chapter you will be able to:

- Explain what a relational database is and how it differs from a spreadsheet
- Navigate the CabotTrail Outdoor database environment in SSMS

- Describe the purpose of schemas and explain how they organise tables
- Write a basic SELECT statement that returns specific columns from a table
- Control the number and order of rows returned using TOP and ORDER BY
- Write meaningful comments in SQL code
- Identify the four categories of SQL: DQL, DML, DDL, and DCL

Table of Contents

1. What Is a Database — and Why Not Just Use a Spreadsheet?
2. Relational Databases: The Core Idea
3. SQL: The Language of Data
4. Connecting to SQL Server with SSMS
5. The CabotTrail Outdoor Environment
6. Schemas: Organising Tables into Groups
7. Your First SELECT Statement
8. Controlling Output: TOP and ORDER BY
9. Writing Comments in SQL
10. The System Catalog: Asking the Database About Itself
11. Chapter Summary
12. Review Questions
13. Deeper Dive

---

1. What Is a Database — and Why Not Just Use a Spreadsheet?

If you have worked with data before, you have almost certainly used a spreadsheet — Excel, Google Sheets, or something similar. Spreadsheets are excellent tools. They are visual, immediate, and flexible. You can type numbers, write formulas, and see results without learning anything formal.

So why use a database at all?

The answer is scale, structure, and integrity — and understanding each one will shape how you think about data for the rest of your career.

1.1 Scale

A spreadsheet comfortably handles thousands of rows. With some patience, it can handle tens of thousands. Beyond that, performance degrades, files become unwieldy, and sharing becomes a problem.

CabotTrail Outdoor's sales data has over 16,000 invoice lines across four years of business. That is a manageable spreadsheet — today. Add another year and it doubles. Add more product lines and customers, and within a few years you have hundreds of thousands of rows. A database handles this without breaking a sweat. SQL queries against tables with millions of rows execute in seconds.

1.2 Structure

A spreadsheet is a flat grid. Every piece of information lives in a cell, and there is no formal relationship between cells except what you create with formulas. If a customer's address changes, you find every row where that customer appears and update it manually — or use Find and Replace and hope nothing goes wrong.

A relational database stores *related* information in *related tables*. A customer's address is stored exactly once. Every transaction that references that customer links to the customer record — it does not copy the address. When the address changes, you update it in one place and every transaction instantly reflects the new value.

This is the relational model: organized, precise, and non-redundant.

1.3 Integrity

Spreadsheets enforce no rules. You can type text in a column meant for numbers, enter a date that does not exist, or delete a row that other rows depend on — nothing stops you. The result is data corruption that may not be obvious until it matters.

Databases enforce **constraints** — rules that data must satisfy before it can be stored. A date column cannot accept text. A quantity column cannot be negative if the business rule says so. A transaction cannot reference a customer that does not exist. The database is the last line of defence against bad data entering the system.

1.4 Concurrency

A spreadsheet is a file. If two people open the same file simultaneously, one will overwrite the other's changes. Coordinating access requires awkward workarounds — emailing files, shared network drives with locking, manual merging.

A database is designed for simultaneous access by many users. Thousands of people can query and update the same database at the same time without overwriting each other's work. The database manages this through a system of locks and transactions that is invisible to most users.

---

2. Relational Databases: The Core Idea

A **relational database** organises data into **tables** — structured grids with named columns and typed rows. The "relational" in the name does not mean the tables are friendly with each other — it comes from the mathematical concept of a *relation*, which is essentially what we call a table.

2.1 Tables, Rows, and Columns

Every table in a relational database has:

- **Columns** (also called *fields* or *attributes*): the named categories of data in the table. Every column has a data type — text, number, date, etc.
- **Rows** (also called *records* or *tuples*): the individual entries in the table. Every row in the same table has the same columns.
- **A primary key**: one or more columns whose value uniquely identifies each row. No two rows in the same table can have the same primary key value.

This structure is rigid by design. Every row in the `Products` table has exactly the same columns as every other row. You cannot add extra columns for some products and not others. This rigidity is what makes SQL queries fast and predictable.

2.2 Relationships Between Tables

The power of the relational model comes from linking tables together. A **foreign key** is a column in one table that refers to the primary key of another table.

Consider CabotTrail's sales data:

- The `Sales.Orders` table has a `CustomerID` column
- `CustomerID` is the primary key of the `Sales.Customers` table
- So `Orders.CustomerID` is a *foreign key* — it links each order to the customer who placed it

This linkage means you do not store the customer's name on every order row. You store the customer's ID. When you want the name, you ask the database to *join* the two tables together.

This is why the relational model is so efficient: information is stored once and referenced everywhere. Changes propagate automatically.

2.3 The Difference Between a Database and a Schema and a Table

These three words are often used loosely, but they have precise meanings:

Term	What it is	CabotTrail example
Database	A named collection of schemas, tables, and other objects, stored in SQL Server	`CabotTrailOutdoorsSales`
Schema	A namespace that groups related tables within a database	`fact`, `dim`
Table	A structured grid of columns and rows within a schema	`fact.Sales`, `dim.Customer`

When you refer to a table fully, you can use three-part notation: `DatabaseName.SchemaName.TableName`. For example: `CabotTrailOutdoorsSales.fact.Sales`.

In practice, once you are working within a specific database (using the `USE` command), you only need `SchemaName.TableName`.

---

3. SQL: The Language of Data

SQL stands for **Structured Query Language**. It is the standard language for interacting with relational databases. Nearly every relational database system — SQL Server, MySQL, PostgreSQL, Oracle, SQLite, Snowflake, BigQuery — understands SQL. The syntax varies in small ways between systems, but the core concepts are universal.

SQL is not a general-purpose programming language like Python or Java. It does not have loops, functions in the traditional sense, or graphical output. What it does, it does with remarkable precision and efficiency: it asks questions of data.

3.1 The Four Categories of SQL

SQL statements are divided into four categories based on what they do:

Category	Abbreviation	Purpose	Key statements
Data Query Language	DQL	Read and retrieve data	`SELECT`
Data Manipulation Language	DML	Add, change, or remove data	`INSERT`, `UPDATE`, `DELETE`
Data Definition Language	DDL	Create or modify database structures	`CREATE`, `ALTER`, `DROP`
Data Control Language	DCL	Manage access and permissions	`GRANT`, `REVOKE`

This course focuses primarily on DQL. The SELECT statement is the tool of the analyst — it lets you ask any question of the data without changing anything. You will touch DML and DDL in later chapters, but the vast majority of your SQL career as an analyst will be writing SELECT statements.

3.2 SQL Is Declarative

Most programming languages are *imperative* — you tell the computer *how* to do something, step by step. SQL is *declarative* — you tell the database *what* you want, and the database decides how to retrieve it.

When you write:

```
SELECT ProductName, UnitPrice FROM dim.Product WHERE CategoryName = 'Sleeping';
```

You are not telling SQL Server how to find these rows — whether to scan the entire table, use an index, or some other method. You are declaring what you want: the product name and unit price for products in the Sleeping category. The database engine figures out the most efficient way to deliver it.

This is both a strength and, occasionally, a source of confusion. Understanding *why* a query returns what it returns requires understanding the relational model — not just the syntax.

3.3 SQL Is Not Case-Sensitive (Mostly)

SQL keywords (`SELECT`, `FROM`, `WHERE`) are not case-sensitive in SQL Server. `select`, `SELECT`, and `Select` all work. Convention is to write keywords in uppercase to distinguish them from table and column names — but this is style, not requirement.

String comparisons *are* case-sensitive in some database configurations. SQL Server's default collation is case-insensitive (`Latin1_General_CI_AS` — CI = Case Insensitive), which means `WHERE CategoryName = 'sleeping'` and `WHERE CategoryName = 'Sleeping'` return the same results. Other databases may behave differently. When in doubt, match the case of the data.

---

4. Connecting to SQL Server with SSMS

SQL Server Management Studio (SSMS) is the graphical tool for working with SQL Server. It provides a visual interface for connecting to databases, writing and running SQL queries, and exploring database objects.

4.1 Opening a Connection

When SSMS opens, a **Connect to Server** dialog appears. Fill in:

- **Server type:** Database Engine
- **Server name:** *(provided by your instructor — typically a server name or* `localhost` *)*
- **Authentication:** Windows Authentication *(uses your Windows login; no separate password required)*

Click **Connect**. If successful, the **Object Explorer** panel on the left populates with a tree showing all available databases.

> ***Troubleshooting:*** *If the connection fails, check that the server name is correct and that SQL Server is running. Your instructor can verify this. A common mistake is leaving the server name blank or typing* `localhost` *when the server is on a different machine.*

4.2 Navigating Object Explorer

The Object Explorer shows the database hierarchy. Expand nodes by clicking the triangle (▶) beside them:

```
[Server name]
  └── Databases
        └── CabotTrailOutdoorsSales
              └── Tables
                    ├── dim.Calendar
                    ├── dim.Customer
                    ├── dim.DeliveryMethod
                    ├── dim.Employee
                    ├── dim.Product
                    └── fact.Sales
```

Right-clicking on any table gives you useful options:
- **Select Top 1000 Rows** — opens a query window pre-populated with a SELECT query
- **Design** — shows the table's column structure, data types, and constraints (read-only in most environments)
- **Properties** — metadata about the table (row count, creation date, etc.)

4.3 The Query Window

The **Query Window** is where you write and run SQL. Open a new one with **File → New → Database Engine Query** or by pressing `Ctrl+N`.

Key controls:
- **Execute** button (▶) or `F5` — runs the query
- **Parse** button or `Ctrl+F5` — checks syntax without running
- **Results pane** — appears below the query window after execution
- **Messages pane** — shows row counts, warnings, and errors

Important: Always set the active database at the top of the query window using `USE`:

```
USE CabotTrailOutdoorsSales;
```

This tells SQL Server which database you are working in. If you forget, queries may fail with "Invalid object name" errors because SQL Server is looking in the wrong database.

---

5. The CabotTrail Outdoor Environment

CabotTrail Outdoor is a fictional outdoor gear retailer based in Nova Scotia, Canada. It sells products across thirteen categories — Shelter, Sleeping, Footwear, Accessories, Navigation, and more — to wholesale and retail customers across Canada.

You will use CabotTrail's data throughout this course and the two courses that follow. By the time you finish DBAS 2103, you will know this business's data better than most of its fictional employees.

5.1 The Learning Environment

The databases are designed to support learning:

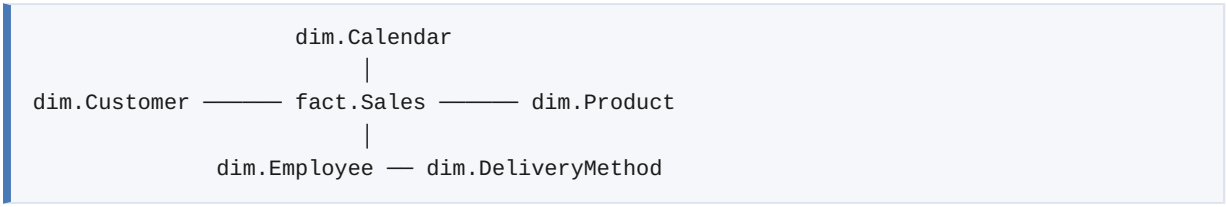
- **Varied data:** Revenue varies meaningfully across products, categories, and territories. Margin rates differ by category. Returns have realistic reasons and refund percentages. This means analytical queries produce interesting, non-uniform results.
- **Realistic size:** 16,359 sales invoice lines across 4+ years. Large enough to see trends; small enough to understand every query result.
- **Progressive complexity:** The Sales datamart has six tables. The full data warehouse has dozens. You will start simple and build up.

5.2 The Sales Data Mart: Your Starting Database

For the first half of this course, you will work in `CabotTrailOutdoorsSales` — the Sales data mart. It has exactly six tables:

Table	What it contains	Rows
`fact.Sales`	One row per product on each customer invoice	16,359
`dim.Calendar`	One row per calendar day	4,017
`dim.Customer`	One row per customer	100
`dim.Product`	One row per product	142
`dim.Employee`	One row per employee	50
`dim.DeliveryMethod`	One row per delivery method	12

This is a **star schema** — a dimensional data model where a central fact table (`fact.Sales`) is surrounded by dimension tables that provide context. The diagram looks like a star:



You will learn why this structure exists, and how to query it effectively, throughout this course. For now, the key insight is: `fact.Sales` contains the measurements (quantities, prices, totals), and the dimension tables contain the descriptions (who, what, when, how).

5.3 Exploring the Tables in SSMS

Before writing any queries, spend a few minutes exploring. Right-click each table and choose **Select Top 1000 Rows**. Observe:

- `dim.Product` is human-readable on its own — you can see product names, categories, and prices immediately
- `dim.Customer` shows customer names, territories, and credit information
- `fact.Sales` is full of numbers and IDs — it is not readable without the dimension tables

This observation is intentional. The fact table stores raw measurements. The dimension tables give those measurements meaning. Together, they answer business questions.

---

6. Schemas: Organising Tables into Groups

You have noticed that every table name in `CabotTrailOutdoorsSales` has a prefix: `fact.` or `dim.`. This prefix is the **schema** — a namespace that groups related tables.

6.1 What a Schema Does

A schema serves two purposes:

Organisation: Tables that serve similar purposes are grouped together. In `CabotTrailOutdoorsSales`, all dimension tables are in the `dim` schema and the fact table is in the `fact` schema. In the full CabotTrail OLTP database (which you will meet in Chapter 9), tables are grouped by business function: `Sales`, `Purchasing`, `Inventory`, `Application`.

Security: Permissions can be granted at the schema level. An analyst might be given read access to the `fact` and `dim` schemas but not to the `ETL` schema (which contains internal pipeline tables).

6.2 Referencing Tables with Schema Names

Always include the schema name when referencing a table:

```
-- Correct: schema-qualified name
SELECT ProductName FROM dim.Product;

-- Risky: unqualified name (SQL Server will look in the default schema, usually dbo)
SELECT ProductName FROM Product;  -- May fail or return the wrong table
```

The habit of schema-qualifying table names prevents subtle bugs, especially in databases with many schemas where the same table name might exist in more than one.

---

7. Your First SELECT Statement

The `SELECT` statement is the foundation of all SQL querying. Everything you do as a data analyst starts with `SELECT`.

7.1 The Basic Structure

```
SELECT  column1, column2, column3
FROM    schema.TableName;
```

Two clauses are required:
- `SELECT` — specifies which columns to return
- `FROM` — specifies which table to read from

The semicolon (`;`) marks the end of a statement. In SQL Server it is technically optional for single statements, but it is good practice to include it — especially when writing scripts with multiple statements.

7.2 Your First Query

```
-- Connect to the Sales data mart
USE CabotTrailOutdoorsSales;

-- Return three columns from the Product dimension
SELECT  ProductID,
        ProductName,
        CategoryName
FROM    dim.Product;
```

Run this query by pressing F5. You should see 142 rows in the Results pane — one for each CabotTrail product.

Reading the result: Each row is one product. The three columns show the product's ID, name, and category. SQL Server returns the rows in no guaranteed order unless you specify one (covered in section 8.2).

7.3 Selecting All Columns with *

The asterisk (`*`) is shorthand for "all columns":

```
SELECT  *
FROM    dim.Product;
```

This returns every column in the table. Try it — you will see many more columns than the three in the previous query.

When to use `*`: For exploration — when you want to see what columns a table has. Not for production queries or submitted work. Reasons:

1. You often do not need all columns. Returning columns you do not use wastes memory and makes results harder to read.
2. If someone adds a column to the table later, your `SELECT *` query silently starts returning it. This can break downstream code that expected a specific column count.
3. It communicates nothing about what you wanted. `SELECT CustomerName, SalesTerritory FROM dim.Customer` is clear; `SELECT * FROM dim.Customer` is vague.

Good practice: Name your columns explicitly. Use `SELECT *` only when exploring a new table for the first time.

7.4 Column Formatting and Readability

SQL Server ignores extra whitespace. These two queries are identical:

```
SELECT ProductID, ProductName, CategoryName FROM dim.Product;
```

```
SELECT  ProductID,
        ProductName,
        CategoryName
FROM    dim.Product;
```

The second is easier to read. Professional SQL aligns keywords and column names vertically, making queries scannable at a glance. When queries grow to 20 or 30 lines, this consistency matters enormously. Adopt the habit now.

7.5 Changing the Active Database with USE

The `USE` statement sets the active database for the current session. Everything after it applies to that database until you `USE` a different one:

```
USE CabotTrailOutdoorsSales;

-- This query runs against CabotTrailOutdoorsSales
SELECT CustomerName FROM dim.Customer;

USE CabotTrailOutdoorDW;

-- This query now runs against CabotTrailOutdoorDW
SELECT CustomerName FROM Dimension.DimCustomer;
```

Alternatively, you can use **three-part naming** to reference a table in any database without switching:

```
-- Three-part name: DatabaseName.SchemaName.TableName
SELECT CustomerName
FROM   CabotTrailOutdoorsSales.dim.Customer;
```

Three-part naming is useful when a query needs data from multiple databases simultaneously. You will use it extensively in later chapters.

---

8. Controlling Output: TOP and ORDER BY

A raw `SELECT` returns all rows in no guaranteed order. Two clauses let you control this: `TOP` limits how many rows are returned, and `ORDER BY` determines their sequence.

8.1 Limiting Rows with TOP

`TOP n` returns only the first *n* rows from the result set. It always works together with `ORDER BY` — otherwise, which rows are "first" is undefined:

```
-- The 5 most expensive products
SELECT  TOP 5
        ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
ORDER BY UnitPrice DESC;
```

`TOP` is useful for:

- **Exploratory queries:** See a sample of the data without loading all rows
- **"Best of" questions:** Top 10 customers by revenue, top 5 products by return rate
- **Debugging:** Check whether a query returns the right structure before running it against millions of rows

8.2 Sorting Results with ORDER BY

`ORDER BY` sorts the result set by one or more columns. It always appears last in the query:

```
-- Products sorted alphabetically by name
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
ORDER BY ProductName ASC;    -- ASC = ascending (A to Z, 0 to 9). This is the default.
```

```
-- Products sorted by price from most to least expensive
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
ORDER BY UnitPrice DESC;     -- DESC = descending (Z to A, highest to lowest)
```

Sorting by multiple columns: List them in priority order, separated by commas:

```
-- Products sorted by category (A to Z), then by price within each category (high to low)
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
ORDER BY CategoryName ASC,
         UnitPrice    DESC;
```

This sorts the entire result set by `CategoryName` first. Where two products share the same category, they are further sorted by `UnitPrice` descending.

Sorting by column position: You can use a column's position number instead of its name:

```
ORDER BY 2 ASC, 3 DESC;  -- Sort by the 2nd column ascending, then 3rd descending
```

This works but is fragile — if you reorder the columns in the SELECT list, the sort changes unexpectedly. Prefer using column names.

8.3 ORDER BY Without TOP

`ORDER BY` without `TOP` sorts all rows in the result:

```
-- All 50 employees sorted alphabetically by full name
SELECT  FullName,
        JobTitle,
        Department
FROM    dim.Employee
ORDER BY FullName ASC;
```

Note on performance: Sorting is an expensive operation on large tables. SQL Server must read all qualifying rows, then sort them, before returning results. For exploratory queries against small tables (like CabotTrail's dimension tables), this is instant. Against large fact tables, an `ORDER BY` without appropriate indexes can be slow. You will return to this topic in Chapter 5.

8.4 TOP With Ties

A subtle issue: what if the 5th and 6th products have exactly the same `UnitPrice`? `TOP 5` returns 5 rows — but which one it excludes between two tied values is arbitrary. `TOP 5 WITH TIES` returns all rows that tie for the last position:

```
-- Top 5 most expensive products — include all products that tie for 5th place
SELECT  TOP 5 WITH TIES
        ProductName,
        UnitPrice
FROM    dim.Product
ORDER BY UnitPrice DESC;
-- May return more than 5 rows if there are ties at the 5th price level
```

---

9. Writing Comments in SQL

Comments are text in your SQL code that the database ignores. They exist for the human reading the code — explaining what a query does, why a decision was made, or what a particular expression means.

9.1 Single-Line Comments

A single-line comment starts with two dashes (`--`). Everything from the `--` to the end of the line is ignored:

```
-- Return all products in the Sleeping category
SELECT  ProductName,
        UnitPrice           -- Price before tax
FROM    dim.Product
WHERE   CategoryName = 'Sleeping';  -- Filter to one category
```

9.2 Multi-Line Comments

A multi-line comment is wrapped in `/* */`. Useful for longer explanations or for temporarily disabling blocks of SQL:

```
/*
   This query answers the business question:
   "What are our most expensive products in each category?"

   Written by: Patrick Dolinger
   Date: 2026-09-10
   Updated: Added UnitPrice column
*/
SELECT  TOP 10
        ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
ORDER BY UnitPrice DESC;
```

9.3 When to Comment

Comments are not optional decoration — they are professional practice. Every submitted lab and production query should have:

1. A brief description of what business question the query answers
2. Comments on any non-obvious expressions or decisions
3. Your name and date for long scripts

A query without a comment says "I wrote this and understood it at the time." A query with good comments says "I wrote this, understood it, and want the next person to understand it too."

---

10. The System Catalog: Asking the Database About Itself

SQL Server stores information about its own structure — table names, column names, data types, constraints — in a set of system views. You can query these views just like regular tables. This is called the **system catalog** or **information schema**.

10.1 Listing Tables in a Database

```
-- What tables exist in the current database?
USE CabotTrailOutdoorsSales;

SELECT  TABLE_SCHEMA,
        TABLE_NAME,
        TABLE_TYPE
FROM    INFORMATION_SCHEMA.TABLES
ORDER BY TABLE_SCHEMA, TABLE_NAME;
```

`INFORMATION_SCHEMA.TABLES` is a system view that returns one row for each table and view in the database. `TABLE_SCHEMA` is the schema name; `TABLE_NAME` is the table name; `TABLE_TYPE` is either 'BASE TABLE' (a regular table) or 'VIEW'.

10.2 Listing Columns in a Table

```
-- What columns does fact.Sales have?
SELECT  COLUMN_NAME,
        DATA_TYPE,
        CHARACTER_MAXIMUM_LENGTH,
        NUMERIC_PRECISION,
        NUMERIC_SCALE,
        IS_NULLABLE
FROM    INFORMATION_SCHEMA.COLUMNS
WHERE   TABLE_SCHEMA = 'fact'
AND     TABLE_NAME   = 'Sales'
ORDER BY ORDINAL_POSITION;
```

This query is enormously useful when you encounter a new table and want to understand its structure without right-clicking through SSMS menus. You will use variations of it throughout this course.

10.3 Why the System Catalog Matters

As an analyst, you will regularly work with databases you did not design and have not seen before. The system catalog lets you explore unfamiliar databases systematically:

- Which tables exist and which schemas they belong to?
- What columns does this table have and what are their data types?
- Which columns allow NULL values (meaning the column might be empty)?

These are the first questions you ask about any new table. The system catalog answers them.

---

11. Chapter Summary

- A **relational database** stores data in structured tables with named columns and typed rows. Related tables are linked through primary and foreign keys — data is stored once and referenced many times.
- Databases offer **scale** (millions of rows), **structure** (rigid schemas), **integrity** (enforced constraints), and **concurrency** (simultaneous access) that spreadsheets cannot match.
- **SQL** (Structured Query Language) is the standard language for interacting with relational databases. It is declarative — you specify what you want, not how to get it. This course focuses on **DQL** (Data Query Language), centred on the `SELECT` statement.
- **SSMS** (SQL Server Management Studio) is the tool for connecting to and querying SQL Server. Use `USE DatabaseName` to set the active database; reference tables as `schema.TableName`.

- The **CabotTrail Outdoor** Sales data mart (`CabotTrailOutdoorsSales`) has six tables. The central `fact.Sales` table holds measurements; the five dimension tables (`dim.Calendar` , `dim.Customer` , `dim.Product` , `dim.Employee` , `dim.DeliveryMethod`) provide context.
- A **SELECT statement** requires at minimum `SELECT` (what columns) and `FROM` (what table). Add `TOP n` to limit rows and `ORDER BY` to sort. Use `ORDER BY` with `ASC` (default) or `DESC` .
- **Comments** (`--` for single-line, `/* */` for multi-line) explain intent. They are professional practice, not optional decoration.
- The **system catalog** (`INFORMATION_SCHEMA.TABLES` , `INFORMATION_SCHEMA.COLUMNS`) lets you query the database about its own structure — essential for exploring unfamiliar databases.

---

12. Review Questions

1. A colleague suggests storing the company's sales data in Excel because "it's simpler." Describe three specific limitations of Excel that would become problems as the sales data grows, and explain how a relational database addresses each.
2. In `CabotTrailOutdoorsSales` , `fact.Sales` contains a `CustomerKey` column rather than `CustomerName` . Explain why the designer made this choice. What advantage does storing a key rather than a name provide?
3. Write a query that returns the `FullName` , `JobTitle` , and `Department` of all employees in `dim.Employee` , sorted alphabetically by `FullName` .
4. Write a query that returns the top 3 most expensive products from `dim.Product` . Include the `ProductName` , `CategoryName` , and `UnitPrice` . What does `TOP 3 WITH TIES` do differently, and when might you prefer it?
5. A developer writes `SELECT * FROM Sales` and gets an error: "Invalid object name 'Sales'." Explain two separate reasons this error might occur and how to fix each.
6. The `INFORMATION_SCHEMA.COLUMNS` query for `fact.Sales` returns a column called `IS_NULLABLE` . Some columns show 'YES' and others show 'NO'. What does this mean, and why might an analyst care whether a column allows NULL values?
7. Write a query that returns all delivery methods from `dim.DeliveryMethod` , sorted by `DeliveryMethodName` alphabetically. Add a comment above the query explaining what business question it answers.

8. Without running it, predict what this query returns and explain your reasoning:

```sql
SELECT TOP 1 ProductName, UnitPrice FROM dim.Product ORDER BY UnitPrice ASC;
```

Deeper Dive

Going Further with the Relational Model

The History of the Relational Model

The relational model was proposed by **E.F. Codd**, a computer scientist at IBM, in a landmark 1970 paper: *"A Relational Model of Data for Large Shared Data Banks."* Published in the *Communications of the ACM*, it described a mathematical framework — based on set theory and predicate logic — for organising data that would become the foundation of every relational database system built since.

Before Codd's paper, databases were *hierarchical* or *network* models — data was navigated through physical pointers, and queries required knowledge of the database's physical layout. Codd's insight was that data should be described by its logical relationships, not its physical storage, and that queries should be declarative (what you want) rather than navigational (how to find it).

SQL, as the language implementation of the relational model, was standardised by ANSI (American National Standards Institute) in 1986 and ISO in 1987. The current standard, **ISO/IEC 9075**, is updated approximately every four years. SQL Server implements a substantial portion of the standard with its own extensions (T-SQL — Transact-SQL).

For the curious: Codd's original 1970 paper is freely available and surprisingly readable for a foundational computer science paper. It is worth reading to understand why the relational model was such a conceptual breakthrough.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

SSMS Keyboard Shortcuts Worth Learning

SSMS is a full-featured IDE with many keyboard shortcuts. These are the ones analysts use most often:

Shortcut	Action
`F5`	Execute the query (or selected text)
`Ctrl+F5`	Parse the query (check syntax without running)
`Ctrl+N`	New query window
`Ctrl+K, Ctrl+C`	Comment out selected lines
`Ctrl+K, Ctrl+U`	Uncomment selected lines
`Ctrl+L`	Display estimated execution plan (not running the query)
`Ctrl+M`	Toggle actual execution plan display
`Ctrl+R`	Show/hide results pane
`Ctrl+Space`	IntelliSense completion (suggest column/table names)
`Alt+F1`	Show object properties for selected object name

Investing 20 minutes learning these shortcuts saves hours over a course. Start with F5, Ctrl+K/C, and Ctrl+K/U — the most used by far.

T-SQL vs Standard SQL

This book uses **T-SQL** (Transact-SQL) — Microsoft's implementation of SQL with extensions specific to SQL Server. Most of what you learn here is standard SQL and will transfer directly to other database systems. A few T-SQL-specific features to be aware of:

Feature	T-SQL syntax	Standard SQL	Notes
`TOP n`	`SELECT TOP 10 ...`	`FETCH FIRST 10 ROWS ONLY`	PostgreSQL/Oracle/MySQL use LIMIT
Date functions	`GETDATE()`	`CURRENT_TIMESTAMP`	Most platforms support CURRENT_TIMESTAMP
String functions	`LEN()`	`LENGTH()`	PostgreSQL, MySQL use LENGTH
Identity columns	`INT IDENTITY(1,1)`	`INT GENERATED ALWAYS AS IDENTITY`	Standard SQL:2003 syntax

When you encounter a different database platform — Snowflake, PostgreSQL, BigQuery — your SQL knowledge transfers directly. You will need to look up syntax differences for a handful of functions, but the SELECT, WHERE, JOIN, and GROUP BY foundations are identical everywhere.

Microsoft's T-SQL documentation is comprehensive and freely available:
Transact-SQL Reference — Microsoft Learn

What Is a Star Schema?

The CabotTrail Sales data mart uses a **star schema** — a specific design pattern for analytical databases developed by Ralph Kimball in the 1990s. It is called a star schema because of its visual appearance: a central fact table surrounded by dimension tables, like the spokes of a star.

Star schemas are designed for *reading*, not writing. The same data exists in multiple places (a customer's name appears on every order, conceptually) to make queries simple and fast. This is the opposite design philosophy from a normalized OLTP database, which stores each piece of information exactly once.

You will encounter star schemas constantly as a data analyst — they power most BI tools, most dashboards, and most analytical databases. Understanding them deeply is one of the most valuable things you can do in this course.

The full explanation of dimensional modelling — why star schemas exist and how to design them — is the subject of the companion textbook *ETL for Business Intelligence*, Chapter 2.

Industry Perspectives

SQL in the Analyst's Toolkit

A 2023 survey of data professionals by Stack Overflow found SQL to be the most widely used language in data science and analytics work — more common than Python, R, or any other tool. This is not because SQL is the most powerful language (Python is far more flexible) but because SQL is where the data *is*.

Data lives in relational databases. Analytical results need to be communicated in terms of that data. SQL is the lingua franca that connects analysts to data, regardless of what BI tool, programming language, or platform sits on top.

Learning SQL well is arguably the highest-return investment a data analyst can make.

SQL Server and the Microsoft Data Platform

SQL Server is one of three dominant relational database platforms alongside Oracle Database and PostgreSQL. In business intelligence environments — particularly in organizations already invested in

Microsoft products — SQL Server is the most common choice. It integrates natively with Excel (via Power Query and Power Pivot), Power BI, and Azure.

Microsoft's cloud-native analytical platform, **Azure Synapse Analytics**, extends SQL Server concepts to cloud-scale data warehousing — the same T-SQL syntax applies to tables with billions of rows. The skills you develop in this course transfer directly to the cloud platform.

SQL Server Technical Documentation

References and Further Reading

1. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. — The foundational paper. Remarkably accessible.
2. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — A rigorous treatment of the relational model for those who want the mathematical foundations.
3. Vieira, R. (2021). *Beginning SQL Server: A Beginner's Guide* (5th ed.). Apress. — Comprehensive beginner-to-intermediate coverage of SQL Server.
4. Microsoft. (2024). *Transact-SQL Reference*. https://learn.microsoft.com/en-us/sql/t-sql/language-reference
5. Microsoft. (2024). *SQL Server Management Studio documentation*. https://learn.microsoft.com/en-us/sql/ssms/sql-server-management-studio-ssms
6. Microsoft. (2024). *INFORMATION_SCHEMA Views*. https://learn.microsoft.com/en-us/sql/relational-databases/system-information-schema-views/system-information-schema-views-transact-sql
7. Stack Overflow. (2023). *Developer Survey 2023 — Most popular technologies*. https://survey.stackoverflow.co/2023/

Chapter 2: Filtering and Sorting — Asking Precise Questions

> ***SQL for Analysts***
> *A practical introduction to querying, transforming, and understanding relational data*
>

Chapter Overview

Chapter 1 taught you how to retrieve data from a table. But retrieving *all* rows is rarely useful. A table with 16,000 rows returns 16,000 rows — and most of the time, you want to answer a specific question about a subset of them.

This chapter is about precision. The `WHERE` clause is the mechanism that turns a broad retrieval into a specific question. Combined with logical operators, pattern matching, range tests, and NULL handling, it gives you the tools to ask exactly the question you mean.

By the end of this chapter you will be able to:

- Write `WHERE` clauses using all standard comparison operators
- Combine multiple conditions using `AND`, `OR`, and `NOT`
- Use brackets to control the order of evaluation in compound conditions
- Filter rows using `IN`, `BETWEEN`, and `LIKE`
- Handle NULL values correctly using `IS NULL` and `IS NOT NULL`
- Remove duplicate rows from results using `DISTINCT`
- Combine filtering and sorting to produce precise, ordered results
- Predict the results of compound conditions without running the query

Table of Contents

1. The WHERE Clause: Filtering Rows

2. Comparison Operators
3. Logical Operators: AND, OR, NOT
4. Operator Precedence and Brackets
5. The IN Operator
6. The BETWEEN Operator
7. Pattern Matching with LIKE
8. NULL Values and IS NULL
9. Removing Duplicates with DISTINCT
10. Putting It Together: WHERE, ORDER BY, and TOP
11. Chapter Summary
12. Review Questions
13. Deeper Dive

---

1. The WHERE Clause: Filtering Rows

The `WHERE` clause specifies a condition that each row must satisfy to be included in the result. Rows that satisfy the condition are returned; rows that do not are excluded.

1.1 Position in the Query

`WHERE` always comes after `FROM` and before `ORDER BY`:

```
SELECT  column1, column2
FROM    schema.TableName
WHERE   condition
ORDER BY column1;
```

This sequence is the **SQL clause order** — a rule, not a convention. Writing clauses in the wrong order produces a syntax error.

1.2 A First Example

```
-- Return only products in the Sleeping category
USE CabotTrailOutdoorsSales;

SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   CategoryName = 'Sleeping';
```

Run this. You should see 9 or 10 rows — only products whose `CategoryName` is exactly 'Sleeping'. The other 130+ products are excluded.

Observation: The string value `'Sleeping'` is wrapped in single quotes. In SQL, text values are always single-quoted. Numbers are not:

```
-- Text value: single quotes required
WHERE   CategoryName = 'Sleeping'

-- Numeric value: no quotes
WHERE   UnitPrice = 249.99
```

Putting quotes around a number (`WHERE UnitPrice = '249.99'`) may still work in SQL Server due to implicit type conversion, but it is imprecise and can cause unexpected behaviour. Match the data type: quotes for text, no quotes for numbers.

1.3 The WHERE Clause Processes Every Row

It helps to picture what SQL Server does when it executes a query with a `WHERE` clause:

1. Open the table (`FROM dim.Product`)
2. Read each row one at a time
3. Evaluate the `WHERE` condition for that row
4. If TRUE → include the row in the result
5. If FALSE or NULL → exclude the row
6. After all rows are processed, apply `ORDER BY` and `TOP` if present

This mental model — the database checking each row against the condition — helps you reason about what a complex `WHERE` clause will do without running it.

---

2. Comparison Operators

A comparison operator compares two values and returns TRUE, FALSE, or (in the case of NULLs) a special value called UNKNOWN. The `WHERE` clause includes rows where the comparison evaluates to TRUE.

2.1 The Standard Comparison Operators

Operator	Meaning	Example
`=`	Equal to	`WHERE CategoryName = 'Sleeping'`
`<>`	Not equal to	`WHERE CategoryName <> 'Sleeping'`
`!=`	Not equal to (same as `<>`)	`WHERE CategoryName != 'Sleeping'`
`>`	Greater than	`WHERE UnitPrice > 100`
`<`	Less than	`WHERE UnitPrice < 50`
`>=`	Greater than or equal to	`WHERE UnitPrice >= 100`
`<=`	Less than or equal to	`WHERE UnitPrice <= 50`

> ***Style note:*** *`<>` is the SQL standard for "not equal to." `!=` works in SQL Server but is not part of the standard. Most experienced SQL developers use `<>`. Either is correct.*

2.2 Comparing Strings

String comparisons in SQL Server are case-insensitive by default (see Deeper Dive for details). This means:

```
-- These three conditions return the same rows
WHERE CategoryName = 'Sleeping'
WHERE CategoryName = 'sleeping'
WHERE CategoryName = 'SLEEPING'
```

String comparisons are also alphabetical — 'A' < 'B', 'Apple' < 'Banana':

```
-- Products whose name comes alphabetically before 'D'
SELECT  ProductName
FROM    dim.Product
WHERE   ProductName < 'D'
ORDER BY ProductName;
```

This returns products whose names start with A, B, or C — 'Cape Breton' comes before 'D', so it is included.

2.3 Comparing Dates

Date comparisons work intuitively — later dates are "greater than" earlier dates:

```
-- Employees hired after January 1, 2023
SELECT  FullName,
        HireDate
FROM    dim.Employee
WHERE   HireDate > '2023-01-01'
ORDER BY HireDate;
```

Date literals in SQL Server are written as strings in quotes, in `'YYYY-MM-DD'` format. SQL Server converts them to the DATE data type automatically. Other formats (`'01/01/2023'`, `'Jan 1 2023'`) also work but are ambiguous across regional settings — always use `'YYYY-MM-DD'` for clarity and reliability.

2.4 Negation with `<>`

`<>` is frequently overlooked as a tool for "everything except." It is often cleaner than listing what you *do* want:

```
-- All categories except Shelter
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   CategoryName <> 'Shelter'
ORDER BY CategoryName, ProductName;
```

This returns products from all 12 other categories — far more concise than listing 12 category names explicitly.

---

3. Logical Operators: AND, OR, NOT

Real analytical questions often have multiple conditions. Logical operators combine them.

3.1 AND: Both Conditions Must Be True

`AND` returns rows where *all* conditions are TRUE:

```
-- Sleeping products that cost more than $200
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   CategoryName = 'Sleeping'
AND     UnitPrice    > 200
ORDER BY UnitPrice DESC;
```

Only rows where `CategoryName = 'Sleeping'` is TRUE *and* `UnitPrice > 200` is TRUE are included. A sleeping bag at $150 is excluded (price condition fails). A tent at $350 is excluded (category condition fails). Only sleeping bags over $200 qualify.

You can chain as many `AND` conditions as needed:

```
-- Active products in the Sleeping category priced between $100 and $300
SELECT  ProductName,
        UnitPrice,
        IsDiscontinued
FROM    dim.Product
WHERE   CategoryName    = 'Sleeping'
AND     UnitPrice       >= 100
AND     UnitPrice       <= 300
AND     IsDiscontinued  = 0
ORDER BY UnitPrice;
```

3.2 OR: At Least One Condition Must Be True

`OR` returns rows where *at least one* condition is TRUE:

```
-- Products in either the Sleeping or Shelter category
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   CategoryName = 'Sleeping'
OR      CategoryName = 'Shelter'
ORDER BY CategoryName, UnitPrice;
```

A product qualifies if its category is 'Sleeping' *or* 'Shelter' (or both — though a product cannot be in both simultaneously since `CategoryName` holds one value per row).

`OR` is more permissive than `AND`. Adding more `OR` conditions expands the result set; adding more `AND` conditions narrows it.

3.3 NOT: Inverting a Condition

`NOT` reverses the TRUE/FALSE result of a condition:

```
-- All products that are NOT discontinued
SELECT  ProductName,
        CategoryName,
        IsDiscontinued
FROM    dim.Product
WHERE   NOT IsDiscontinued = 1
ORDER BY CategoryName, ProductName;
```

This is equivalent to `WHERE IsDiscontinued = 0` — but `NOT` is useful when the condition being negated is complex. It also has specific uses with `IN`, `BETWEEN`, and `LIKE` that you will see shortly.

---

4. Operator Precedence and Brackets

4.1 The Precedence Problem

SQL evaluates `AND` before `OR` — just as multiplication is evaluated before addition in arithmetic. This means:

```
WHERE CategoryName = 'Sleeping' OR CategoryName = 'Shelter' AND UnitPrice > 200
```

is evaluated as:

```
WHERE CategoryName = 'Sleeping' OR (CategoryName = 'Shelter' AND UnitPrice > 200)
```

Not as:

```
WHERE (CategoryName = 'Sleeping' OR CategoryName = 'Shelter') AND UnitPrice > 200
```

The first interpretation returns *all* sleeping products (any price) plus expensive shelter products. The second returns only sleeping and shelter products that cost more than $200. These produce very different results.

4.2 Always Use Brackets to Make Intent Clear

When you mix `AND` and `OR`, use brackets to make the evaluation order explicit — not to rely on precedence rules that you and the next developer may both misremember:

```
-- CLEAR: brackets show exactly what is intended
-- Sleeping OR Shelter, but only if price > $200
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   (CategoryName = 'Sleeping' OR CategoryName = 'Shelter')
AND     UnitPrice > 200
ORDER BY CategoryName, UnitPrice;
```

Compare to the ambiguous version without brackets:

```
-- AMBIGUOUS: easy to misread
WHERE   CategoryName = 'Sleeping'
OR      CategoryName = 'Shelter'
AND     UnitPrice > 200
```

Both SQL Server and human readers will parse these differently. The bracketed version leaves no doubt.

Rule: Whenever you mix `AND` and `OR` in the same `WHERE` clause, use brackets. No exceptions.

4.3 A Precedence Test

Before running this query, predict what it returns:

```
SELECT  ProductName, CategoryName, UnitPrice
FROM    dim.Product
WHERE   CategoryName = 'Sleeping'
OR      CategoryName = 'Accessories'
AND     UnitPrice < 30;
```

Due to `AND` before `OR`, this is:

```
WHERE CategoryName = 'Sleeping'
OR   (CategoryName = 'Accessories' AND UnitPrice < 30)
```

It returns *all* sleeping products (regardless of price) plus accessories under $30. Is that what you wanted?

Probably not — you likely wanted sleeping or accessories products, both under $30. The corrected query:

```
WHERE   (CategoryName = 'Sleeping' OR CategoryName = 'Accessories')
AND     UnitPrice < 30;
```

Run both versions and compare the row counts. The difference will be instructive.

---

5. The IN Operator

5.1 What IN Does

`IN` tests whether a value matches any value in a list. It is shorthand for multiple `OR` conditions on the same column:

```
-- Without IN: verbose
WHERE   CategoryName = 'Sleeping'
OR      CategoryName = 'Shelter'
OR      CategoryName = 'Footwear'
OR      CategoryName = 'Apparel - Tops'

-- With IN: concise
WHERE   CategoryName IN ('Sleeping', 'Shelter', 'Footwear', 'Apparel - Tops')
```

Both produce identical results. `IN` is not faster — it compiles to the same execution plan. But it is dramatically more readable when the list has more than two values.

5.2 IN with a List of Values

```
-- Customers in Atlantic provinces
SELECT  CustomerName,
        ProvinceCode,
        SalesTerritory
FROM    dim.Customer
WHERE   ProvinceCode IN ('NS', 'NB', 'PE', 'NL')
ORDER BY ProvinceCode, CustomerName;
```

```
-- Products in high-margin categories
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   CategoryName IN (
    'Accessories',
    'Hydration',
    'Navigation',
    'Safety and First Aid'
)
ORDER BY CategoryName, UnitPrice DESC;
```

The list can be written on one line or spread across multiple lines. When lists are long, spreading across lines with consistent indentation makes them scannable.

5.3 NOT IN

`NOT IN` returns rows where the value does *not* match any value in the list:

```
-- Products NOT in gear-intensive categories
SELECT  ProductName,
        CategoryName
FROM    dim.Product
WHERE   CategoryName NOT IN ('Climbing and Rappelling', 'Paddling', 'Winter Sports')
ORDER BY CategoryName, ProductName;
```

> ***Important warning about NOT IN and NULLs:*** *`NOT IN` behaves unexpectedly when the list contains a NULL value. If any value in the list is NULL, `NOT IN` returns no rows — because `value <> NULL` evaluates to UNKNOWN, not FALSE. This is a subtle but significant trap. When using `NOT IN` with a subquery that might return NULLs, use `NOT EXISTS` instead (covered in Chapter 7).*

---

6. The BETWEEN Operator

6.1 What BETWEEN Does

`BETWEEN` tests whether a value falls within a range, *inclusive of both endpoints*:

```
-- Products priced between $50 and $150
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   UnitPrice BETWEEN 50 AND 150
ORDER BY UnitPrice;
```

This includes products at exactly $50, at exactly $150, and everything in between. It is equivalent to:

```
WHERE UnitPrice >= 50 AND UnitPrice <= 150
```

The `BETWEEN` version is more readable for range conditions.

6.2 BETWEEN with Dates

`BETWEEN` works on dates as naturally as on numbers:

```
-- Calendar dates in Q3 2024 (July, August, September)
SELECT  FullDate,
        MonthName,
        DayName,
        IsWeekend
FROM    dim.Calendar
WHERE   FullDate BETWEEN '2024-07-01' AND '2024-09-30'
ORDER BY FullDate;
```

> ***Inclusive endpoints matter:*** *`'2024-09-30'` includes September 30th. If the column were a DATETIME rather than a DATE, `'2024-09-30'` would be interpreted as `2024-09-30 00:00:00` — which would exclude transactions that occurred on September 30th after midnight. For DATETIME columns, use `< '2024-10-01'` rather than `<= '2024-09-30'` to avoid this trap. Since `dim.Calendar.FullDate` is a DATE column in CabotTrail, `BETWEEN` works correctly here.*

6.3 NOT BETWEEN

`NOT BETWEEN` returns rows outside the range:

```
-- Products outside the mid-range price band
SELECT  ProductName,
        CategoryName,
        UnitPrice,
        CASE
            WHEN UnitPrice < 50  THEN 'Budget'
            WHEN UnitPrice > 300 THEN 'Premium'
        END AS PriceBand
FROM    dim.Product
WHERE   UnitPrice NOT BETWEEN 50 AND 300
ORDER BY UnitPrice;
```

This returns products under $50 or over $300 — the extremes of the price range.

---

7. Pattern Matching with LIKE

7.1 What LIKE Does

`LIKE` tests whether a string value matches a pattern. It is the tool for "contains," "starts with," and "ends with" searches that exact equality (`=`) cannot express.

Two wildcard characters build the pattern:

Wildcard	Matches	Example
`%`	Any sequence of zero or more characters	`'%Trail%'` matches 'Cabot Trail Tent', 'TrailBlazer', 'On the Trail'
`_`	Exactly one character	`'CT_001'` matches 'CT-001', 'CT0001', 'CTA001'

7.2 Common LIKE Patterns

Starts with:

```
-- Products whose name starts with 'Cabot'
SELECT  ProductName, CategoryName
FROM    dim.Product
WHERE   ProductName LIKE 'Cabot%'
ORDER BY ProductName;
```

Ends with:

```
-- Products whose name ends with 'Kit'
SELECT  ProductName, CategoryName
FROM    dim.Product
WHERE   ProductName LIKE '%Kit'
ORDER BY ProductName;
```

Contains:

```
-- Products that contain 'Trail' anywhere in the name
SELECT  ProductName, CategoryName, UnitPrice
FROM    dim.Product
WHERE   ProductName LIKE '%Trail%'
ORDER BY ProductName;
```

Exactly one character wildcard:

```
-- Product codes matching a specific pattern: two letters, a dash, three digits
SELECT  ProductCode, ProductName
FROM    dim.Product
WHERE   ProductCode LIKE '__-___'
ORDER BY ProductCode;
-- Two underscores = two any characters, dash is literal, three underscores = three any
chars
```

7.3 NOT LIKE

`NOT LIKE` returns rows where the pattern does not match:

```
-- Products whose name does NOT contain 'Cabot'
SELECT  ProductName, CategoryName
FROM    dim.Product
WHERE   ProductName NOT LIKE '%Cabot%'
ORDER BY ProductName;
```

7.4 LIKE Performance Considerations

`LIKE 'Cabot%'` (pattern anchored at the start) can use a database index efficiently — SQL Server knows where to start looking alphabetically. `LIKE '%Trail%'` (pattern with a leading `%`) cannot use an index — every row must be examined. On small dimension tables, this is imperceptible. On large fact tables with millions of rows, leading wildcards can be very slow.

For analytical queries against the CabotTrail dimension tables (142 products, 100 customers), LIKE performance is not a concern. Keep this limitation in mind as you work with larger datasets.

---

8. NULL Values and IS NULL

8.1 What NULL Means

`NULL` is SQL's way of representing "no value" or "unknown." It is not zero, it is not an empty string, it is not the word "null" — it is the complete absence of a value.

NULL appears in columns where information is optional or simply not available. In CabotTrail's `dim.Product` table, the `ProductCode` column is nullable — some products have a code, others do not:

```
-- See which products have no product code
SELECT  ProductName,
        ProductCode,
        CategoryName
FROM    dim.Product
WHERE   ProductCode IS NULL
ORDER BY CategoryName, ProductName;
```

8.2 Why You Cannot Use = NULL

The most common NULL mistake is trying to compare with `=`:

```
-- WRONG: this returns no rows in SQL Server
WHERE ProductCode = NULL

-- CORRECT: use IS NULL
WHERE ProductCode IS NULL
```

The reason: NULL represents an unknown value. Comparing an unknown to anything — even another NULL — produces UNKNOWN, not TRUE. `NULL = NULL` is UNKNOWN, not TRUE. SQL Server therefore correctly excludes all rows from `WHERE ProductCode = NULL`.

This is counterintuitive but consistent with the mathematical semantics of NULL as "unknown." It is one of SQL's most important rules to internalize early.

8.3 IS NULL and IS NOT NULL

```
-- Products WITH a product code
SELECT  ProductName,
        ProductCode,
        CategoryName
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
ORDER BY ProductCode;
```

```
-- Employees with no email address on record
SELECT  FullName,
        JobTitle,
        EmailAddress
FROM    dim.Employee
WHERE   EmailAddress IS NULL
ORDER BY FullName;
```

8.4 NULL in Comparisons and Calculations

NULL propagates through comparisons and calculations:

```
-- Any comparison involving NULL returns UNKNOWN
NULL = 5        → UNKNOWN
NULL <> 5       → UNKNOWN
NULL > 5        → UNKNOWN
NULL = NULL     → UNKNOWN

-- Any arithmetic involving NULL returns NULL
NULL + 5        → NULL
NULL * 100      → NULL
10 / NULL       → NULL
```

This means: if a column has NULL values, any `WHERE` clause that compares that column with `=`, `<>`, `>`, `<`, etc. will silently exclude the NULL rows — because the comparison produces UNKNOWN, not TRUE.

```
-- This does NOT return rows where UnitPrice is NULL
WHERE UnitPrice > 0

-- To also catch NULL prices, you need:
WHERE UnitPrice > 0 OR UnitPrice IS NULL
```

Remembering that NULL propagates through comparisons prevents a whole category of subtle bugs.

---

9. Removing Duplicates with DISTINCT

9.1 What DISTINCT Does

`DISTINCT` removes duplicate rows from the result set. Rows are considered duplicate if every column in the SELECT list has the same value.

```
-- What distinct categories exist in the product table?
SELECT  DISTINCT CategoryName
FROM    dim.Product
ORDER BY CategoryName;
```

Without `DISTINCT`, this returns 142 rows — one per product, with many repeated category names. With `DISTINCT`, it returns 13 rows — one per unique category.

9.2 DISTINCT on Multiple Columns

When applied to multiple columns, `DISTINCT` removes rows where the *combination* of all selected columns is identical:

```
-- What distinct province/territory combinations exist in the customer table?
SELECT  DISTINCT
        ProvinceCode,
        SalesTerritory
FROM    dim.Customer
ORDER BY SalesTerritory, ProvinceCode;
```

This returns one row per unique (ProvinceCode, SalesTerritory) pair — not one per unique province alone or territory alone.

9.3 DISTINCT vs GROUP BY

`DISTINCT` and `GROUP BY` both eliminate duplicates. For simple cases (no aggregation needed), they produce the same result:

```
-- These return identical results
SELECT DISTINCT CategoryName FROM dim.Product ORDER BY CategoryName;
SELECT CategoryName FROM dim.Product GROUP BY CategoryName ORDER BY CategoryName;
```

`GROUP BY` is more powerful — it allows aggregation (counting how many products are in each category, for example). `DISTINCT` is simpler when you only need the unique values and no aggregation. Chapter 6 covers `GROUP BY` in depth.

9.4 When DISTINCT Is the Wrong Tool

`DISTINCT` is sometimes used to "fix" a query that is returning unexpected duplicates. If your query produces duplicate rows that you did not expect, the duplicates are usually a symptom of an incorrect `JOIN` — a join condition that matches more rows than you intended. Adding `DISTINCT` hides the symptom without fixing the cause. It is better to understand why the duplicates exist and fix the query logic.

You will encounter this issue in Chapter 5 when learning JOINs. For now, the takeaway: use `DISTINCT` intentionally, not as a band-aid.

---

10. Putting It Together: WHERE, ORDER BY, and TOP

All the filtering tools covered in this chapter combine naturally with the `ORDER BY` and `TOP` from Chapter 1. The key is remembering the clause order:

```
SELECT   [TOP n] column1, column2
FROM     schema.TableName
WHERE    condition
ORDER BY column1 [ASC|DESC];
```

10.1 Business Questions Answered

Let's apply everything from this chapter to real analytical questions about CabotTrail.

Q: What are the ten least expensive active products, and what categories are they in?

```
SELECT  TOP 10
        ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY UnitPrice ASC;
```

Q: Which customers in Ontario or British Columbia have a credit limit above $5,000?

```
SELECT  CustomerName,
        ProvinceCode,
        SalesTerritory,
        CreditLimit
FROM    dim.Customer
WHERE   ProvinceCode IN ('ON', 'BC')
AND     CreditLimit > 5000
ORDER BY CreditLimit DESC;
```

Q: Which calendar dates in 2024 are public holidays that fall on a weekday?

```
SELECT  FullDate,
        DayName,
        HolidayName,
        IsWeekend
FROM    dim.Calendar
WHERE   CalendarYear    = 2024
AND     IsHoliday       = 1
AND     IsWeekend       = 0
ORDER BY FullDate;
```

Q: Which products have 'Cabot' in their name and cost between $100 and $500?

```
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   ProductName LIKE '%Cabot%'
AND     UnitPrice BETWEEN 100 AND 500
ORDER BY UnitPrice;
```

Q: Which employees have a NULL email address or work in the 'Operations' department?

```
SELECT  FullName,
        Department,
        EmailAddress
FROM    dim.Employee
WHERE   EmailAddress IS NULL
OR      Department = 'Operations'
ORDER BY Department, FullName;
```

10.2 Reading Queries Backwards

A useful technique for understanding complex `WHERE` clauses: read them from the result backward to the conditions.

Given:

```
SELECT  ProductName, CategoryName, UnitPrice
FROM    dim.Product
WHERE   (CategoryName IN ('Sleeping', 'Shelter') OR UnitPrice > 400)
AND     IsDiscontinued = 0
ORDER BY UnitPrice DESC;
```

Reading backward: "I want active products (IsDiscontinued = 0) that are either in Sleeping/Shelter OR cost more than $400."

This mental habit helps you predict results without running a query and catch logic errors before they happen.

---

11. Chapter Summary

- The `WHERE` clause filters rows by specifying conditions. Rows where the condition evaluates to TRUE are returned; FALSE and UNKNOWN rows are excluded.
- **Comparison operators** (`=`, `<>`, `>`, `<`, `>=`, `<=`) compare column values to literals. Strings use single quotes; numbers and NULLs do not.
- **AND** returns rows where all conditions are TRUE. **OR** returns rows where at least one condition is TRUE. **NOT** reverses a condition. `AND` has higher precedence than `OR`.
- **Always use brackets** when mixing `AND` and `OR`. Without them, queries are ambiguous and easy to misread.
- **IN** is shorthand for multiple `OR` conditions on the same column. **NOT IN** excludes matching values (but use with caution when NULLs may be present).
- **BETWEEN** tests inclusive ranges. For DATETIME columns, prefer `>= start AND < end_plus_one_day` to avoid boundary issues.
- **LIKE** matches string patterns using `%` (any characters) and `_` (exactly one character). Leading `%` prevents index use on large tables.
- **NULL** means "no value" — it is not zero or empty string. Use `IS NULL` and `IS NOT NULL`, never `= NULL`. NULL propagates through comparisons and arithmetic, producing UNKNOWN or NULL respectively.
- **DISTINCT** removes duplicate rows where all selected columns are identical. Use it intentionally; do not use it to mask JOIN errors.

---

12. Review Questions

1. Explain why `WHERE ProductCode = NULL` returns no rows in SQL Server. What should you write instead, and what is the mathematical reason the equals operator does not work with NULL?
2. Predict what the following query returns *without running it*, then verify by running it. Explain the difference between the intended question and what the query actually asks:
   `sql SELECT ProductName, CategoryName, UnitPrice FROM dim.Product WHERE CategoryName = 'Sleeping' OR CategoryName = 'Footwear' AND UnitPrice > 150;`
3. Write a query that returns all customers from Nova Scotia (`ProvinceCode = 'NS'`) or New Brunswick (`ProvinceCode = 'NB'`) who have a credit limit between $2,000 and $8,000 (inclusive). Include their name, province code, sales territory, and credit limit. Sort by credit limit descending.
4. Write a query that returns all products where `ProductCode` is NOT NULL and the product name contains the word 'Trail'. Include the product code, product name, and category. Sort alphabetically by product name.
5. The following query is meant to return "products in the Sleeping or Shelter category that cost less than $200." Identify the logic error, explain what the query actually returns, and rewrite it correctly:
   `sql SELECT ProductName, CategoryName, UnitPrice FROM dim.Product WHERE CategoryName = 'Sleeping' OR CategoryName = 'Shelter' AND UnitPrice < 200;`
6. Write a query against `dim.Calendar` that returns all dates in the first quarter of fiscal year 2025 (`FiscalYearNumber = 2025` and `FiscalQuarterNumber = 1`). Include `FullDate` , `DayName` , `MonthName` , and `IsHoliday` . Sort chronologically.
7. Write a query that returns all employees whose `Department` is NOT IN the list `('IT', 'Legal', 'Executive')` . Include their full name, job title, and department. Sort by department, then by full name.
8. A colleague writes this query to find products that have either no product code or a price under $25:
   `sql SELECT ProductName, ProductCode, UnitPrice FROM dim.Product WHERE ProductCode = NULL OR UnitPrice < 25;`
   The query runs without error but the results are wrong — products with NULL product codes are missing. Explain the bug and write the corrected query.

Deeper Dive

Going Further with Filtering and Sorting

The Three-Valued Logic of SQL

SQL does not use simple true/false (Boolean) logic. It uses **three-valued logic** with TRUE, FALSE, and UNKNOWN. UNKNOWN arises whenever NULL is involved in a comparison.

The rules of three-valued logic:

Expression	Result
`TRUE AND UNKNOWN`	UNKNOWN
`FALSE AND UNKNOWN`	FALSE
`TRUE OR UNKNOWN`	TRUE
`FALSE OR UNKNOWN`	UNKNOWN
`NOT UNKNOWN`	UNKNOWN

These rules have practical consequences. `FALSE AND UNKNOWN` is FALSE (because AND requires both to be TRUE — if one is FALSE, it does not matter what the other is). `TRUE OR UNKNOWN` is TRUE (because OR needs only one TRUE). But `TRUE AND UNKNOWN` is UNKNOWN — and rows with UNKNOWN conditions are excluded from results.

This is why NULL comparisons are subtle: they can produce UNKNOWN in ways that look like they should produce TRUE, silently excluding rows from your result set.

The `WHERE` clause includes only TRUE rows. Both FALSE and UNKNOWN rows are excluded. This asymmetry between FALSE and UNKNOWN matters for `NOT` — `NOT FALSE = TRUE`, but `NOT UNKNOWN = UNKNOWN`. Both exclude the row, but for different reasons.

Understanding three-valued logic explains many surprising SQL behaviours. For a thorough treatment: Date, C. J. (2011). *SQL and Relational Theory: How to Write Accurate SQL Code* (2nd ed.). O'Reilly Media. — Chapter 4 covers NULL and three-valued logic in depth.

Case Sensitivity and Collations

SQL Server's default collation (`SQL_Latin1_General_CP1_CI_AS`) is **case-insensitive** (`CI`). String comparisons treat 'A' and 'a' as equal. This is convenient for most business queries but means you cannot distinguish uppercase from lowercase.

If you need case-sensitive comparison, you can force it using the `COLLATE` clause:

```
-- Case-sensitive comparison (returns only exact uppercase match)
SELECT ProductName
FROM   dim.Product
WHERE  ProductName COLLATE Latin1_General_CS_AS = 'Cape Breton Sleeping Bag';
-- CS = Case Sensitive
```

Collations also affect sort order. `CI_AS` sorts case-insensitively: 'apple', 'Apple', and 'APPLE' sort together. A case-sensitive collation would sort them separately.

Database administrators set the collation at the server, database, or column level. As an analyst, you usually work within the default — but knowing collations exist explains why sometimes a query on a different database seems to behave differently with string comparisons.

Microsoft documentation on collations:
Collation and Unicode support — SQL Server

LIKE Escape Characters

What if you need to search for a literal `%` or `_` character in a string? These are wildcards in LIKE patterns — how do you match them literally?

Use the `ESCAPE` clause to define an escape character:

```
-- Find product codes containing a literal underscore
SELECT ProductCode, ProductName
FROM   dim.Product
WHERE  ProductCode LIKE '%!_%' ESCAPE '!';
-- The ! before _ tells SQL Server: the next character is literal, not a wildcard
```

In this example, `!` is the escape character. `!_` means "a literal underscore." Any character can be the escape character — `!`, `\`, or `~` are common choices since they rarely appear in data.

This is an edge case in most analytical work — but if you are ever querying financial data, product codes, or file paths that contain underscores or percent signs, you will need it.

Regular Expressions in Other Databases

SQL Server's `LIKE` operator is limited to `%` and `_` wildcards. Other database systems — PostgreSQL, MySQL, BigQuery, Snowflake — support **regular expressions** for pattern matching, which are far more powerful:

```
-- PostgreSQL: find products matching a regex pattern
WHERE ProductName ~ '^Cabot.*Bag$'
-- Matches: starts with 'Cabot', ends with 'Bag'

-- SQL Server equivalent (no native regex):
WHERE ProductName LIKE 'Cabot%Bag'
-- Less precise: does not require 'Bag' to be at the very end
```

Regular expressions can match complex patterns — digits, letters, specific character classes, exact repetitions — that `LIKE` cannot express. If you work with PostgreSQL or cloud databases in future roles, learning regular expression syntax opens powerful filtering capabilities.

Sargability: Writing WHERE Clauses That Use Indexes

The term **sargable** (Search ARGument ABLE) describes a `WHERE` condition that can use a database index efficiently. Non-sargable conditions force SQL Server to scan every row in the table.

Sargable patterns (fast on large tables):

```
WHERE OrderDate >= '2024-01-01'                  -- direct column comparison
WHERE CustomerName = 'Fundy Bay Outfitters'      -- direct column comparison
WHERE UnitPrice BETWEEN 50 AND 150               -- range on column directly
WHERE ProductCode LIKE 'CT%'                     -- anchored-start LIKE
```

Non-sargable patterns (slow on large tables):

```
WHERE YEAR(OrderDate) = 2024                     -- function on column prevents index
WHERE UPPER(CustomerName) = 'FUNDY BAY'         -- function on column
WHERE ProductName LIKE '%Trail%'                 -- leading % prevents index
WHERE UnitPrice * 1.15 > 200                     -- expression on column
```

The general rule: **apply functions to the literal, not the column.** Instead of `WHERE YEAR(OrderDate) = 2024`, use `WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'`. The first requires SQL Server to compute `YEAR()` for every row; the second lets it find qualifying rows directly using an index.

For the CabotTrail dimension tables (100–142 rows), sargability is irrelevant — the tables are tiny. For large fact tables and in production work, writing sargable conditions is a fundamental performance skill.

Industry Perspectives

The WHERE Clause as a Business Rule

Every `WHERE` clause you write encodes a business rule. `WHERE IsDiscontinued = 0` encodes "we only analyse active products." `WHERE CalendarYear >= 2022` encodes "we only look at data from 2022

onward." `WHERE ProvinceCode IN ('NS', 'NB', 'PE', 'NL')` encodes "Atlantic Canada includes these four provinces."

Business rules encoded in SQL queries are invisible unless documented. The next analyst who uses your query will not know *why* those filters were applied — was it a technical requirement, a business decision, or a mistake?

Good commenting practice (from Chapter 1) is essential here: document not just *what* a filter does but *why* it exists. "Filter to active products per business requirement dated 2024-03-01" is far more informative than no comment at all.

NULL in Practice: The Most Common Source of Subtle Bugs

In surveys of experienced SQL developers, NULL-related bugs consistently rank as the most common source of query errors that produce wrong results without errors being raised. The query runs; the results look plausible; but they are missing rows due to NULL propagation.

The most common manifestations:
- `WHERE column = NULL` (should be `IS NULL`)
- Forgetting that `NOT IN` with NULLs returns nothing
- Arithmetic on nullable columns producing NULL totals
- Aggregations (`SUM`, `AVG`) silently ignoring NULL rows

The discipline of checking for NULLs before writing `WHERE` conditions — using `INFORMATION_SCHEMA.COLUMNS` to see which columns are nullable — is a professional habit that prevents this class of error.

References and Further Reading

1. Date, C. J. (2011). *SQL and Relational Theory: How to Write Accurate SQL Code* (2nd ed.). O'Reilly Media. — Chapter 4 covers NULL and three-valued logic more rigorously than any other introductory text.
2. Microsoft. (2024). *WHERE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/where-transact-sql
3. Microsoft. (2024). *LIKE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/like-transact-sql
4. Microsoft. (2024). *NULL and UNKNOWN (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/null-and-unknown-transact-sql

5. Microsoft. (2024). *Collation and Unicode support*. https://learn.microsoft.com/en-us/sql/relational-databases/collations/collation-and-unicode-support

6. Fritchey, G. (2018). *SQL Server Query Performance Tuning* (5th ed.). Apress. — Chapter 3 covers sargability and index-friendly WHERE clauses in depth.

7. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 2 provides an excellent treatment of filtering, NULL handling, and three-valued logic for SQL Server specifically.

Chapter 3: Data Types, Math, and Calculated Columns

> ***SQL for Analysts***
> *A practical introduction to querying, transforming, and understanding relational data*
>

Chapter Overview

The first two chapters focused on which rows to return. This chapter focuses on what you do with the values in those rows once you have them. Specifically: what kind of values SQL Server stores, how to do arithmetic on them, how to create new calculated columns, and how to name those columns meaningfully.

You will also meet the introductory concepts of DML — the SQL statements that modify data. While this course focuses on reading data, understanding INSERT, UPDATE, and DELETE rounds out your picture of how SQL works and prepares you for the source-to-target work in Chapter 9.

By the end of this chapter you will be able to:

- Identify and describe the major SQL Server data types and explain why choosing the right one matters
- Perform arithmetic operations on numeric columns in a SELECT statement
- Explain integer division and avoid its most common pitfall
- Use the ROUND function to control decimal precision
- Create calculated columns using the AS alias keyword
- Query the system catalog to discover data types for any table
- Write INSERT, UPDATE, and DELETE statements safely
- Apply the "test with SELECT before modifying" discipline for DML

Table of Contents

1. Why Data Types Matter

2. Numeric Data Types
3. Text Data Types
4. Date and Time Data Types
5. Other Data Types
6. Discovering Data Types with the System Catalog
7. Arithmetic in SQL
8. Integer Division: The Silent Precision Thief
9. The ROUND Function
10. Column Aliases with AS
11. Introduction to DML: INSERT, UPDATE, DELETE
12. Chapter Summary
13. Review Questions
14. Deeper Dive

---

1. Why Data Types Matter

Every column in every table has a **data type** — a declaration of what kind of values it stores. Data types are not just labels; they determine:

- **What values are valid:** A `DATE` column cannot store the text 'hello'. An `INT` column cannot store 3.14.
- **How much storage a value uses:** An `INT` uses 4 bytes; a `BIGINT` uses 8. At millions of rows, this difference adds up.
- **What operations make sense:** You can add two `INT` values. You can concatenate two `NVARCHAR` values. Mixing them without conversion produces an error.
- **How comparisons work:** `'10' > '9'` is FALSE in text comparison (alphabetically, '1' < '9'). `10 > 9` is TRUE in numeric comparison. The same values, interpreted differently.

Choosing the right data type at design time is an act of precision and care. Choosing the wrong one — storing a price as `NVARCHAR` instead of `DECIMAL`, for example — creates bugs that are tedious to fix after data exists.

As an analyst, you do not usually design tables yourself (that comes in DBAS 2010). But you need to understand data types because they affect every calculation, comparison, and function you apply in a query.

---

2. Numeric Data Types

SQL Server has two families of numeric types: **exact** and **approximate**.

2.1 Exact Numeric Types

Exact numeric types store values precisely — no rounding, no approximation. Use these for any data where precision matters: prices, quantities, counts, financial amounts.

Whole number types:

Type	Storage	Range	Typical use
`TINYINT`	1 byte	0 to 255	Very small counts (e.g., number of items in an order line)
`SMALLINT`	2 bytes	−32,768 to 32,767	Small codes and counts
`INT`	4 bytes	−2.1 billion to +2.1 billion	Most integer uses — the default choice
`BIGINT`	8 bytes	−9.2 quintillion to +9.2 quintillion	Row counts, financial systems, large IDs

Choosing between them: Use `INT` by default for integers. Use `BIGINT` only when values genuinely exceed 2.1 billion. Use `TINYINT` and `SMALLINT` only in tables with millions of rows where storage savings justify the reduced range. The most expensive bug is choosing a type too small and running out of range after data accumulates for years.

Decimal types:

Type	Description	Syntax	Typical use
`DECIMAL(p,s)` or `NUMERIC(p,s)`	Exact decimal with p total digits and s decimal places	`DECIMAL(18,2)`	Prices, financial amounts, rates
`MONEY`	Fixed-precision currency, 4 decimal places	`MONEY`	Legacy currency; prefer DECIMAL for new designs
`SMALLMONEY`	Like MONEY but smaller range	`SMALLMONEY`	Legacy; avoid

`DECIMAL(p,s)` is the workhorse. `p` is **precision** — the total number of significant digits. `s` is **scale** — the number of digits after the decimal point. So `DECIMAL(18,2)` stores up to 16 digits before the decimal and exactly 2 after: up to 9,999,999,999,999,999.99.

```
-- Data types in CabotTrail: examine dim.Product
USE CabotTrailOutdoorsSales;

SELECT  COLUMN_NAME,
        DATA_TYPE,
        NUMERIC_PRECISION,
        NUMERIC_SCALE
FROM    INFORMATION_SCHEMA.COLUMNS
WHERE   TABLE_SCHEMA    = 'dim'
AND     TABLE_NAME      = 'Product'
AND     DATA_TYPE IN ('decimal', 'numeric', 'int', 'tinyint', 'smallint', 'bigint')
ORDER BY ORDINAL_POSITION;
```

Run this. You will see that `UnitPrice` and `RecommendedRetailPrice` are `DECIMAL(18,2)` — storing prices to exactly two decimal places. `ProductID` is `INT`. `IsDiscontinued` is `BIT` (covered in section 5.1).

2.2 Approximate Numeric Types

Approximate types store numbers as floating-point representations. They can represent a vastly wider range of values than exact types, but at the cost of precision — stored values are close to the original, not identical.

Type	Storage	Approximate precision
`FLOAT`	4 or 8 bytes	~7 or ~15 significant digits
`REAL`	4 bytes	~7 significant digits

When to use approximate types: Scientific calculations, statistics, machine learning features — contexts where you need a huge range of values and can accept slight precision loss.

Never use for financial data: `FLOAT` arithmetic produces rounding errors that accumulate unpredictably:

```
-- The floating-point surprise
SELECT CAST(0.1 AS FLOAT) + CAST(0.2 AS FLOAT);
-- Returns: 0.30000000000000004 (approximately)
-- Not 0.3. This is not a SQL Server bug — it is fundamental to floating-point
arithmetic.

SELECT CAST(0.1 AS DECIMAL(18,2)) + CAST(0.2 AS DECIMAL(18,2));
-- Returns: 0.30 (exact)
```

If you are ever asked to sum financial values and the totals are slightly off, check whether the column is `FLOAT` rather than `DECIMAL`. This is a surprisingly common legacy database issue.

---

3. Text Data Types

Text types store character strings. SQL Server has two families: Unicode and non-Unicode.

3.1 Unicode vs Non-Unicode

Unicode stores characters from all writing systems in the world — Latin, Cyrillic, Chinese, Arabic, emoji. Every character uses 2 bytes of storage.

Non-Unicode stores characters from a single code page — typically Western European Latin characters. Each character uses 1 byte.

Type	Unicode	Variable/Fixed length	Max size
`NVARCHAR(n)`	Yes	Variable	Up to 4,000 characters (or MAX for up to 2 GB)
`NCHAR(n)`	Yes	Fixed	Up to 4,000 characters
`VARCHAR(n)`	No	Variable	Up to 8,000 characters (or MAX)
`CHAR(n)`	No	Fixed	Up to 8,000 characters

The practical rule: Use `NVARCHAR` for any column that stores human-readable text. The storage cost (2 bytes per character vs 1) is negligible for names, addresses, and descriptions — and using `NVARCHAR` guarantees that international characters, accented letters, and extended punctuation are stored correctly.

Use `VARCHAR` only when you are certain the content will never contain non-Latin characters and storage is a critical concern (a column storing billions of short ASCII codes, for example).

3.2 Variable vs Fixed Length

Variable length (`NVARCHAR` , `VARCHAR`) stores only the actual characters in each value. 'Halifax' in a `NVARCHAR(100)` column uses 14 bytes (7 chars × 2), not 200 bytes (100 chars × 2).

Fixed length (`NCHAR` , `CHAR`) pads shorter values with spaces to fill the declared length. 'NS' in a `CHAR(10)` column uses 10 bytes. Fixed-length types are useful for codes that are always the same length (province codes, ISO country codes, fixed-format identifiers) where the padding is an intentional design choice.

In CabotTrail, `ProvinceCode` is `NCHAR(2)` — always exactly 2 characters. `CustomerName` is `NVARCHAR(100)` — up to 100 characters, using only what is needed.

3.3 Text Literals in SQL

String literals in SQL are always enclosed in **single quotes**. Never double quotes for data values (double quotes have a different meaning in SQL — they quote identifiers like column names):

```
-- Correct string literals
WHERE CategoryName = 'Sleeping'
WHERE ProvinceCode = 'NS'

-- Wrong: double quotes are for identifiers, not values
WHERE CategoryName = "Sleeping"   -- May cause an error or unexpected behaviour
```

To include a single quote within a string literal, double it:

```
-- Searching for a name containing an apostrophe
WHERE CustomerName LIKE 'O''Brien%'
-- The '' inside the string is a single literal apostrophe
```

---

4. Date and Time Data Types

Dates and times have their own family of types in SQL Server. Choosing the right one avoids storage waste and prevents subtle time-zone or precision bugs.

4.1 The Date and Time Types

Type	What it stores	Storage	Precision	Example
`DATE`	Date only (no time)	3 bytes	Day	`2024-03-15`
`TIME(n)`	Time only (no date)	3–5 bytes	100ns (n=7)	`14:30:00.000`
`DATETIME`	Date and time	8 bytes	~3ms	`2024-03-15 14:30:00.000`
`DATETIME2(n)`	Date and time, higher precision	6–8 bytes	100ns	`2024-03-15 14:30:00.0000000`
`SMALLDATETIME`	Date and time	4 bytes	1 minute	`2024-03-15 14:30:00`
`DATETIMEOFFSET`	Date, time, and UTC offset	8–10 bytes	100ns	`2024-03-15 14:30:00 -04:00`

The practical choices:

- **DATE:** Use for dates with no time component — order dates, birth dates, calendar dates. Saves 5 bytes vs DATETIME, avoids time-related filtering bugs. CabotTrail's `dim.Calendar.FullDate` is a `DATE`.
- **DATETIME2:** Use for timestamps where you need date and time — log entries, transaction times, event records. More precise and slightly smaller than `DATETIME`.
- **DATETIME:** Legacy type, widely used in older SQL Server databases. Still works but `DATETIME2` is preferred for new designs.
- **DATETIMEOFFSET:** Use when working across time zones — global applications, cloud systems.

4.2 Date Literals

Date literals are written as strings in `'YYYY-MM-DD'` format:

```
WHERE OrderDate  = '2024-03-15'       -- DATE comparison
WHERE InvoiceDate > '2024-01-01'      -- Date range
WHERE HireDate BETWEEN '2022-01-01' AND '2023-12-31'
```

SQL Server converts the string to the appropriate date type automatically. The `'YYYY-MM-DD'` format is unambiguous regardless of regional settings — always use it.

4.3 Dates in the CabotTrail Databases

In `CabotTrailOutdoorsSales`, date information is stored as an integer key in `fact.Sales`:

```
-- The OrderDateKey in fact.Sales is an INT, not a DATE
SELECT  TOP 5
        OrderDateKey,       -- e.g., 20240315
        InvoiceDateKey,
        DueDateKey
FROM    fact.Sales;
```

The integer `20240315` represents March 15, 2024 in YYYYMMDD format. You join to `dim.Calendar` to get the actual date attributes:

```
-- Get full date information by joining to dim.Calendar
SELECT  TOP 5
        cal.FullDate,       -- Actual DATE value
        cal.MonthName,
        cal.CalendarYear,
        fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey;
```

You will learn JOIN syntax formally in Chapter 5. For now, observe that `dim.Calendar.DateKey` is the integer bridge between the date key in the fact table and the rich date attributes in the calendar dimension.

---

5. Other Data Types

5.1 BIT: Boolean Values

`BIT` stores 1 or 0 — representing TRUE or FALSE, yes or no, active or inactive:

```
-- BIT columns in dim.Product
SELECT  ProductName,
        IsDiscontinued      -- BIT: 0 = active, 1 = discontinued
FROM    dim.Product
WHERE   IsDiscontinued = 1  -- Filter for discontinued products
ORDER BY ProductName;
```

Note: `BIT` does not use TRUE and FALSE keywords in comparisons — use `1` and `0`. Also, SQL Server compacts multiple `BIT` columns into the same byte of storage (up to 8 BIT columns share 1 byte), making them very efficient.

5.2 UNIQUEIDENTIFIER: GUIDs

`UNIQUEIDENTIFIER` stores a 16-byte globally unique identifier (GUID) — a value like `{6F9619FF-8B86-D011-B42D-00C04FC964FF}`. GUIDs are used as primary keys in distributed systems where auto-incrementing integers would create conflicts across servers.

In analytical work, you will occasionally encounter GUID columns as foreign keys referencing systems that use them as PKs. They are not human-friendly — a GUID tells you nothing about the row it identifies — but you treat them like any other key column in queries.

5.3 XML and JSON

SQL Server supports `XML` as a native data type with its own query language (XQuery). More recently, JSON values are commonly stored in `NVARCHAR` columns and queried using JSON functions (`JSON_VALUE`, `JSON_QUERY`, `OPENJSON`).

You will not encounter XML or JSON in the CabotTrail environment, but knowing they exist explains some columns you may see in enterprise systems — a column called `AdditionalProperties` or `Metadata` that contains structured text is often XML or JSON.

---

6. Discovering Data Types with the System Catalog

Before working with any table, it is good practice to inspect its data types. The system catalog query from Chapter 1 extended with numeric and character detail:

```
-- Complete column inspection for any table
USE CabotTrailOutdoorsSales;

SELECT
    c.COLUMN_NAME,
    -- Build a readable data type description
    c.DATA_TYPE +
    CASE
        WHEN c.DATA_TYPE IN ('nvarchar','varchar','nchar','char')
            AND c.CHARACTER_MAXIMUM_LENGTH = -1
        THEN '(MAX)'
        WHEN c.DATA_TYPE IN ('nvarchar','varchar','nchar','char')
        THEN '(' + CAST(c.CHARACTER_MAXIMUM_LENGTH AS VARCHAR) + ')'
        WHEN c.DATA_TYPE IN ('decimal','numeric')
        THEN '(' + CAST(c.NUMERIC_PRECISION AS VARCHAR)
                 + ',' + CAST(c.NUMERIC_SCALE AS VARCHAR) + ')'
        ELSE ''
    END                     AS DataType,
    c.IS_NULLABLE           AS Nullable,
    c.COLUMN_DEFAULT        AS DefaultValue,
    c.ORDINAL_POSITION      AS Position
FROM    INFORMATION_SCHEMA.COLUMNS c
WHERE   c.TABLE_SCHEMA  = 'fact'
AND     c.TABLE_NAME    = 'Sales'
ORDER BY c.ORDINAL_POSITION;
```

Run this for `fact.Sales`. The output shows every column with its full data type — `DECIMAL(18,2)` for prices, `INT` for quantities, `DATE` for order dates. This is your first step whenever you encounter a table you have not seen before.

---

7. Arithmetic in SQL

SQL can perform arithmetic directly inside a `SELECT` statement. The result is a new computed value, calculated for each row.

7.1 The Arithmetic Operators

Operator	Operation	Example
`+`	Addition	`UnitPrice + ShippingCost`
`-`	Subtraction	`LineTotal - TaxAmount`
`*`	Multiplication	`UnitPrice * Quantity`
`/`	Division	`GrossProfit / LineTotal`
`%`	Modulo (remainder)	`OrderID % 100`

7.2 Arithmetic in the SELECT List

Computed expressions appear directly in the SELECT list:

```
-- Calculate a 15% markup on every product price
SELECT  ProductName,
        CategoryName,
        UnitPrice,
        UnitPrice * 1.15            AS MarkupPrice
FROM    dim.Product
ORDER BY CategoryName, UnitPrice;
```

```
-- Calculate a 10% discount
SELECT  ProductName,
        UnitPrice,
        UnitPrice * 0.90            AS DiscountedPrice,
        UnitPrice - (UnitPrice * 0.90) AS DiscountAmount
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY UnitPrice DESC;
```

7.3 Working with the Fact Table

The `fact.Sales` table already has pre-computed columns — `LineTotal`, `GrossProfit`, `UnitCost` — but you can still build new calculations on top of them:

```
-- Verify gross profit margin from stored columns
SELECT  TOP 10
        InvoiceID,
        LineTotal,
        UnitCost,
        GrossProfit,
        -- Manual calculation to verify
        LineTotal - UnitCost                            AS CalculatedProfit,
        -- Margin as a percentage
        ROUND((LineTotal - UnitCost) / LineTotal * 100, 2) AS MarginPct
FROM    fact.Sales
WHERE   LineTotal > 0
ORDER BY LineTotal DESC;
```

Notice `ROUND` wrapping the margin calculation — covered in section 9.

7.4 Arithmetic in WHERE Clauses

Expressions can appear in `WHERE` conditions as well:

```
-- Products where markup over cost exceeds $100
-- (RecommendedRetailPrice minus UnitPrice > 100)
SELECT  ProductName,
        UnitPrice,
        RecommendedRetailPrice,
        RecommendedRetailPrice - UnitPrice  AS MarkupOverCost
FROM    dim.Product
WHERE   RecommendedRetailPrice - UnitPrice > 100
ORDER BY MarkupOverCost DESC;
```

> ***Performance note:*** *When you apply an expression to a column in a WHERE clause (e.g., `WHERE UnitPrice * 1.15 > 200`), the expression must be evaluated for every row — SQL Server cannot use an index on the raw `UnitPrice` column to speed this up. For large tables, rewrite as a direct comparison: `WHERE UnitPrice > 200 / 1.15`. The result is the same; the performance is far better. (You saw this concept as "sargability" in Chapter 2's Deeper Dive.)*

---

8. Integer Division: The Silent Precision Thief

This section covers one of the most important and frequently misunderstood behaviours in SQL arithmetic.

8.1 The Problem

When you divide two integers in SQL Server, the result is an **integer** — the decimal portion is simply discarded, with no rounding:

```
SELECT 5 / 2;    -- Returns: 2   (not 2.5)
SELECT 7 / 3;    -- Returns: 2   (not 2.333...)
SELECT 1 / 4;    -- Returns: 0   (not 0.25)
```

This is not a bug. It is the defined behaviour for integer division. The problem arises when you do not realize your columns are integers and you write a division that should return a decimal:

```
-- WRONG: if both columns are INT, this returns 0 for all rows where
-- GrossProfit < LineTotal (which is most rows)
SELECT  GrossProfit / LineTotal AS Margin
FROM    fact.Sales;

-- RIGHT: cast at least one operand to DECIMAL
SELECT  CAST(GrossProfit AS DECIMAL(18,4)) / LineTotal AS Margin
FROM    fact.Sales;
```

The silent nature of this bug makes it dangerous — the query runs without an error, the results look plausible (0.0 might seem like "zero margin"), but the data is wrong.

8.2 Checking for Integer Division

Before writing division, check the data types of both columns:

```
SELECT  COLUMN_NAME, DATA_TYPE
FROM    INFORMATION_SCHEMA.COLUMNS
WHERE   TABLE_SCHEMA = 'fact'
AND     TABLE_NAME   = 'Sales'
AND     COLUMN_NAME IN ('GrossProfit', 'LineTotal')
ORDER BY ORDINAL_POSITION;
```

If both columns are `DECIMAL`, you are safe. If either is `INT`, you need to cast.

8.3 Solving Integer Division

Three solutions, in order of preference:

Option 1 — CAST one operand to DECIMAL:

```
CAST(numerator AS DECIMAL(18,4)) / denominator
```

Option 2 — Multiply by 1.0 to force decimal arithmetic:

```
numerator * 1.0 / denominator
-- Multiplying by 1.0 (a decimal literal) converts the integer to float
-- Less precise than CAST but quick
```

Option 3 — Multiply by a decimal literal:

```
numerator * 100.0 / denominator   -- for a percentage calculation
-- The decimal literal 100.0 forces decimal arithmetic
```

For analytical work, `CAST` is the most explicit and readable. Always prefer it.

8.4 Avoiding Division by Zero

Dividing by zero causes an error in SQL Server:

```
SELECT 100 / 0;    -- Error: Divide by zero error encountered
```

In data that may have zero values in the denominator (a product with zero sales, a territory with zero customers), you need to guard against this:

```
-- NULLIF: returns NULL when the second argument equals the first
-- NULL / anything = NULL (no error)
SELECT  CASE WHEN LineTotal = 0 THEN 0
             ELSE ROUND(GrossProfit / LineTotal * 100, 2)
        END AS MarginPct
FROM    fact.Sales;

-- Or more concisely using NULLIF:
SELECT  ROUND(GrossProfit / NULLIF(LineTotal, 0) * 100, 2) AS MarginPct
FROM    fact.Sales;
-- NULLIF(LineTotal, 0) returns NULL when LineTotal = 0
-- GrossProfit / NULL = NULL (no error, NULL result)
```

`NULLIF(expression, value)` returns NULL when the expression equals the value — otherwise returns the expression unchanged. It is the standard SQL guard against divide-by-zero.

---

9. The ROUND Function

`ROUND` controls how many decimal places appear in a numeric result.

9.1 Syntax

```
ROUND(numeric_expression, decimal_places)
```

`decimal_places` specifies how many digits to keep after the decimal point:

```
SELECT  ROUND(249.9875, 2);   -- Returns: 249.99  (rounds to 2 decimal places)
SELECT  ROUND(249.9875, 1);   -- Returns: 250.0   (rounds to 1 decimal place)
SELECT  ROUND(249.9875, 0);   -- Returns: 250.0   (rounds to nearest whole number)
SELECT  ROUND(249.9875, -1);  -- Returns: 250.0   (rounds to nearest 10)
SELECT  ROUND(249.9875, -2);  -- Returns: 200.0   (rounds to nearest 100)
```

Negative values for `decimal_places` round to the left of the decimal point — unusual but occasionally useful for summarising large numbers.

9.2 Using ROUND in Practice

```
-- Calculated margin percentage, rounded to 2 decimal places
SELECT  TOP 10
        InvoiceID,
        LineTotal,
        GrossProfit,
        ROUND(
            CAST(GrossProfit AS DECIMAL(18,4)) / LineTotal * 100,
        2)                          AS MarginPct
FROM    fact.Sales
WHERE   LineTotal > 0
ORDER BY LineTotal DESC;
```

```
-- Product pricing analysis: original, discounted, rounded
SELECT  ProductName,
        UnitPrice                                   AS OriginalPrice,
        ROUND(UnitPrice * 0.85, 2)                  AS SalePrice15Pct,
        ROUND(UnitPrice * 0.70, 2)                  AS ClearancePrice30Pct
FROM    dim.Product
WHERE   IsDiscontinued = 1   -- Only apply to discontinued products
ORDER BY UnitPrice DESC;
```

9.3 ROUND vs Storing Fewer Decimal Places

`ROUND` in a SELECT does not change the stored value — it only affects the displayed result. The underlying data in the table is unchanged. If you need to store a rounded value permanently, you would use `UPDATE` (covered in section 11.3) or round during an `INSERT`.

9.4 CEILING and FLOOR

Two related functions:

- `CEILING(x)` returns the smallest integer *greater than or equal to* x — always rounds up
- `FLOOR(x)` returns the largest integer *less than or equal to* x — always rounds down

```
SELECT  CEILING(4.1);    -- Returns: 5
SELECT  FLOOR(4.9);      -- Returns: 4
SELECT  CEILING(4.0);    -- Returns: 4  (already an integer)
SELECT  FLOOR(-4.1);     -- Returns: -5 (rounds toward negative infinity)
```

These are used when rounding direction matters — calculating the number of boxes needed to ship a quantity (always round up), or calculating completed full months (always round down).

---

10. Column Aliases with AS

10.1 What an Alias Does

When you compute an expression in a SELECT list, SQL Server displays it with the raw expression text as the column header — not helpful:

```
SELECT UnitPrice * 1.15 FROM dim.Product;
-- Column header: (No column name)  or the expression text
```

An **alias** gives the computed column a meaningful name using the `AS` keyword:

```
SELECT  UnitPrice * 1.15   AS PriceWithMarkup
FROM    dim.Product;
-- Column header: PriceWithMarkup
```

10.2 AS on Any Column

Aliases work on any column expression, not just computed ones. You can rename an existing column in the output:

```
SELECT
    ProductName             AS [Product],
    CategoryName            AS [Category],
    UnitPrice               AS [Unit Price],
    UnitPrice * 1.15        AS [Price with 15% Markup],
    ROUND(UnitPrice * 0.90, 2) AS [10% Discount Price]
FROM    dim.Product
ORDER BY UnitPrice DESC;
```

Square brackets `[ ]` around an alias allow spaces and special characters. Without brackets, aliases cannot contain spaces. Convention: use brackets for aliases with spaces; use no brackets for single-word aliases.

10.3 Aliases in ORDER BY

Column aliases can be used in `ORDER BY` — SQL Server resolves them:

```
SELECT
    ProductName,
    UnitPrice * 1.15    AS MarkupPrice
FROM    dim.Product
ORDER BY MarkupPrice DESC;   -- Can reference the alias in ORDER BY
```

However, you **cannot** use a column alias in a `WHERE` clause — the WHERE clause is evaluated before the SELECT list, so the alias does not yet exist:

```
-- WRONG: alias not yet defined when WHERE runs
SELECT  ProductName, UnitPrice * 1.15 AS MarkupPrice
FROM    dim.Product
WHERE   MarkupPrice > 200;    -- Error: Invalid column name 'MarkupPrice'

-- RIGHT: repeat the expression in WHERE
SELECT  ProductName, UnitPrice * 1.15 AS MarkupPrice
FROM    dim.Product
WHERE   UnitPrice * 1.15 > 200;
```

This is one of the more counterintuitive aspects of SQL's evaluation order. The rule: aliases can only be referenced in `ORDER BY` — not in `WHERE`, `GROUP BY`, or `HAVING`.

10.4 Meaningful Alias Names

A good alias communicates what the value represents and its unit:

```
-- Uninformative aliases
SELECT UnitPrice * 0.85 AS Calc1,
       UnitPrice * 0.70 AS Calc2

-- Meaningful aliases
SELECT UnitPrice * 0.85 AS SalePrice15PctOff,
       UnitPrice * 0.70 AS ClearancePrice30PctOff
```

For submitted lab work and production queries, aliases must be descriptive. A query with columns named `Calc1` and `Calc2` communicates nothing to the next person who reads it.

---

11. Introduction to DML: INSERT, UPDATE, DELETE

So far, every query in this book has been read-only — a `SELECT` statement that retrieves data without changing anything. DML (Data Manipulation Language) statements modify data. Understanding them rounds out your SQL picture and prepares you for the source-to-target work in Chapter 9.

> ***Important:*** *DML statements change the database permanently (unless wrapped in a transaction that is rolled back). For learning purposes, your instructor will provide a dedicated practice table. Never run DML statements against dimension or fact tables in the CabotTrail environment without explicit instruction.*

11.1 The Practice Table

Your instructor will create this table for the exercises that follow:

```
-- Instructor runs this once to create the practice table
USE CabotTrailOutdoorsSales;

CREATE TABLE dbo.ProductPriceLog
(
    LogID           INT IDENTITY(1,1)   PRIMARY KEY,
    ProductID       INT                 NOT NULL,
    OldPrice        DECIMAL(18,2)       NOT NULL,
    NewPrice        DECIMAL(18,2)       NOT NULL,
    ChangedBy       NVARCHAR(100)       NOT NULL,
    ChangeDate      DATE                NOT NULL DEFAULT GETDATE()
);
```

This table records price changes — a simple but realistic log structure.

11.2 INSERT: Adding New Rows

`INSERT INTO` adds one or more rows to a table:

```
-- Insert a single row
INSERT INTO dbo.ProductPriceLog
    (ProductID, OldPrice, NewPrice, ChangedBy)
VALUES
    (83, 249.99, 259.99, 'Your Name Here');

-- Verify the insert
SELECT * FROM dbo.ProductPriceLog;
```

Key rules:

- The column list `(ProductID, OldPrice, NewPrice, ChangedBy)` and the values list `VALUES (...)`

must have the same number of entries in the same order

- `LogID` is omitted because it is `IDENTITY` — SQL Server generates it automatically
- `ChangeDate` is omitted because it has a `DEFAULT` — today's date is used automatically

11.3 UPDATE: Modifying Existing Rows

`UPDATE` changes values in existing rows:

```
-- ALWAYS write and verify the WHERE clause with SELECT first
SELECT  LogID, NewPrice, ChangedBy
FROM    dbo.ProductPriceLog
WHERE   LogID = 1;

-- Only then run the UPDATE
UPDATE  dbo.ProductPriceLog
SET     NewPrice    = 264.99,
        ChangedBy   = 'Your Name Here — Corrected'
WHERE   LogID = 1;

-- Verify the change
SELECT * FROM dbo.ProductPriceLog WHERE LogID = 1;
```

The golden rule of UPDATE: Always write a `SELECT` with the same `WHERE` clause first. Verify that it returns exactly the rows you intend to update. Then run the `UPDATE`. The consequences of a missing or wrong `WHERE` clause are:

```
-- This updates EVERY row in the table — usually catastrophic
UPDATE dbo.ProductPriceLog
SET    ChangedBy = 'Mistake';
-- No WHERE clause = all 1,000 rows updated
```

SQL Server does not ask for confirmation before executing this. Once it runs, the damage is done.

11.4 DELETE: Removing Rows

`DELETE FROM` removes rows from a table:

```
-- ALWAYS verify first
SELECT *
FROM   dbo.ProductPriceLog
WHERE  LogID = 1;

-- Only then delete
DELETE FROM dbo.ProductPriceLog
WHERE  LogID = 1;

-- Verify deletion
SELECT * FROM dbo.ProductPriceLog;
```

The same golden rule applies: A `DELETE` without a `WHERE` clause removes every row in the table:

```
-- This deletes EVERY row — effectively empties the table
DELETE FROM dbo.ProductPriceLog;
-- No WHERE clause = all rows deleted
```

11.5 The SELECT-Before-DML Discipline

Every professional SQL developer follows this discipline without exception:

1. Write the `WHERE` clause you intend to use in your DML statement
2. Run it as a `SELECT` first and confirm the rows returned are exactly the ones you want to affect
3. Then change `SELECT column1, column2` to `UPDATE table SET ...` or `DELETE FROM table` — keeping the `WHERE` clause identical

This single habit prevents the most destructive class of SQL mistakes.

```
-- Step 1: Identify the rows
SELECT LogID, ProductID, NewPrice
FROM   dbo.ProductPriceLog
WHERE  ProductID = 83
AND    ChangeDate = '2026-09-22';
-- Shows: LogID 2, ProductID 83, NewPrice 259.99

-- Step 2: Same WHERE clause, now as DELETE
DELETE FROM dbo.ProductPriceLog
WHERE  ProductID = 83
AND    ChangeDate = '2026-09-22';
-- Deletes exactly the rows identified in Step 1
```

---

12. Chapter Summary

- **Data types** define what values a column stores, how much space they use, and what operations are valid. Choose the smallest type that can represent all valid values correctly.
- **Numeric types:** `INT` for whole numbers by default; `DECIMAL(p,s)` for exact decimals (prices, financial amounts); never `FLOAT` for financial data.
- **Text types:** `NVARCHAR(n)` for variable-length text — Unicode, stores only what is used. `NCHAR(n)` for fixed-length codes.
- **Date types:** `DATE` for dates with no time; `DATETIME2` for timestamps; avoid `DATETIME` in new designs.
- **Arithmetic** is written directly in the SELECT list. Addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`), modulo (`%`).
- **Integer division** silently truncates the decimal portion. If both operands are integers, `5 / 2 = 2`, not `2.5`. Use `CAST(numerator AS DECIMAL(18,4))` to force decimal arithmetic.
- **NULLIF(expression, value)** returns NULL when the expression equals the value — the standard guard against divide-by-zero errors.
- **ROUND(expression, n)** rounds to `n` decimal places. Negative `n` rounds to the left of the decimal. `CEILING` always rounds up; `FLOOR` always rounds down.
- **AS** assigns a meaningful name to any column or computed expression. Aliases can be used in `ORDER BY` but not in `WHERE`.
- **DML:** `INSERT INTO ... VALUES (...)` adds rows. `UPDATE ... SET ... WHERE` modifies rows. `DELETE FROM ... WHERE` removes rows. Always test with SELECT first before running UPDATE or DELETE.

---

13. Review Questions

1. A developer stores product prices in a `FLOAT` column rather than `DECIMAL(18,2)`. The prices are summed in a report and the total is $0.003 off from the expected value. Explain the root cause and describe the correct data type to use.
2. Predict the result of `SELECT 9 / 4` in SQL Server. Explain why the result is what it is, and write the corrected expression that returns a decimal result.

3. Write a query against `dim.Product` that returns `ProductName`, `UnitPrice`, and three calculated columns:
   - `BudgetPrice`: the unit price reduced by 20%, rounded to 2 decimal places
   - `PremiumPrice`: the unit price increased by 35%, rounded to 2 decimal places
   - `PriceRange`: the difference between `PremiumPrice` and `BudgetPrice`, rounded to 2 decimal places
   Sort by `UnitPrice` descending.
4. Using the `INFORMATION_SCHEMA.COLUMNS` query, inspect the data types of `GrossProfit` and `LineTotal` in `fact.Sales`. Based on what you find, explain whether the following expression is safe or not: `GrossProfit / LineTotal * 100`
5. Write a query that uses `NULLIF` to calculate `GrossProfitMarginPct` from `GrossProfit` and `LineTotal` in `fact.Sales`, guarding against divide-by-zero. Round the result to 2 decimal places. Filter to only rows where `LineTotal > 0` and limit to the top 10 rows ordered by `LineTotal` descending.
6. Explain why the following query produces an error and rewrite it correctly:
   `sql SELECT ProductName, UnitPrice * 1.20 AS MarkedUpPrice FROM dim.Product WHERE MarkedUpPrice > 300;`
7. Write the three steps of the SELECT-before-DML discipline as they would apply to this scenario: you want to delete all rows from `dbo.ProductPriceLog` where `ChangedBy = 'Test User'`. Show each step as executable SQL.
8. A calendar date column is stored as `NVARCHAR(10)` in a legacy system, holding values like `'2024-03-15'`. List two specific problems this causes for analytical queries, and explain how you would handle each.

Deeper Dive

Going Further with Data Types, Math, and Calculations

Why DECIMAL, Not MONEY

SQL Server's `MONEY` type seems convenient for financial data — it has a fixed 4-decimal-place precision and a descriptive name. However, experienced database designers consistently recommend `DECIMAL(18,2)` or `DECIMAL(19,4)` instead. The reasons:

MONEY is not standard SQL: It is a SQL Server extension. Code using `MONEY` does not port cleanly to PostgreSQL, Oracle, or other platforms.

MONEY arithmetic has precision quirks: Division of `MONEY` values is done in `MONEY` arithmetic, which can introduce rounding errors in intermediate calculations that `DECIMAL` does not have.

MONEY has a fixed scale of 4: If your business requires more or fewer decimal places, `DECIMAL` gives you explicit control.

The SQL Server documentation acknowledges these limitations:
money and smallmoney (Transact-SQL)

Implicit vs Explicit Type Conversion

SQL Server performs **implicit conversions** — automatically converting one type to another when needed for a comparison or calculation. For example, comparing an `INT` to a `DECIMAL` works without explicit casting because SQL Server promotes the `INT` to `DECIMAL` automatically.

The conversion hierarchy matters: SQL Server always converts toward the "higher" type in the precedence order (FLOAT > DECIMAL > INT > SMALLINT > TINYINT). An `INT` divided by a `DECIMAL` produces a `DECIMAL` result — no integer truncation.

Implicit conversion risks:
1. **Performance:** Converting a column implicitly in a WHERE clause prevents index use (the sargability problem from Chapter 2)
2. **Unexpected results:** Implicit conversions can produce results you did not anticipate, especially when text and numbers mix

Explicit conversion with CAST and CONVERT:

```
-- CAST: standard SQL syntax
CAST(expression AS data_type)
CAST(UnitPrice AS INT)          -- Truncates decimal, no rounding
CAST('2024-03-15' AS DATE)     -- String to date
CAST(42 AS NVARCHAR(10))       -- Integer to text

-- CONVERT: SQL Server specific, with optional style for dates
CONVERT(data_type, expression [, style])
CONVERT(NVARCHAR(10), GETDATE(), 120)  -- Date to 'YYYY-MM-DD' string
```

`CAST` is preferred for portability. `CONVERT` is useful when you need SQL Server's date formatting styles.

Microsoft documentation:
CAST and CONVERT (Transact-SQL)

Numeric Precision and the IEEE 754 Standard

The floating-point behaviour described in section 2.2 — where `0.1 + 0.2 ≠ 0.3` — is not a SQL Server bug. It is a consequence of the **IEEE 754** standard for floating-point arithmetic, used by virtually every programming language and computer system.

Floating-point numbers are stored in binary (base 2). Most decimal fractions — including 0.1 and 0.2 — cannot be represented exactly in binary, just as 1/3 cannot be represented exactly in decimal. The stored value is a very close approximation. When you add two approximations, the tiny errors accumulate.

For financial calculations, this is never acceptable. `DECIMAL` arithmetic uses a different internal representation (fixed-point binary-coded decimal) that guarantees exact results for values within its precision range.

This is one of the most important practical distinctions in database design: `FLOAT` for science and statistics, `DECIMAL` for money and measurements.

Transactions: Protecting DML Operations

Section 11 introduced `INSERT`, `UPDATE`, and `DELETE` without mentioning transactions. A **transaction** groups multiple DML statements into a single atomic unit — either all succeed or all roll back:

```
BEGIN TRANSACTION;

    UPDATE dbo.ProductPriceLog SET NewPrice = 264.99 WHERE LogID = 1;
    UPDATE dbo.ProductPriceLog SET NewPrice = 319.99 WHERE LogID = 2;

    -- Check if both look correct before committing
    SELECT * FROM dbo.ProductPriceLog WHERE LogID IN (1, 2);

COMMIT;     -- Both changes are permanently saved
-- or
ROLLBACK;   -- Both changes are reversed (nothing was saved)
```

A transaction wrapped in `BEGIN TRANSACTION` / `ROLLBACK` is the safest way to test destructive DML operations. You can see exactly what changed, and if anything looks wrong, `ROLLBACK` undoes all of it.

In production systems, transactions also provide **ACID guarantees** — Atomicity, Consistency, Isolation, and Durability. These ensure that partial failures (a server crash mid-UPDATE) leave the database in a consistent state.

Microsoft documentation:
Transactions (Transact-SQL)

The SQL Evaluation Order

Chapter 10's observation that aliases cannot be used in WHERE clauses is a consequence of SQL's internal evaluation order. Understanding it explains many "why doesn't this work?" moments:

Step	Clause	What happens
1	`FROM`	Table(s) are identified; JOINs are applied
2	`WHERE`	Rows are filtered; aliases do not exist yet
3	`GROUP BY`	Rows are grouped
4	`HAVING`	Groups are filtered
5	`SELECT`	Columns and expressions are evaluated; aliases are assigned
6	`DISTINCT`	Duplicate rows are removed
7	`ORDER BY`	Results are sorted; aliases are now available
8	`TOP` / `LIMIT`	Row count is limited

This order explains:
- Why `WHERE` cannot reference a `SELECT` alias (step 2 precedes step 5)
- Why `HAVING` can reference `GROUP BY` columns but not `SELECT` aliases
- Why `ORDER BY` can use aliases (step 7 follows step 5)
- Why `TOP` is applied after sorting (step 8 follows step 7)

Industry Perspectives

Data Type Mistakes That Cost Real Money

Poor data type choices are a surprisingly common source of production bugs in enterprise systems. Some real-world patterns (anonymized):

The FLOAT price bug: A financial system stored transaction amounts as `FLOAT`. Overnight batch reconciliations showed discrepancies of a few cents — always. After months of investigation, the root cause was floating-point accumulation in the summation. Fixing it required migrating terabytes of data to `DECIMAL`. The lesson: always use `DECIMAL` for money.

The NVARCHAR date bug: A legacy system stored dates as `NVARCHAR(10)` in the format `'DD/MM/YYYY'`. A new analyst wrote `WHERE TransactionDate > '2024-01-01'` and got wrong results —

alphabetically, `'15/03/2024'` sorts before `'01/01/2024'` because '1' < '2' in string comparison. The fix required casting every value. The lesson: always use a proper DATE type for dates.

The INT overflow bug: A retail system used `INT` for a transaction counter that should have used `BIGINT`. After 2.1 billion transactions, the counter rolled over to negative values and started overwriting old records. The lesson: size data types for the maximum plausible value, not the current value.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 2 covers expressions, operators, and data types with depth and clarity.
2. Microsoft. (2024). *Data types (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql
3. Microsoft. (2024). *CAST and CONVERT (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/cast-and-convert-transact-sql
4. Microsoft. (2024). *ROUND (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/round-transact-sql
5. Microsoft. (2024). *Transactions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/transactions-transact-sql
6. Microsoft. (2024). *money and smallmoney (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/data-types/money-and-smallmoney-transact-sql
7. Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48. — The authoritative treatment of IEEE 754 floating-point arithmetic. Technical but freely available. https://dl.acm.org/doi/10.1145/103162.103163

Chapter 4: Functions — String, Date, and NULL Handling

Chapter Overview

The previous three chapters established how to retrieve rows, filter them, and perform arithmetic on their values. This chapter expands your toolkit with **functions** — pre-built operations that transform individual values.

SQL Server has hundreds of built-in functions. This chapter covers the three families you will use most often as an analyst: string functions (for working with text), date functions (for working with dates and times), and NULL-handling functions (for gracefully managing missing values). The chapter closes with `CASE` expressions — SQL's conditional logic that ties everything together.

By the end of this chapter you will be able to:

- Apply string functions to manipulate and inspect text data: LEN, LEFT, RIGHT, SUBSTRING, UPPER, LOWER, TRIM, REPLACE, CONCAT
- Apply date functions to extract parts from dates and calculate intervals: YEAR, MONTH, DAY, GETDATE, DATEDIFF, DATEADD, FORMAT
- Handle NULL values gracefully using ISNULL, COALESCE, and NULLIF
- Write CASE expressions for conditional logic in both SELECT and WHERE clauses
- Combine multiple functions in a single expression
- Recognise which functions are sargable and which are not

Table of Contents

1. What Functions Are and How They Work
2. String Functions: Length and Extraction
3. String Functions: Case and Whitespace
4. String Functions: Searching and Replacing
5. String Functions: Building Strings
6. Date Functions: Extracting Parts
7. Date Functions: Calculating Intervals
8. Date Functions: Adding and Subtracting
9. Date Functions: Formatting and Converting
10. NULL-Handling Functions
11. CASE Expressions: Conditional Logic
12. Combining Functions
13. Chapter Summary
14. Review Questions
15. Deeper Dive

---

1. What Functions Are and How They Work

A **function** takes one or more input values, performs an operation, and returns a single output value. Functions in SQL work the same way as functions in mathematics: `f(x) = result`.

1.1 Function Syntax

All SQL functions follow the same basic pattern:

```
FUNCTION_NAME(argument1, argument2, ...)
```

Functions can appear anywhere an expression is valid: in the `SELECT` list, in a `WHERE` condition, in an `ORDER BY` clause, or as an argument to another function.

```
-- Function in SELECT list
SELECT  LEN(ProductName)   AS NameLength
FROM    dim.Product;

-- Function in WHERE clause
SELECT  ProductName
FROM    dim.Product
WHERE   LEN(ProductName) > 30;

-- Function as argument to another function (nested)
SELECT  UPPER(TRIM(CustomerName))   AS CleanUpperName
FROM    dim.Customer;
```

1.2 Scalar Functions

The functions in this chapter are **scalar functions** — they operate on a single value and return a single value. Every row produces its own output independently. This is distinct from **aggregate functions** (like `SUM`, `AVG`, `COUNT`) which operate on a set of rows and return a single result for the whole group. Aggregate functions are covered in Chapter 6.

1.3 The Function Reference Habit

No analyst memorises every function. The professional habit is knowing *what kind of operation you need* and knowing where to look up the syntax. Microsoft's T-SQL function reference, organised by category, is the right place:

Built-in functions (Transact-SQL) — Microsoft Learn

This chapter teaches you the most commonly used functions in each category. More importantly, it teaches you the patterns — once you understand how string and date functions work conceptually, new ones are quick to learn.

---

2. String Functions: Length and Extraction

2.1 LEN: String Length

`LEN(string)` returns the number of characters in a string, not counting trailing spaces:

```
SELECT  ProductName,
        LEN(ProductName)    AS NameLength
FROM    dim.Product
ORDER BY NameLength DESC;
```

```
-- Find products with unusually short names (possible data quality issue)
SELECT  ProductName,
        LEN(ProductName)    AS NameLength
FROM    dim.Product
WHERE   LEN(ProductName) < 10
ORDER BY NameLength;
```

> ***LEN vs DATALENGTH:*** *`LEN` returns the character count. `DATALENGTH` returns the byte count — for `NVARCHAR`, this is twice the character count (2 bytes per Unicode character). For most analytical purposes, `LEN` is what you want.*

2.2 LEFT and RIGHT: Extract from Ends

`LEFT(string, n)` returns the leftmost `n` characters. `RIGHT(string, n)` returns the rightmost `n` characters:

```
-- Extract the first two characters of province code (already 2 chars, but illustrative)
SELECT  CustomerName,
        ProvinceCode,
        LEFT(ProvinceCode, 1)   AS FirstLetter
FROM    dim.Customer
ORDER BY ProvinceCode;
```

```
-- Extract the last 3 characters of product codes
SELECT  ProductCode,
        ProductName,
        RIGHT(ProductCode, 3)   AS CodeSuffix
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
ORDER BY ProductCode;
```

2.3 SUBSTRING: Extract from the Middle

`SUBSTRING(string, start_position, length)` extracts a portion of a string starting at `start_position` (1-indexed) and taking `length` characters:

```
-- Extract characters 4 through 6 of the product code
SELECT  ProductCode,
        SUBSTRING(ProductCode, 4, 3)    AS CodeMiddle
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
ORDER BY ProductCode;
```

```
-- Extract everything after the first dash in a product code like 'CT-083'
-- Position 4 onward (positions 1-2 = 'CT', 3 = '-', 4+ = the numeric part)
SELECT  ProductCode,
        SUBSTRING(ProductCode, 4, LEN(ProductCode) - 3)    AS NumericPart
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
AND     LEN(ProductCode) > 3
ORDER BY ProductCode;
```

Notice how `LEN` is nested inside `SUBSTRING` — the length argument is itself a calculated expression. Combining functions this way is the norm in practice.

2.4 CHARINDEX: Find a Character's Position

`CHARINDEX(search_string, target_string [, start_position])` returns the position of the first occurrence of `search_string` within `target_string`. Returns 0 if not found:

```
-- Find the position of a space in full names (separates first from last name)
SELECT  FullName,
        CHARINDEX(' ', FullName)            AS SpacePosition,
        -- Extract first name: everything before the space
        LEFT(FullName, CHARINDEX(' ', FullName) - 1)   AS FirstName,
        -- Extract last name: everything after the space
        SUBSTRING(FullName, CHARINDEX(' ', FullName) + 1,
                  LEN(FullName))                        AS LastName
FROM    dim.Employee
WHERE   CHARINDEX(' ', FullName) > 0    -- Only process names that have a space
ORDER BY FullName;
```

`CHARINDEX` is the building block for splitting a string on a delimiter when you do not know the exact position in advance.

---

3. String Functions: Case and Whitespace

3.1 UPPER and LOWER: Change Case

`UPPER(string)` converts all characters to uppercase. `LOWER(string)` converts all to lowercase:

```
SELECT  CustomerName,
        UPPER(CustomerName)     AS UpperName,
        LOWER(CustomerName)     AS LowerName
FROM    dim.Customer
ORDER BY CustomerName;
```

Practical use: Normalising data for case-insensitive comparison when the database collation might not guarantee it, or for generating display labels in consistent case.

```
-- Use UPPER to ensure consistent comparison regardless of source data case
SELECT  ProductName
FROM    dim.Product
WHERE   UPPER(CategoryName) = 'SLEEPING';
-- Safe even if source has 'sleeping', 'SLEEPING', or 'Sleeping'
```

> ***Sargability reminder:*** *`WHERE UPPER(CategoryName) = 'SLEEPING'` applies a function to the column, making the condition non-sargable. On small dimension tables this is fine. On large tables, this prevents index use. The sargable alternative is `WHERE CategoryName = 'Sleeping'` — relying on SQL Server's case-insensitive collation instead of forcing case conversion.*

3.2 TRIM, LTRIM, RTRIM: Remove Whitespace

Extra spaces at the beginning or end of strings are a common data quality issue — they look invisible but cause string comparisons to fail silently.

- `TRIM(string)` removes both leading and trailing spaces (SQL Server 2017+)
- `LTRIM(string)` removes leading (left) spaces
- `RTRIM(string)` removes trailing (right) spaces

```
-- Demonstrate the difference
SELECT  '  Halifax  '               AS Original,
        LTRIM('  Halifax  ')        AS LeadingRemoved,  -- 'Halifax  '
        RTRIM('  Halifax  ')        AS TrailingRemoved, -- '  Halifax'
        TRIM('  Halifax  ')         AS BothRemoved;     -- 'Halifax'
```

```
-- Clean customer names before comparison
SELECT  CustomerName,
        TRIM(CustomerName)          AS CleanName,
        LEN(CustomerName)           AS OriginalLength,
        LEN(TRIM(CustomerName))     AS CleanLength
FROM    dim.Customer
WHERE   LEN(CustomerName) <> LEN(TRIM(CustomerName))
ORDER BY CustomerName;
-- Returns customers whose names have leading or trailing spaces (a data quality flag)
```

4. String Functions: Searching and Replacing

4.1 REPLACE: Substitute Text

`REPLACE(string, search_string, replacement_string)` replaces all occurrences of `search_string` within `string` with `replacement_string`:

```
-- Replace 'Cabot' with 'Cape' in product names (hypothetical rebrand)
SELECT  ProductName,
        REPLACE(ProductName, 'Cabot', 'Cape')   AS RebrandedName
FROM    dim.Product
WHERE   ProductName LIKE '%Cabot%'
ORDER BY ProductName;
```

```
-- Remove dashes from product codes
SELECT  ProductCode,
        REPLACE(ProductCode, '-', '')           AS CodeNoDash
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
ORDER BY ProductCode;
```

`REPLACE` is case-sensitive in default SQL Server collation. `REPLACE('CabotTrail', 'cabot', 'Cape')` would NOT replace 'Cabot' because 'cabot' ≠ 'Cabot'. If case-insensitive replacement is needed, use `REPLACE(LOWER(string), LOWER(search), replacement)` — though this changes the case of the entire string.

4.2 STUFF: Replace at a Position

`STUFF(string, start, length, replacement)` removes `length` characters starting at `start` and inserts `replacement` in their place:

```
-- Insert a dash after the 2nd character of a code (e.g., 'CT083' → 'CT-083')
SELECT  ProductCode,
        STUFF(ProductCode, 3, 0, '-')   AS FormattedCode
FROM    dim.Product
WHERE   ProductCode IS NOT NULL
AND     ProductCode NOT LIKE '%-%'  -- Only codes without a dash already
ORDER BY ProductCode;
```

`STUFF` with a length of 0 inserts without removing. `STUFF` with length > 0 replaces that many characters. It is more surgical than `REPLACE` when position matters.

---

5. String Functions: Building Strings

5.1 Concatenation with + and CONCAT

Strings are joined using the `+` operator or the `CONCAT` function:

```
-- Using + operator
SELECT  FirstName + ' ' + LastName      AS FullName
FROM    SomeTable;

-- Using CONCAT function (preferred for NULL safety)
SELECT  CONCAT(FirstName, ' ', LastName)    AS FullName
FROM    SomeTable;
```

The critical difference: The `+` operator returns NULL if any operand is NULL. `CONCAT` treats NULL as an empty string:

```
SELECT  'Hello' + NULL;             -- Returns: NULL
SELECT  CONCAT('Hello', NULL);      -- Returns: 'Hello'
```

```
-- Build a product label: code + pipe separator + name
-- Using + (breaks when ProductCode is NULL)
SELECT  ProductCode + ' | ' + ProductName   AS ProductLabel
FROM    dim.Product;
-- Returns NULL for rows where ProductCode IS NULL

-- Using CONCAT (handles NULL gracefully)
SELECT  CONCAT(ProductCode, ' | ', ProductName)  AS ProductLabel
FROM    dim.Product;
-- Returns ' | ProductName' for NULL codes — still might not be ideal

-- Best: handle NULL explicitly
SELECT  ISNULL(ProductCode, 'No Code') + ' | ' + ProductName   AS ProductLabel
FROM    dim.Product;
```

5.2 CONCAT_WS: Concatenate with a Separator

`CONCAT_WS(separator, value1, value2, ...)` (SQL Server 2017+) joins multiple strings with a consistent separator, skipping NULLs:

```
-- Build a geographic label: City, Province, Country
SELECT  CustomerName,
        CONCAT_WS(', ', CityName, ProvinceName, CountryName)    AS Location
FROM    dim.Customer
ORDER BY CustomerName;
-- Result: 'Halifax, Nova Scotia, Canada'
-- NULL values in any argument are skipped (no double separators)
```

`CONCAT_WS` is cleaner than chaining `+` operators for multi-part concatenation where some values might be NULL.

---

6. Date Functions: Extracting Parts

These functions pull a specific component out of a date or datetime value.

6.1 YEAR, MONTH, DAY

The simplest date extraction functions return an integer:

```
-- Extract calendar components from the Calendar dimension
SELECT  FullDate,
        YEAR(FullDate)      AS YearPart,
        MONTH(FullDate)     AS MonthPart,
        DAY(FullDate)       AS DayPart
FROM    dim.Calendar
WHERE   FullDate = '2024-03-15';
-- Returns: 2024, 3, 15
```

```
-- Use YEAR in a filter
SELECT  FullDate, DayName, MonthName, IsHoliday
FROM    dim.Calendar
WHERE   YEAR(FullDate) = 2024
AND     MONTH(FullDate) = 12
ORDER BY FullDate;
-- All December 2024 dates
```

> ***Performance note:*** *`WHERE YEAR(FullDate) = 2024` is non-sargable — it applies a function to the column. The sargable equivalent is `WHERE FullDate >= '2024-01-01' AND FullDate < '2025-01-01'`. In the `dim.Calendar` table with only 4,017 rows, the difference is imperceptible. On large tables with millions of rows, always prefer the range comparison.*

6.2 DATEPART: General Component Extraction

`DATEPART(datepart, date)` is the general-purpose date component extractor. It returns the same integer values as `YEAR`, `MONTH`, and `DAY`, but also handles week numbers, quarters, hours, and many more components:

datepart	Returns	Example
`year`	Year number	`2024`
`quarter`	1–4	`1` (January–March)
`month`	1–12	`3` (March)
`week`	1–53	`11` (week 11 of the year)
`dayofyear`	1–366	`75` (75th day of the year)
`day`	1–31	`15`
`weekday`	1–7	`6` (Friday in default US setting)
`hour`	0–23	`14`
`minute`	0–59	`30`

```
-- Extract calendar quarter
SELECT  FullDate,
        DATEPART(quarter, FullDate)     AS CalendarQuarter,
        DATEPART(week,    FullDate)     AS WeekNumber
FROM    dim.Calendar
WHERE   YEAR(FullDate) = 2024
ORDER BY FullDate;
```

```
-- Find all Mondays in 2024
SELECT  FullDate,
        DayName,
        WeekNumber
FROM    dim.Calendar
WHERE   YEAR(FullDate) = 2024
AND     DATEPART(weekday, FullDate) = 2   -- 2 = Monday (1=Sunday in SQL Server default)
ORDER BY FullDate;
```

6.3 DATENAME: Returns the Name, Not the Number

`DATENAME(datepart, date)` returns the name of the component as a string rather than a number:

```
SELECT  FullDate,
        DATENAME(month,   FullDate)     AS MonthName,
        DATENAME(weekday, FullDate)     AS DayName
FROM    dim.Calendar
WHERE   FullDate = '2024-03-15';
-- Returns: 'March', 'Friday'
```

Note that `dim.Calendar` already has pre-computed `MonthName` and `DayName` columns — you do not need `DATENAME` in those queries. These functions are most useful when working with a table that stores raw dates without pre-computed name columns (which you will encounter in the OLTP database in Chapter 9).

6.4 GETDATE and SYSDATETIME

`GETDATE()` returns the current date and time as a `DATETIME`:

```
SELECT  GETDATE()   AS CurrentDateTime;
-- Returns something like: 2026-09-10 14:23:07.847
```

`SYSDATETIME()` returns higher precision:

```
SELECT  SYSDATETIME()   AS HighPrecisionNow;
-- Returns: 2026-09-10 14:23:07.8476532
```

`CAST(GETDATE() AS DATE)` strips the time component when you only need today's date:

```
SELECT  CAST(GETDATE() AS DATE)     AS Today;
-- Returns: 2026-09-10
```

---

7. Date Functions: Calculating Intervals

7.1 DATEDIFF: Difference Between Two Dates

`DATEDIFF(datepart, start_date, end_date)` returns the number of complete `datepart` intervals between two dates:

```
DATEDIFF(day,   '2024-01-01', '2024-03-15')  -- Returns: 74 (74 days)
DATEDIFF(month, '2024-01-01', '2024-03-15')  -- Returns: 2  (2 months)
DATEDIFF(year,  '2020-06-15', '2024-03-15')  -- Returns: 3  (3 full years elapsed)
```

Note: `DATEDIFF` counts boundary crossings, not exact intervals. `DATEDIFF(year, '2020-12-31', '2021-01-01')` returns 1, even though only one day has passed — a year boundary was crossed.

```
-- Calculate how long each customer has been with CabotTrail
SELECT  CustomerName,
        AccountOpenedDate,
        CAST(GETDATE() AS DATE)                              AS Today,
        DATEDIFF(day,  AccountOpenedDate, GETDATE())         AS DaysSinceOpened,
        DATEDIFF(year, AccountOpenedDate, GETDATE())         AS ApproxYearsSince
FROM    dim.Customer
WHERE   AccountOpenedDate IS NOT NULL
ORDER BY AccountOpenedDate ASC;
```

```
-- From fact.Sales: days between order date and invoice date
-- (requires joining to dim.Calendar for actual dates)
USE CabotTrailOutdoorDW;

SELECT  TOP 10
        fs.InvoiceID,
        fs.OrderDate,
        fs.InvoiceDate,
        DATEDIFF(day, fs.OrderDate, fs.InvoiceDate)     AS DaysToInvoice
FROM    Fact.FactSales fs
ORDER BY DaysToInvoice DESC;
```

7.2 Using DATEDIFF in the CabotTrail Sales Datamart

`fact.Sales` in `CabotTrailOutdoorsSales` already has pre-computed `DaysUntilInvoice` and `DaysUntilDue` columns — exactly the results of `DATEDIFF` applied during ETL. You can verify them:

```
USE CabotTrailOutdoorsSales;

SELECT  TOP 5
        InvoiceID,
        OrderDateKey,
        InvoiceDateKey,
        DaysUntilInvoice,       -- Pre-computed column
        -- Verify using calendar dimension:
        DATEDIFF(day,
            ord_cal.FullDate,
            inv_cal.FullDate)   AS CalculatedDays
FROM    fact.Sales fs
INNER JOIN dim.Calendar ord_cal ON ord_cal.DateKey = fs.OrderDateKey
INNER JOIN dim.Calendar inv_cal ON inv_cal.DateKey = fs.InvoiceDateKey
ORDER BY InvoiceID;
-- DaysUntilInvoice and CalculatedDays should match
```

This query previews the JOIN syntax you will learn in Chapter 5, but demonstrates an important principle: pre-computed columns in fact tables can always be verified by re-computing from source data.

---

8. Date Functions: Adding and Subtracting

8.1 DATEADD: Shift a Date

`DATEADD(datepart, number, date)` adds `number` units of `datepart` to `date`:

```
SELECT  DATEADD(day,    7,  '2024-03-15')   -- Returns: 2024-03-22  (one week later)
SELECT  DATEADD(month,  3,  '2024-03-15')   -- Returns: 2024-06-15  (3 months later)
SELECT  DATEADD(year,  -1,  '2024-03-15')   -- Returns: 2023-03-15  (1 year ago)
SELECT  DATEADD(day,  -30,  GETDATE())      -- Returns: 30 days ago from today
```

Negative values move backwards in time. `DATEADD` is useful for:

- Calculating a date range relative to today

- Finding a due date given a start date and term

- Generating a rolling window (last 90 days, last 12 months)

```
-- All calendar dates within the last 90 days from today
SELECT  FullDate, MonthName, DayName
FROM    dim.Calendar
WHERE   FullDate >= CAST(DATEADD(day, -90, GETDATE()) AS DATE)
AND     FullDate <= CAST(GETDATE() AS DATE)
ORDER BY FullDate;
```

```
-- What date is 30 days after each customer's account was opened?
SELECT  CustomerName,
        AccountOpenedDate,
        DATEADD(day, 30, AccountOpenedDate)     AS ThirtyDayMark
FROM    dim.Customer
WHERE   AccountOpenedDate IS NOT NULL
ORDER BY AccountOpenedDate;
```

8.2 Combining DATEDIFF and DATEADD

Together, `DATEDIFF` and `DATEADD` enable sophisticated date arithmetic:

```
-- First day of the current month
SELECT  DATEADD(day, 1 - DAY(GETDATE()), CAST(GETDATE() AS DATE))
        AS FirstDayOfMonth;
-- EXPLANATION: DAY(GETDATE()) = today's day number (e.g., 15)
-- 1 - 15 = -14
-- DATEADD subtracts 14 days from today → returns the 1st of this month

-- Last day of the previous month
SELECT  DATEADD(day, -DAY(GETDATE()), CAST(GETDATE() AS DATE))
        AS LastDayOfPreviousMonth;
```

These patterns are common in financial reporting — generating period boundaries dynamically without hardcoding dates.

---

9. Date Functions: Formatting and Converting

9.1 FORMAT: Display Dates as Strings

`FORMAT(value, format_string [, culture])` converts a date or number to a formatted string:

```
-- Date formatting
SELECT  FORMAT(GETDATE(), 'yyyy-MM-dd')             AS ISO_Format,    -- 2026-09-10
        FORMAT(GETDATE(), 'MMMM dd, yyyy')          AS LongDate,      -- September 10,
2026
        FORMAT(GETDATE(), 'MMM yyyy')               AS ShortMonthYear, -- Sep 2026
        FORMAT(GETDATE(), 'dd/MM/yyyy')             AS UKFormat;       -- 10/09/2026

-- Number formatting
SELECT  FORMAT(1234567.89, 'N2')                    AS NumberFormat,  -- 1,234,567.89
        FORMAT(1234567.89, 'C', 'en-CA')            AS CanadianDollar; -- $1,234,567.89
        FORMAT(0.4567,    'P1')                     AS PercentFormat; -- 45.7%
```

Important performance warning: `FORMAT` is a .NET CLR function — it is significantly slower than native T-SQL date functions, especially on large row sets. Use it for display purposes on small result sets (report outputs, test queries) — not for filtering or intermediate calculations in large queries.

The sargable alternatives for date formatting in large queries:

```
-- For filtering: use direct comparison, not FORMAT
-- SLOW (non-sargable):
WHERE FORMAT(OrderDate, 'yyyy') = '2024'

-- FAST (sargable):
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'
```

9.2 CONVERT with Style Codes

`CONVERT(data_type, expression, style)` is the older SQL Server approach to date formatting:

```
SELECT  CONVERT(NVARCHAR(10), GETDATE(), 120)   AS ISO_Date,    -- 2026-09-10
        CONVERT(NVARCHAR(12), GETDATE(), 107)   AS MonDdYyyy;   -- Sep 10, 2026
```

Style codes are numeric and specific to SQL Server. `FORMAT` with descriptive format strings is more readable for most purposes — use `CONVERT` when compatibility with older SQL Server versions matters.

---

10. NULL-Handling Functions

Chapter 2 introduced NULL conceptually. This section covers the functions that deal with NULL values in expressions and calculations.

10.1 ISNULL: Replace NULL with a Default

`ISNULL(expression, replacement)` returns `replacement` if `expression` is NULL; otherwise returns `expression`:

```
-- Replace NULL product codes with 'No Code'
SELECT  ProductName,
        ISNULL(ProductCode, 'No Code')  AS ProductCode
FROM    dim.Product
ORDER BY ProductName;
```

```
-- Replace NULL email addresses with a placeholder
SELECT  FullName,
        ISNULL(EmailAddress, 'no-email@cabot-trail.ca')     AS ContactEmail
FROM    dim.Employee
ORDER BY FullName;
```

`ISNULL` takes exactly two arguments: the expression to check and the default value. Both must be compatible data types (or SQL Server must be able to implicitly convert one to the other).

10.2 COALESCE: First Non-NULL from a List

`COALESCE(value1, value2, value3, ...)` returns the first non-NULL value from its argument list:

```
-- Return the best available identifier for each product
SELECT  ProductName,
        COALESCE(ProductCode, ProductName, 'Unknown')   AS BestIdentifier
FROM    dim.Product;
-- Uses ProductCode if available; falls back to ProductName; then 'Unknown'
```

```
-- Use first available contact method
SELECT  EmployeeName,
        COALESCE(MobilePhone, WorkPhone, Email, 'No contact')   AS ContactMethod
FROM    SomeEmployeeTable;
```

`COALESCE` is the standard SQL equivalent of `ISNULL`. Prefer `COALESCE` when:

- You have more than two values to check
- You want SQL-standard syntax that works across database platforms

Prefer `ISNULL` when:

- You have exactly two values (slightly faster in SQL Server)

- You are in a SQL Server–only environment and prefer the explicit two-argument form

10.3 NULLIF: Return NULL When Values Are Equal

`NULLIF(expression1, expression2)` returns NULL when the two expressions are equal; otherwise returns `expression1`:

```
SELECT  NULLIF(5,  5);    -- Returns: NULL  (equal)
SELECT  NULLIF(5,  3);    -- Returns: 5     (not equal)
SELECT  NULLIF(0,  0);    -- Returns: NULL  (equal)
```

Primary use case — divide-by-zero protection:

```
-- Safe margin calculation: NULLIF converts zero denominators to NULL
-- NULL / anything = NULL (no error)
SELECT  InvoiceID,
        GrossProfit,
        LineTotal,
        ROUND(GrossProfit / NULLIF(LineTotal, 0) * 100, 2)     AS MarginPct
FROM    fact.Sales
ORDER BY InvoiceID;
```

Secondary use case — treat a sentinel value as NULL:

```
-- A legacy system uses 0 to mean "no credit limit" but should be NULL
SELECT  CustomerName,
        NULLIF(CreditLimit, 0)  AS CreditLimitOrNull
FROM    dim.Customer;
-- Converts 0 credit limits to NULL, making them identifiable as "not set"
```

---

11. CASE Expressions: Conditional Logic

The `CASE` expression is SQL's equivalent of an IF/THEN/ELSE construct. It evaluates conditions and returns different values based on which condition is TRUE.

11.1 Searched CASE

The **searched CASE** evaluates a series of conditions in order and returns the result of the first TRUE condition:

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN condition3 THEN result3
    ELSE default_result
END
```

```
-- Classify products into price tiers
SELECT  ProductName,
        UnitPrice,
        CASE
            WHEN UnitPrice < 50     THEN 'Budget'
            WHEN UnitPrice < 150    THEN 'Mid-Range'
            WHEN UnitPrice < 400    THEN 'Premium'
            ELSE                         'Luxury'
        END                         AS PriceTier
FROM    dim.Product
ORDER BY UnitPrice;
```

CASE conditions are evaluated top to bottom — the first TRUE condition wins. This means the ranges above do not need to include upper bounds: `WHEN UnitPrice < 150` only reaches products between $50 and $150, because products under $50 were already captured by the first condition.

11.2 Simple CASE

The **simple CASE** compares a single expression to multiple values:

```
CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE default_result
END
```

```
-- Convert province code to full name
SELECT  CustomerName,
        ProvinceCode,
        CASE ProvinceCode
            WHEN 'NS' THEN 'Nova Scotia'
            WHEN 'NB' THEN 'New Brunswick'
            WHEN 'PE' THEN 'Prince Edward Island'
            WHEN 'NL' THEN 'Newfoundland and Labrador'
            WHEN 'QC' THEN 'Quebec'
            WHEN 'ON' THEN 'Ontario'
            WHEN 'AB' THEN 'Alberta'
            WHEN 'BC' THEN 'British Columbia'
            WHEN 'MB' THEN 'Manitoba'
            WHEN 'SK' THEN 'Saskatchewan'
            ELSE            'Other'
        END                     AS ProvinceName
FROM    dim.Customer
ORDER BY ProvinceCode;
```

The simple CASE is more concise when testing equality on the same expression. The searched CASE is more flexible — it can test any condition, including inequalities and NULL checks.

11.3 CASE with NULL Handling

CASE can check for NULL values using `IS NULL`:

```
-- Label products as 'Has Code' or 'No Code'
SELECT  ProductName,
        CASE
            WHEN ProductCode IS NULL THEN 'No Code Assigned'
            ELSE ProductCode
        END                         AS ProductCodeDisplay
FROM    dim.Product
ORDER BY ProductName;
```

11.4 CASE in WHERE Clauses

`CASE` can appear in a `WHERE` clause to implement conditional filtering logic, though this is less common than using it in the SELECT list:

```
-- Return high-value items (price varies by category)
SELECT  ProductName, CategoryName, UnitPrice
FROM    dim.Product
WHERE
    CASE
        WHEN CategoryName IN ('Shelter', 'Paddling') THEN
            CASE WHEN UnitPrice > 500 THEN 1 ELSE 0 END
        ELSE
            CASE WHEN UnitPrice > 200 THEN 1 ELSE 0 END
    END = 1
ORDER BY CategoryName, UnitPrice DESC;
```

This returns Shelter and Paddling items over $500, and all other categories over $200. The CASE expression returns 1 (TRUE) or 0 (FALSE), and `WHERE ... = 1` filters to the TRUE rows.

11.5 CASE with Aggregation (Preview)

CASE becomes especially powerful inside aggregate functions — a pattern you will use extensively in Chapter 6:

```
-- Count of discontinued vs active products by category
-- (This previews Chapter 6 aggregation — run it to see the results)
SELECT  CategoryName,
        COUNT(CASE WHEN IsDiscontinued = 0 THEN 1 END)  AS ActiveCount,
        COUNT(CASE WHEN IsDiscontinued = 1 THEN 1 END)  AS DiscontinuedCount,
        COUNT(*)                                         AS TotalCount
FROM    dim.Product
GROUP BY CategoryName
ORDER BY CategoryName;
```

---

12. Combining Functions

In practice, multiple functions are combined in a single expression. The key is working from the inside out — the innermost function runs first.

12.1 Nested Functions

```
-- Build a display name: "FIRST LAST (Dept)"
SELECT  FullName,
        Department,
        UPPER(LEFT(FullName, CHARINDEX(' ', FullName) - 1)) -- First name, uppercase
        + ' '
        + UPPER(SUBSTRING(FullName, CHARINDEX(' ', FullName) + 1, LEN(FullName)))  --
Last name
        + ' (' + Department + ')'                           -- Department in brackets
        AS DisplayName
FROM    dim.Employee
WHERE   CHARINDEX(' ', FullName) > 0
ORDER BY FullName;
```

Working through this expression from the inside out:

1. `CHARINDEX(' ', FullName)` → position of the space
2. `LEFT(FullName, position - 1)` → first name
3. `UPPER(first_name)` → first name in uppercase
4. `SUBSTRING(FullName, position + 1, LEN(FullName))` → last name
5. `UPPER(last_name)` → last name in uppercase
6. Concatenate: FIRSTNAME LASTNAME (Department)

12.2 Functions with CASE

```
-- Clean and classify product names
SELECT  ProductName,
        TRIM(ProductName)                               AS CleanName,
        LEN(TRIM(ProductName))                          AS CleanLength,
        CASE
            WHEN LEN(TRIM(ProductName)) <= 20   THEN 'Short'
            WHEN LEN(TRIM(ProductName)) <= 40   THEN 'Medium'
            ELSE                                     'Long'
        END                                             AS NameLengthCategory
FROM    dim.Product
ORDER BY CleanLength DESC;
```

12.3 A Business Analysis Query

Putting the whole chapter together — a query that answers a real analytical question using multiple function types:

```
-- Customer account summary: how long each customer has been with us,
-- what territory they are in, and their credit tier

USE CabotTrailOutdoorsSales;

SELECT
    CustomerName,
    CONCAT_WS(', ', CityName, ProvinceCode)                AS Location,
    SalesTerritory,
    AccountOpenedDate,
    DATEDIFF(year, AccountOpenedDate, GETDATE())            AS YearsAsCustomer,
    -- Credit tier based on credit limit
    CASE
        WHEN CreditLimit = 0     THEN 'No Credit'
        WHEN CreditLimit < 2500  THEN 'Basic'
        WHEN CreditLimit < 7500  THEN 'Standard'
        ELSE                          'Premium'
    END                                                     AS CreditTier,
    CreditLimit
FROM    dim.Customer
WHERE   AccountOpenedDate IS NOT NULL
ORDER BY YearsAsCustomer DESC, CustomerName;
```

---

13. Chapter Summary

- **Scalar functions** operate on one value at a time and return one value per row. They can appear in SELECT, WHERE, and ORDER BY clauses.
- **String functions:** `LEN` (length), `LEFT` / `RIGHT` / `SUBSTRING` (extraction), `CHARINDEX` (find position), `UPPER` / `LOWER` (case), `TRIM` / `LTRIM` / `RTRIM` (whitespace), `REPLACE` / `STUFF` (substitution), `CONCAT` / `CONCAT_WS` (joining — prefer `CONCAT` over `+` for NULL safety).
- **Date functions:** `YEAR` / `MONTH` / `DAY` (simple extraction), `DATEPART` (general extraction by name), `DATENAME` (returns component name as string), `GETDATE` / `SYSDATETIME` (current time), `DATEDIFF` (interval between dates), `DATEADD` (shift a date), `FORMAT` (display formatting — use sparingly on large tables), `CONVERT` (type conversion with style).
- **NULL-handling functions:** `ISNULL(expr, default)` (two arguments, SQL Server), `COALESCE(v1, v2, ...)` (first non-NULL, standard SQL), `NULLIF(a, b)` (returns NULL when a=b — divide-by-zero guard).
- **CASE expressions** provide conditional logic. Searched CASE evaluates arbitrary conditions; simple CASE tests equality against a single expression. CASE can appear in SELECT, WHERE, and (as a preview of Chapter 6) inside aggregate functions.

- **Sargability:** Functions applied to columns in WHERE clauses prevent index use. Apply functions to literals, not columns, whenever possible for large-table performance.

---

14. Review Questions

1. Write a query against `dim.Employee` that returns `FullName` and two calculated columns: `FirstName` (everything before the first space) and `LastName` (everything after the first space). Use `CHARINDEX`, `LEFT`, `SUBSTRING`, and `LEN`. Include only employees whose full name contains a space.
2. Write a query against `dim.Product` that returns `ProductName`, `CategoryName`, `UnitPrice`, and a calculated column `ProductLabel` that combines `ProductCode` and `ProductName` with a `|` separator. Handle NULL `ProductCode` values by substituting `'UNASSIGNED'`. Sort by `CategoryName`, then `ProductName`.
3. Using `dim.Customer`, write a query that returns `CustomerName`, `AccountOpenedDate`, and three calculated date columns:
   - `AccountAgeYears`: full years since the account was opened (using `DATEDIFF`)
   - `AccountAgeMonths`: full months since the account was opened
   - `RenewalDate`: 365 days after `AccountOpenedDate` (using `DATEADD`)

   Filter to customers whose account opened before January 1, 2023. Sort by `AccountOpenedDate`.
4. Explain the difference between `ISNULL` and `COALESCE`. Under what circumstances would you choose one over the other? Give a specific example where `COALESCE` is more appropriate.
5. Write a CASE expression that classifies each customer's `CreditLimit` from `dim.Customer` into four tiers: `'No Credit'` (0), `'Basic'` (1–2,499), `'Standard'` (2,500–9,999), `'Premium'` (10,000+). Return `CustomerName`, `CreditLimit`, and `CreditTier`, sorted by `CreditLimit` descending.
6. The following query is intended to return all products from the year 2024 that are holidays, using the `dim.Calendar` table, but it performs poorly on large datasets. Explain why it is non-sargable and rewrite it in a sargable form:

   ```sql
   SELECT FullDate, HolidayName FROM dim.Calendar WHERE FORMAT(FullDate, 'yyyy') = '2024' AND IsHoliday = 1;
   ```
7. Write a query against `dim.Product` that returns `ProductName`, `CategoryName`, and a column `WeightCategory` using a CASE expression: `'Light'` if `TypicalWeightPerUnit < 1`, `'Medium'` if between 1 and 5 (inclusive), `'Heavy'` if over 5, and `'Unknown'` if `TypicalWeightPerUnit IS NULL`. Sort by `WeightCategory`, then `TypicalWeightPerUnit`.

8. Using `CONCAT_WS`, write a query that builds a shipping label for each customer in the format `"CustomerName | CityName, ProvinceCode"`. Return customers in British Columbia (`ProvinceCode = 'BC'`) only, sorted by `CustomerName`.

Deeper Dive

Going Further with Functions

String Functions and Data Quality

String functions are the analyst's primary tool for **data cleansing** — identifying and correcting messy text data. Several patterns appear repeatedly in production work:

Detecting leading/trailing spaces:

```
-- Find rows where the stored value differs from the trimmed value
SELECT column_name, LEN(column_name), LEN(TRIM(column_name))
FROM   some_table
WHERE  LEN(column_name) <> LEN(TRIM(column_name));
```

Detecting non-printable characters:

Sometimes data contains hidden non-printable characters (tab `CHAR(9)`, carriage return `CHAR(13)`, newline `CHAR(10)`) that cause invisible comparison failures:

```
-- Remove common non-printable characters
SELECT REPLACE(REPLACE(REPLACE(CustomerName, CHAR(9), ''), CHAR(10), ''), CHAR(13), '')
AS CleanName
FROM dim.Customer;
```

Standardising formats:

```
-- Standardise phone number formats: remove all non-digits
-- (Requires a recursive CTE or CLR function for true regex-based cleaning)
-- Simplified: remove common punctuation
SELECT REPLACE(REPLACE(REPLACE(PhoneNumber, '(', ''), ')', ''), '-', '')
AS StandardPhone
FROM SomeContactTable;
```

In production ETL systems, these cleansing operations are encoded in the source-to-target mapping and applied during the transformation stage.

Date Arithmetic Patterns

A few date calculation patterns come up so frequently that they are worth memorising:

```
-- First day of the current month
DATEADD(month, DATEDIFF(month, 0, GETDATE()), 0)

-- Last day of the current month
DATEADD(day, -1, DATEADD(month, DATEDIFF(month, 0, GETDATE()) + 1, 0))

-- First day of the current year
DATEADD(year, DATEDIFF(year, 0, GETDATE()), 0)

-- First day of the current quarter
DATEADD(quarter, DATEDIFF(quarter, 0, GETDATE()), 0)

-- Same date last year
DATEADD(year, -1, CAST(GETDATE() AS DATE))
```

The pattern `DATEADD(part, DATEDIFF(part, 0, date), 0)` strips the sub-part components — it finds the first moment of the current period. The number `0` is the SQL Server date origin (January 1, 1900). `DATEDIFF` counts periods from the origin; `DATEADD` adds them back. The result is always the first day of the period.

These patterns are workhorses in financial reporting where period boundaries must be calculated dynamically.

The IIF Function: A Shorthand CASE

SQL Server 2012 introduced `IIF(condition, true_value, false_value)` as a shorthand for a two-branch CASE expression:

```
-- IIF equivalent of a simple CASE
SELECT  ProductName,
        IIF(IsDiscontinued = 1, 'Discontinued', 'Active')   AS Status
FROM    dim.Product;

-- Equivalent CASE expression
SELECT  ProductName,
        CASE WHEN IsDiscontinued = 1 THEN 'Discontinued' ELSE 'Active' END  AS Status
FROM    dim.Product;
```

`IIF` is convenient for simple binary conditions. For more than two outcomes, `CASE` is required. `IIF` is not standard SQL — it is a SQL Server and Microsoft Access extension. Prefer `CASE` for portability and when the condition has more than two outcomes.

Regular Expressions via CLR

SQL Server does not have built-in regular expression functions in T-SQL. However, SQL Server supports **CLR (Common Language Runtime) integration** — meaning you can write a .NET function in C# and register it in SQL Server, making it available as a T-SQL function.

Many organizations deploy CLR-based regex functions for complex pattern matching and replacement. A common pattern:

```
-- Hypothetical CLR function (not built-in)
SELECT dbo.RegexIsMatch(EmailAddress, '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')
AS IsValidEmail
FROM dim.Customer;
```

If you find yourself writing deeply nested `CHARINDEX`, `SUBSTRING`, and `REPLACE` expressions to achieve what a regex would express in one line, it may be worth discussing CLR function deployment with your DBA.

Window Functions: A Preview

Chapter 8 covers window functions in depth, but it is worth previewing how they relate to the scalar functions in this chapter. Functions like `LAG`, `LEAD`, `ROW_NUMBER`, and `RANK` look syntactically similar to scalar functions but operate across rows rather than on individual values:

```
-- LAG: compare a value to the previous row's value
-- (Full explanation in Chapter 8)
SELECT  MonthName,
        Revenue,
        LAG(Revenue) OVER (ORDER BY MonthNumber)    AS PreviousMonthRevenue
FROM    MonthlyRevenueSummary;
```

The `OVER (ORDER BY ...)` clause is what distinguishes a window function from a scalar function — it defines the set of rows to operate over. For now, know that the function families you have learned in this chapter (scalar functions) are the building blocks that window functions build upon.

Industry Perspectives

Functions as Business Rules

Every function call in a SQL query is a business rule in disguise. `CASE WHEN UnitPrice < 50 THEN 'Budget'` encodes the business's definition of a budget product. `DATEDIFF(year, AccountOpenedDate, GETDATE())` encodes how the business measures customer tenure.

These rules belong in documentation. When an analyst asks "how is customer tier calculated?", the answer should be findable in the S2T mapping or data dictionary — not only in a SQL query buried in a report. The discipline of documenting the business rules behind your calculations is as important as writing the calculations correctly.

The functions in this chapter — especially CASE expressions — are where analytical SQL most closely resembles business logic. Writing them precisely, naming the results meaningfully, and documenting their intent is the mark of a professional analyst.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 2 covers scalar functions comprehensively with performance considerations.
2. Microsoft. (2024). *String functions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/string-functions-transact-sql
3. Microsoft. (2024). *Date and time functions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/date-and-time-data-types-and-functions-transact-sql
4. Microsoft. (2024). *CASE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/case-transact-sql
5. Microsoft. (2024). *COALESCE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/coalesce-transact-sql
6. Microsoft. (2024). *FORMAT (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/format-transact-sql
7. Microsoft. (2024). *DATEDIFF (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/datediff-transact-sql

---

Chapter 5: Joining Tables — Reading a Relational Model

Chapter Overview

Every chapter so far has queried one table at a time. That is enough for the dimension tables — `dim.Product` and `dim.Customer` are human-readable on their own. But `fact.Sales` is not. It contains keys and numbers, and it only becomes meaningful when connected to the dimension tables that provide context.

The `JOIN` is the operation that connects tables. It is the most important concept in relational databases — the mechanism that makes the relational model work. Without `JOIN`, every table is an island. With it, the full picture emerges.

This chapter starts at the beginning — what a join is and why it exists — and builds through the three join types you will use most often: `INNER JOIN`, `LEFT JOIN`, and self-joins. It covers the star schema join pattern in depth, because that is where most analytical SQL queries live.

By the end of this chapter you will be able to:

- Explain what a join is and why it is necessary in a relational database
- Write `INNER JOIN` queries that connect two or more tables
- Use table aliases to write readable multi-table queries
- Write `LEFT JOIN` queries to include rows that have no match
- Describe the difference between `INNER JOIN` and `LEFT JOIN` and choose the right one
- Join a fact table to all of its dimension tables in a star schema
- Recognise and avoid common join mistakes: Cartesian products, fan-out, and ambiguous column names
- Write a self-join to compare rows within a single table

Table of Contents

1. Why We Need JOINs
2. The JOIN Concept: Matching Rows
3. Table Aliases
4. INNER JOIN: Matching Rows Only
5. Joining More Than Two Tables
6. The Star Schema Join Pattern
7. LEFT JOIN: Keep All Rows from the Left
8. Choosing Between INNER JOIN and LEFT JOIN
9. Common Join Mistakes
10. The Self-Join
11. Cross-Database Joins
12. Chapter Summary
13. Review Questions
14. Deeper Dive

1. Why We Need JOINs

Consider what `fact.Sales` looks like without any joins:

```
USE CabotTrailOutdoorsSales;

SELECT TOP 3 * FROM fact.Sales;
```

You will see rows full of numbers and integer keys:

```
SalesKey | OrderDateKey | CustomerKey | ProductKey | EmployeeKey | LineTotal | ...
1        | 20220103     | 15          | 76         | 7           | 499.98    | ...
2        | 20220103     | 8           | 42         | 3           | 84.99     | ...
3        | 20220104     | 22          | 91         | 12          | 1199.97   | ...
```

This tells you almost nothing useful. `CustomerKey = 15` — which customer is that? `ProductKey = 76` — which product? The fact table is intentionally stripped of descriptive information. Those descriptions live in the dimension tables, linked by the keys.

1.1 The Problem the Join Solves

The relational model stores each piece of information exactly once. `CustomerName` is stored in `dim.Customer`, not on every row of `fact.Sales`. This is efficient and correct — but it means you need to *connect* the two tables to answer "which customer placed this order?"

The `JOIN` is that connection. It matches rows from one table to rows in another, based on a shared key value. When `fact.Sales.CustomerKey = dim.Customer.CustomerKey`, SQL Server finds the matching customer row and makes its columns available to the query.

1.2 The Result

```
-- With a JOIN, the fact table becomes readable
SELECT  TOP 3
        cust.CustomerName,
        fs.LineTotal,
        fs.OrderedQuantity
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey;
```

```
CustomerName              | LineTotal | OrderedQuantity
Fundy Bay Outfitters      | 499.98    | 2
Trail Hut Ltd             | 84.99     | 1
Cape Breton Outdoor       | 1199.97   | 3
```

Now the data tells a story. Fundy Bay Outfitters bought $499.98 worth of something. We do not yet know what — that requires a second join to `dim.Product`. This is the nature of multi-table queries: each join adds another layer of context.

---

2. The JOIN Concept: Matching Rows

2.1 What a JOIN Does

A JOIN combines columns from two tables into a single result set. For each row in the first table, it finds matching rows in the second table based on a join condition. The combined columns of both rows form one row in the output.

The join condition is specified in the `ON` clause:

```
FROM    fact.Sales fs
INNER JOIN dim.Customer cust
    ON cust.CustomerKey = fs.CustomerKey
```

This says: "For each row in `fact.Sales`, find the row in `dim.Customer` where `CustomerKey` values match. Combine both rows into the output."

2.2 Keys Are the Bridge

The join condition almost always connects a **foreign key** (in the child table) to a **primary key** (in the parent table):

- `fact.Sales.CustomerKey` is a foreign key — it references `dim.Customer.CustomerKey`
- `dim.Customer.CustomerKey` is the primary key — it uniquely identifies each customer
- The join finds the one `dim.Customer` row whose `CustomerKey` matches the `CustomerKey` on each `fact.Sales` row

This is why the relational model works: foreign keys create the bridges that JOINs use. Every relationship you identified in the ERD diagrams of DBAS 2010 is a potential join condition.

2.3 Visualising the Match

```
fact.Sales:                        dim.Customer:
SalesKey | CustomerKey | ...       CustomerKey | CustomerName | ...
1        | 15          | ...       ...
2        | 8           | ...       8           | Trail Hut Ltd | ...
3        | 22          | ...       ...
                                   15          | Fundy Bay Outfitters | ...
                                   ...
                                   22          | Cape Breton Outdoor | ...

After INNER JOIN ON fact.Sales.CustomerKey = dim.Customer.CustomerKey:
SalesKey | CustomerKey | CustomerName          | LineTotal | ...
1        | 15          | Fundy Bay Outfitters  | 499.98    | ...
2        | 8           | Trail Hut Ltd         | 84.99     | ...
3        | 22          | Cape Breton Outdoor   | 1199.97   | ...
```

The join finds row 1 in `fact.Sales` (CustomerKey = 15), looks up CustomerKey = 15 in `dim.Customer`, finds 'Fundy Bay Outfitters', and combines the two rows.

---

3. Table Aliases

Multi-table queries become unwieldy without aliases. A **table alias** is a shorthand name for a table, defined in the `FROM` or `JOIN` clause and used throughout the query.

3.1 Defining and Using Aliases

```
-- Without aliases: verbose and hard to read
SELECT  fact.Sales.LineTotal,
        dim.Customer.CustomerName
FROM    fact.Sales
INNER JOIN dim.Customer ON dim.Customer.CustomerKey = fact.Sales.CustomerKey;

-- With aliases: concise and readable
SELECT  fs.LineTotal,
        cust.CustomerName
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey;
```

The alias is defined by writing it immediately after the table name in `FROM` or `JOIN`. There is no `AS` keyword required (though `FROM fact.Sales AS fs` also works — `AS` is optional for table aliases).

3.2 Alias Conventions

Good aliases are short but meaningful:

Table	Common alias
`fact.Sales`	`fs`
`dim.Customer`	`cust`
`dim.Product`	`prod`
`dim.Calendar`	`cal`
`dim.Employee`	`emp`
`dim.DeliveryMethod`	`dlv`

When joining the same table multiple times (covered in section 10), you must use different aliases for each instance: `cal1`, `cal2` or `ord_date`, `inv_date`.

3.3 Aliases Are Required for Ambiguous Columns

When two joined tables have columns with the same name, you must use aliases to specify which table's column you want:

```
-- Both fact.Sales and dim.Customer have a column called CustomerKey
-- Without alias qualification, SQL Server raises an error:
-- "Ambiguous column name 'CustomerKey'"

SELECT  CustomerKey  -- ERROR: which table?
FROM    fact.Sales
INNER JOIN dim.Customer ON dim.Customer.CustomerKey = fact.Sales.CustomerKey;

-- Correct: qualify with alias
SELECT  fs.CustomerKey,     -- fact.Sales version
        cust.CustomerName   -- dim.Customer version
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey;
```

The professional habit is to **always qualify column names with table aliases** in multi-table queries — not just for ambiguous columns. It makes queries self-documenting: a reader can immediately see which table every column comes from.

---

4. INNER JOIN: Matching Rows Only

The `INNER JOIN` is the most common join type. It returns only rows where a match is found in both tables. Rows from either table that have no match in the other are excluded.

4.1 Syntax

```
SELECT  columns
FROM    TableA alias_a
INNER JOIN TableB alias_b ON alias_b.key = alias_a.key;
```

The keyword `INNER` is technically optional — `JOIN` alone defaults to `INNER JOIN`. However, writing `INNER JOIN` explicitly is better practice: it communicates your intent and makes the join type visible when reading the query.

4.2 A Complete INNER JOIN Example

```
-- Which products were sold, and what category are they in?
USE CabotTrailOutdoorsSales;

SELECT  prod.ProductName,
        prod.CategoryName,
        fs.OrderedQuantity,
        fs.UnitPrice,
        fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
ORDER BY prod.CategoryName, fs.LineTotal DESC;
```

This returns 16,359 rows — one per invoice line. Each row combines a `fact.Sales` row with the matching `dim.Product` row, giving both the transaction measures (quantity, price, total) and the product descriptions (name, category).

4.3 Adding a WHERE Clause

`WHERE` filters rows *after* the join is applied:

```
-- Sales of Sleeping products only
SELECT  prod.ProductName,
        prod.CategoryName,
        fs.OrderedQuantity,
        fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
WHERE   prod.CategoryName = 'Sleeping'
ORDER BY fs.LineTotal DESC;
```

The sequence: SQL Server joins the tables first, producing the combined row set, then filters with `WHERE`. You can filter on columns from either table.

4.4 What INNER JOIN Excludes

The exclusion behaviour of `INNER JOIN` is important to understand. If `fact.Sales` has a row with `ProductKey = 999` and there is no row in `dim.Product` with `ProductKey = 999`, that `fact.Sales` row is silently excluded from the result.

In a well-designed dimensional model, this should not happen — every FK value in the fact table has a corresponding PK in the dimension (referential integrity). But if data quality is imperfect, `INNER JOIN` can silently drop rows. This is one reason why reconciliation tests (comparing row counts between source and joined result) matter.

---

5. Joining More Than Two Tables

Most analytical queries join three, four, or more tables. Each additional `JOIN` adds another connection to the query.

5.1 Chaining JOINs

Each `JOIN` clause adds one more table. They are evaluated left to right, with each join extending the result set of all previous joins:

```
-- Sales with customer AND product information
SELECT  cust.CustomerName,
        cust.SalesTerritory,
        prod.ProductName,
        prod.CategoryName,
        fs.OrderedQuantity,
        fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Product  prod ON prod.ProductKey  = fs.ProductKey
ORDER BY cust.SalesTerritory, prod.CategoryName;
```

SQL Server processes this as:

1. Join `fact.Sales` to `dim.Customer` on `CustomerKey`
2. Join the result to `dim.Product` on `ProductKey`
3. Apply `ORDER BY`
4. Return the result

5.2 Adding the Calendar Dimension

```
-- Sales with customer, product, and date
SELECT  cal.FullDate,
        cal.MonthName,
        cal.CalendarYear,
        cust.CustomerName,
        prod.ProductName,
        prod.CategoryName,
        fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Calendar  cal  ON cal.DateKey   = fs.OrderDateKey
INNER JOIN dim.Customer  cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Product   prod ON prod.ProductKey  = fs.ProductKey
WHERE   cal.CalendarYear = 2024
ORDER BY cal.FullDate, cust.CustomerName;
```

Notice that the `WHERE` clause now filters on `cal.CalendarYear` — a column from `dim.Calendar`, not from `fact.Sales`. Once a table is joined, all of its columns are available in `WHERE`, `ORDER BY`, and `SELECT`.

---

6. The Star Schema Join Pattern

The CabotTrail Sales data mart is a star schema — a central fact table surrounded by dimension tables. Querying it effectively means understanding the full star join pattern.

6.1 The Complete Five-Dimension Join

```
-- The full star join: fact.Sales connected to all five dimensions
SELECT
    -- Date dimension
    cal.FullDate                    AS OrderDate,
    cal.MonthName,
    cal.CalendarYear,
    -- Customer dimension
    cust.CustomerName,
    cust.SalesTerritory,
    -- Product dimension
    prod.ProductName,
    prod.CategoryName,
    -- Employee dimension
    emp.FullName                    AS SalesRep,
    emp.Department,
    -- Delivery method dimension
    dlv.DeliveryMethodName,
    -- Fact measures
    fs.OrderedQuantity,
    fs.UnitPrice,
    fs.LineTotal,
    fs.GrossProfit,
    fs.GrossProfitMarginPct
FROM    fact.Sales fs
INNER JOIN dim.Calendar       cal  ON cal.DateKey          = fs.OrderDateKey
INNER JOIN dim.Customer       cust ON cust.CustomerKey     = fs.CustomerKey
INNER JOIN dim.Product        prod ON prod.ProductKey      = fs.ProductKey
INNER JOIN dim.Employee       emp  ON emp.EmployeeKey      = fs.EmployeeKey
INNER JOIN dim.DeliveryMethod dlv  ON dlv.DeliveryMethodKey = fs.DeliveryMethodKey
ORDER BY cal.FullDate, cust.CustomerName;
```

This is the foundational analytical query for the Sales data mart. Once you have this structure, you can answer virtually any question about CabotTrail's sales by adjusting the `SELECT` columns, the `WHERE` filters, and (in Chapter 6) adding `GROUP BY` aggregation.

6.2 Reading the Star Join

The visual pattern of the star join query mirrors the star diagram:
- `fact.Sales` is always in the `FROM` clause — the centre of the star
- Each `INNER JOIN` connects one spoke — one dimension — to the centre
- The `ON` condition matches the FK on the fact to the PK on the dimension
- The `SELECT` list picks columns from wherever they are most useful

6.3 Selective Joins: You Do Not Have to Join Everything

You only need to join dimensions whose columns you actually use. If a query only needs date and product information, there is no reason to join `dim.Customer`, `dim.Employee`, and `dim.DeliveryMethod`:

```
-- Monthly revenue by product category — no need for Customer or Employee joins
SELECT
    cal.MonthName,
    cal.MonthNumber,
    cal.CalendarYear,
    prod.CategoryName,
    COUNT(*)               AS SalesLines,
    SUM(fs.LineTotal)      AS TotalRevenue
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal  ON cal.DateKey  = fs.OrderDateKey
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
WHERE   cal.CalendarYear = 2024
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName, prod.CategoryName
ORDER BY cal.MonthNumber, prod.CategoryName;
```

> *This query uses `GROUP BY` and aggregate functions — covered in Chapter 6. Run it now to see the result, even if the aggregation syntax is new.*

Unnecessary joins add processing overhead and can introduce unexpected row multiplication if the join condition is not tight. Join only what you need.

---

7. LEFT JOIN: Keep All Rows from the Left

The `INNER JOIN` excludes rows with no match. The `LEFT JOIN` is different: it keeps all rows from the *left* table and adds columns from the *right* table where a match exists. Where there is no match, the right table's columns appear as `NULL`.

7.1 Syntax

```
SELECT  columns
FROM    TableA alias_a              -- Left table: ALL rows kept
LEFT JOIN TableB alias_b            -- Right table: NULL where no match
    ON alias_b.key = alias_a.key;
```

`LEFT OUTER JOIN` and `LEFT JOIN` are synonyms. `OUTER` is optional and rarely written.

7.2 LEFT JOIN in Practice

The most common use of `LEFT JOIN` in analytical work: "show everything in this dimension, even if it has no transactions."

```
-- All products, whether or not they have been sold
-- Products with no sales will show NULL for fact columns
SELECT  prod.ProductName,
        prod.CategoryName,
        prod.UnitPrice,
        COUNT(fs.SalesKey)      AS TimesSold,
        SUM(fs.LineTotal)       AS TotalRevenue
FROM    dim.Product prod
LEFT JOIN fact.Sales fs ON fs.ProductKey = prod.ProductKey
GROUP BY prod.ProductName, prod.CategoryName, prod.UnitPrice
ORDER BY TimesSold ASC;
-- Products with zero sales appear at the top (COUNT returns 0 for no matches)
```

Notice the table order is reversed from the previous examples: `dim.Product` is in the `FROM` clause (the left table) and `fact.Sales` is in the `LEFT JOIN` (the right table). This ensures all products are kept, with sales data added where available.

7.3 Visualising LEFT JOIN

```
dim.Product (left — all kept):     fact.Sales (right — NULL where no match):
ProductKey | ProductName           ProductKey | SalesKey | LineTotal
76         | Sleeping Bag          76         | 1        | 499.98
83         | Tent                  76         | 2        | 499.98
99         | Widget (never sold)   83         | 3        | 1199.97

After LEFT JOIN:
ProductKey | ProductName   | SalesKey | LineTotal
76         | Sleeping Bag  | 1        | 499.98
76         | Sleeping Bag  | 2        | 499.98
83         | Tent          | 3        | 1199.97
99         | Widget        | NULL     | NULL    ← kept with NULLs
```

The Widget product has no sales — its `SalesKey` and `LineTotal` are `NULL`, but the product row itself is preserved.

7.4 Using LEFT JOIN to Find Non-Matches

A common analytical pattern: find rows in one table that have no corresponding rows in another. The technique is `LEFT JOIN` + `WHERE right_table.key IS NULL`:

```
-- Which products have never been sold?
SELECT  prod.ProductName,
        prod.CategoryName,
        prod.UnitPrice
FROM    dim.Product prod
LEFT JOIN fact.Sales fs ON fs.ProductKey = prod.ProductKey
WHERE   fs.SalesKey IS NULL     -- NULL SalesKey means no matching sales row
ORDER BY prod.CategoryName, prod.ProductName;
```

The logic: after the `LEFT JOIN`, products with no sales have `NULL` for every `fact.Sales` column. Filtering `WHERE fs.SalesKey IS NULL` isolates exactly those no-match rows.

This pattern — left join plus NULL check — is often more readable than a subquery with `NOT IN` or `NOT EXISTS`, and it performs well.

---

8. Choosing Between INNER JOIN and LEFT JOIN

The join type should reflect the analytical question:

Question type	Join to use	Why
"What did [X] do?"	`INNER JOIN`	Only interested in X entities that have activity
"Show all [X], with [Y] where available"	`LEFT JOIN`	All X entities, regardless of Y activity
"Which [X] have no [Y]?"	`LEFT JOIN` + `WHERE Y IS NULL`	Explicitly finding non-matches
"Which [X] have at least one [Y]?"	`INNER JOIN`	The inner join naturally excludes X with no Y

8.1 A Concrete Decision

Question A: "What revenue did each customer generate in 2024?"

Answer: `INNER JOIN` — you only care about customers who actually bought something in 2024. Customers with no 2024 purchases are irrelevant to this question.

```
SELECT  cust.CustomerName,
        SUM(fs.LineTotal)   AS Revenue2024
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Calendar  cal ON cal.DateKey       = fs.OrderDateKey
WHERE   cal.CalendarYear = 2024
GROUP BY cust.CustomerName
ORDER BY Revenue2024 DESC;
```

Question B: "Show all customers and their 2024 revenue — including customers with no 2024 purchases (show $0 for them)."

Answer: `LEFT JOIN` from `dim.Customer` — you want all customers regardless of purchase activity:

```
SELECT  cust.CustomerName,
        ISNULL(SUM(fs.LineTotal), 0)    AS Revenue2024
FROM    dim.Customer cust
LEFT JOIN fact.Sales fs
    ON  fs.CustomerKey  = cust.CustomerKey
    AND fs.OrderDateKey BETWEEN 20240101 AND 20241231   -- Date filter in ON clause
GROUP BY cust.CustomerName
ORDER BY Revenue2024 DESC;
```

> ***Important technique:*** *The date filter is in the `ON` clause rather than the `WHERE` clause. A `WHERE` filter on a left-joined table's column converts the `LEFT JOIN` back into an `INNER JOIN` — because `WHERE` excludes `NULL` rows, and customers with no sales have `NULL` for all `fact.Sales` columns, including the date key.*
>
> *The rule: when filtering on the right table of a `LEFT JOIN`, put the filter in the `ON` clause, not the `WHERE` clause.*

---

9. Common Join Mistakes

9.1 The Cartesian Product: The Missing ON Clause

A **Cartesian product** (also called a cross join) occurs when a join condition is missing or incorrect, causing every row in one table to match every row in the other:

```
-- WRONG: missing ON clause
SELECT cust.CustomerName, prod.ProductName
FROM   dim.Customer cust, dim.Product prod;
-- Returns: 100 customers × 142 products = 14,200 rows
-- Every customer appears with every product — nonsensical
```

With the correct `ON` clause:

```
-- This has no natural FK relationship between Customer and Product
-- You cannot meaningfully join them directly — this query has no correct form
```

If you ever see a query return far more rows than expected — millions of rows from two small tables — suspect a missing or incorrect join condition.

9.2 Fan-Out: The Wrong Join Column

Fan-out occurs when a join multiplies rows unintentionally because the join column is not the unique primary key of the right table. It is more subtle than a Cartesian product:

```
-- INCORRECT: joining on a non-unique column
-- Suppose dim.Calendar had two rows for the same year (it does not, but imagine)
-- Any join on CalendarYear instead of DateKey would multiply fact rows
SELECT fs.InvoiceID, cal.FullDate
FROM   fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.YearNumber = YEAR(fs.OrderDate);
-- If cal has multiple rows per year (it has ~365), each fact row
-- matches every day in that year — 365x fan-out
```

```
-- CORRECT: join on the actual key
SELECT fs.InvoiceID, cal.FullDate
FROM   fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey;
-- Each fact row matches exactly one calendar row
```

Detection: Compare the row count of a joined query to the row count of the primary table. If the join produced more rows than the primary table, fan-out has occurred.

```
-- Detect fan-out: should return 16,359 for FactSales
SELECT COUNT(*) FROM fact.Sales;                          -- Source count
SELECT COUNT(*) FROM fact.Sales fs                        -- After join
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey;
-- If these match, no fan-out
```

9.3 Ambiguous Column Names

When joining tables with shared column names, failing to qualify column names produces an error:

```
-- Both tables have CustomerKey — this is ambiguous
SELECT CustomerKey, CustomerName, LineTotal
FROM   fact.Sales
INNER JOIN dim.Customer ON dim.Customer.CustomerKey = fact.Sales.CustomerKey;
-- Error: Ambiguous column name 'CustomerKey'

-- Solution: always qualify with alias
SELECT  fs.CustomerKey, cust.CustomerName, fs.LineTotal
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey;
```

The professional habit: qualify every column in multi-table queries, even those that are not ambiguous. It costs nothing and prevents errors when tables gain new columns.

9.4 Filtering Right-Table Columns in a LEFT JOIN's WHERE Clause

As noted in section 8.1, filtering a right-table column in `WHERE` converts a `LEFT JOIN` to an `INNER JOIN`:

```
-- WRONG: this returns only customers who have 2024 sales
-- (The WHERE clause excludes NULL rows from the left join)
SELECT  cust.CustomerName, fs.LineTotal
FROM    dim.Customer cust
LEFT JOIN fact.Sales fs ON fs.CustomerKey = cust.CustomerKey
WHERE   fs.OrderDateKey >= 20240101;   -- Excludes NULL OrderDateKey rows

-- RIGHT: put the date filter in the ON clause
SELECT  cust.CustomerName, fs.LineTotal
FROM    dim.Customer cust
LEFT JOIN fact.Sales fs
    ON  fs.CustomerKey  = cust.CustomerKey
    AND fs.OrderDateKey >= 20240101;   -- Customers with no 2024 sales still appear
```

This is perhaps the most common `LEFT JOIN` mistake. When you write `WHERE rightTable.column = value` after a `LEFT JOIN`, you have effectively written an `INNER JOIN`.

---

10. The Self-Join

A **self-join** joins a table to itself. It is used when rows within the same table have a relationship to other rows in the same table — typically hierarchical data (an employee's manager is also an employee) or comparison scenarios.

10.1 Syntax

The table is referenced twice, each time with a different alias:

```
SELECT  e.FullName          AS Employee,
        mgr.FullName        AS Manager
FROM    Application.Employees e
INNER JOIN Application.Employees mgr ON mgr.EmployeeID = e.ManagerID;
```

Here `e` and `mgr` both refer to the same `Employees` table, but SQL Server treats them as separate instances. `e` provides the employee row; `mgr` provides the manager row (found by looking up the employee's `ManagerID` as another employee's `EmployeeID`).

10.2 Self-Join in the CabotTrail Context

The CabotTrail dimension tables do not have hierarchical structures that require self-joins. But the pattern is common in OLTP databases with org charts, bill-of-materials structures, or category parent-child relationships.

For a demonstration, we can compare product prices within the same category — finding products whose price exceeds the average for their category:

```
-- Products priced higher than any other product in the same category
-- (An alternative to correlated subqueries — covered in Chapter 7)
SELECT  p1.ProductName,
        p1.CategoryName,
        p1.UnitPrice,
        p2.ProductName      AS CheaperProduct,
        p2.UnitPrice        AS CheaperPrice
FROM    dim.Product p1
INNER JOIN dim.Product p2
    ON  p2.CategoryName = p1.CategoryName   -- Same category
    AND p2.ProductKey   <> p1.ProductKey    -- Different product
    AND p2.UnitPrice    < p1.UnitPrice      -- p2 is cheaper than p1
ORDER BY p1.CategoryName, p1.UnitPrice DESC;
```

This shows every (expensive product, cheaper product) pair within the same category. It is not a typical daily-use query, but it illustrates that the self-join is simply a join with two aliases pointing to the same table.

---

11. Cross-Database Joins

SQL Server allows joining tables from different databases on the same server using **three-part naming**: `DatabaseName.SchemaName.TableName`.

11.1 Syntax

```
-- Join dim.Customer from the Sales datamart to DimCustomer in the DW
SELECT
    sm.CustomerName,
    sm.SalesTerritory,
    dw.CreditLimit
FROM    CabotTrailOutdoorsSales.dim.Customer sm
INNER JOIN CabotTrailOutdoorDW.Dimension.DimCustomer dw
    ON dw.CustomerID = sm.CustomerID
WHERE   sm.ProvinceCode = 'NS'
ORDER BY sm.CustomerName;
```

11.2 When Cross-Database Joins Are Appropriate

Cross-database queries are useful for:

- **Reconciliation:** Comparing data in different databases (datamart vs DW)
- **Supplementing a query:** When a column needed for a report lives in a different database
- **ETL verification:** Confirming that data was correctly moved from source to target

Cross-database joins have performance implications — SQL Server must retrieve data across database boundaries. For large tables, it may be more efficient to bring the data into a single database first (using a staging table).

11.3 The Sales Datamart vs the Full DW

You will increasingly work with both `CabotTrailOutdoorsSales` (the simplified mart) and `CabotTrailOutdoorDW` (the full warehouse) as the course progresses. The schemas differ:

CabotTrailOutdoorsSales	CabotTrailOutdoorDW	Notes
`dim.Customer`	`Dimension.DimCustomer`	Same data, different schema/naming
`dim.Product`	`Dimension.DimProduct`	DW has a snowflake (more joins needed)
`fact.Sales`	`Fact.FactSales`	Same data, same structure
`dim.Calendar`	`Dimension.DimDate`	Similar content, some column differences

Three-part naming lets you use both in the same query.

---

12. Chapter Summary

- **JOINs** connect rows from two tables based on matching column values. They are the mechanism that makes the relational model useful — combining information stored in separate tables into a coherent result.
- Every join requires an `ON` clause that specifies the matching condition. The condition almost always connects a **foreign key** in one table to the **primary key** in another.
- **Table aliases** are shorthand names assigned in `FROM` and `JOIN` clauses. Always qualify column names with aliases in multi-table queries.
- **INNER JOIN** returns only rows where a match exists in both tables. Rows with no match are excluded from the result.
- **LEFT JOIN** keeps all rows from the left table, adding right-table columns where a match exists and `NULL` where there is none. Used when you want all rows from one table regardless of matches.
- **The star schema join pattern** places the fact table in `FROM` and each dimension table in an `INNER JOIN`. Join only the dimensions whose columns you actually need.
- **Common mistakes:** Cartesian products (missing `ON` clause), fan-out (joining on a non-unique column), ambiguous column names (missing alias qualification), and filtering a `LEFT JOIN`'s right table in `WHERE` instead of `ON`.
- **Self-joins** use two aliases on the same table to compare rows within the table — used for hierarchies and within-table comparisons.
- **Cross-database joins** use three-part naming (`Database.Schema.Table`) to join tables from different databases on the same server.

---

13. Review Questions

1. Explain in your own words why `fact.Sales` is not useful without joining dimension tables. What specific information is missing from the fact table alone, and where does it live?
2. Write a query that returns `CustomerName`, `SalesTerritory`, `DeliveryMethodName`, `ProductName`, `CategoryName`, `OrderDate` (from `dim.Calendar.FullDate`), and `LineTotal` for all sales in the Atlantic — Nova Scotia territory in 2023. Join all necessary dimensions and filter appropriately.
3. Write a query that shows all 50 employees and the number of sales transactions each has handled (from `fact.Sales`). Include employees with zero sales — they should appear with a count of 0. Use an appropriate alias for the count column. Sort by transaction count descending.

4. Explain the difference between placing a filter condition in the `ON` clause versus the `WHERE` clause of a `LEFT JOIN`. Write an example showing both versions and explain the different results they produce.
5. A developer writes this query and is surprised when it returns over 1.6 million rows instead of 16,359:
   `sql SELECT fs.InvoiceID, cal.FullDate, fs.LineTotal FROM fact.Sales fs INNER JOIN dim.Calendar cal ON cal.CalendarYear = YEAR(fs.OrderDate);`
   Explain the problem, calculate how many rows are expected, and write the correct query.
6. Write a query using a `LEFT JOIN` and a `WHERE IS NULL` filter to find all products in `dim.Product` that have never appeared in `fact.Sales`. Include `ProductName`, `CategoryName`, and `UnitPrice`. Sort by category then product name.
7. Write a query against `CabotTrailOutdoorDW` that joins `Fact.FactSales` to `Dimension.DimDate` (using `OrderDateKey = DateKey`) and `Dimension.DimProduct`. Return `FullDate`, `ProductName`, `CategoryName`, `OrderedQuantity`, and `LineTotal` for the month of March 2024. Use the DW's column naming conventions (`Dimension.DimDate.FullDate`, not `dim.Calendar.FullDate`).
8. Without running it, describe what this query returns and explain whether the join type is appropriate for the question being asked: *"Which delivery methods have never been used for an order over $500?"*
   `sql SELECT dlv.DeliveryMethodName, COUNT(fs.SalesKey) AS HighValueOrders FROM dim.DeliveryMethod dlv LEFT JOIN fact.Sales fs ON fs.DeliveryMethodKey = dlv.DeliveryMethodKey AND fs.LineTotal > 500 GROUP BY dlv.DeliveryMethodName ORDER BY HighValueOrders ASC;`

Deeper Dive

Going Further with JOINs

The Full JOIN Family

This chapter covered `INNER JOIN` and `LEFT JOIN` — the two you will use in 95% of analytical queries. SQL Server supports several others:

RIGHT JOIN: The mirror image of `LEFT JOIN` — keeps all rows from the right table, adds left table columns where a match exists. `RIGHT JOIN` is almost never needed in practice because you can always rewrite it as a `LEFT JOIN` by swapping the table order. Most teams adopt a convention of always using `LEFT JOIN` rather than mixing left and right.

FULL OUTER JOIN: Returns all rows from both tables. Rows with no match in the other table get NULLs for the missing side. Useful for comparing two datasets and finding rows that exist in one but not the other:

```
-- Find customers who exist in the Sales Mart but not the DW, and vice versa
SELECT
    sm.CustomerName     AS SalesMart_Name,
    dw.CustomerName     AS DW_Name
FROM    CabotTrailOutdoorsSales.dim.Customer sm
FULL OUTER JOIN CabotTrailOutdoorDW.Dimension.DimCustomer dw
    ON dw.CustomerID = sm.CustomerID
WHERE   sm.CustomerID IS NULL   -- In DW but not mart
OR      dw.CustomerID IS NULL;  -- In mart but not DW
```

CROSS JOIN: Returns every row from the left table combined with every row from the right table — a deliberate Cartesian product. Occasionally used for generating test data, date grids, or all possible combinations of two sets:

```
-- Generate all month/year combinations from the calendar
SELECT DISTINCT cal.YearNumber, m.MonthNumber
FROM   dim.Calendar cal
CROSS JOIN (VALUES (1),(2),(3),(4),(5),(6),(7),(8),(9),(10),(11),(12))
           AS m(MonthNumber)
ORDER BY cal.YearNumber, m.MonthNumber;
```

Microsoft documentation on join types:
FROM clause plus JOIN, APPLY, PIVOT

Join Algorithms: How SQL Server Executes Joins

Understanding how SQL Server *executes* joins helps you write faster queries. The query optimizer chooses one of three join algorithms based on table sizes, indexes, and statistics:

Nested Loop Join: For each row in the outer table, scan the inner table for matches. Efficient when the outer table is small and the inner table has an index on the join column. Common for joining a small result set to a large indexed table.

Hash Join: Build a hash table from the smaller table, then probe it with rows from the larger table. Efficient for large, unsorted tables with no useful indexes. Memory-intensive.

Merge Join: Sort both inputs on the join key, then scan them simultaneously. Extremely efficient when both inputs are already sorted (e.g., from an index). Requires sorted inputs — SQL Server may add a sort operation if they are not.

You can see which algorithm SQL Server chose using the **Execution Plan** (`Ctrl+M` to toggle in SSMS, or `Ctrl+L` for estimated plan). The icons for nested loop, hash match, and merge join tell you the algorithm used for each join in the query.

For analytical queries against the CabotTrail data mart (small dimension tables), nested loop joins are typical and fast. For large fact tables in production environments, the optimizer choice matters significantly.

The ANSI SQL Join Syntax vs the Old Comma Syntax

You may encounter older SQL code that uses commas in the `FROM` clause to join tables, with the join condition in the `WHERE` clause:

```
-- OLD-STYLE (implicit join — avoid this):
SELECT  fs.LineTotal, cust.CustomerName
FROM    fact.Sales fs, dim.Customer cust
WHERE   cust.CustomerKey = fs.CustomerKey
AND     fs.LineTotal > 500;

-- MODERN (explicit join — always prefer this):
SELECT  fs.LineTotal, cust.CustomerName
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
WHERE   fs.LineTotal > 500;
```

The comma syntax (implicit join) has two problems:
1. **Readability:** Join conditions and filter conditions are mixed in `WHERE` — it is easy to miss one and produce an accidental Cartesian product
2. **Explicit intent:** `INNER JOIN`, `LEFT JOIN`, and `FULL OUTER JOIN` cannot be expressed with the comma syntax; it only supports inner joins

The ANSI SQL join syntax (with explicit `JOIN` keywords) has been standard since SQL-92. All modern SQL code should use it. If you encounter the comma syntax in legacy code, consider rewriting it.

Hash JOIN and Performance on Large Tables

For very large fact tables — hundreds of millions of rows — join performance depends critically on proper indexing. The most important indexes for analytical joins:

On the fact table: Non-clustered indexes on all FK columns (`CustomerKey`, `ProductKey`, `OrderDateKey`, etc.). Without these, every join to the fact table requires a full table scan.

On the dimension tables: The primary key (surrogate key) should have a clustered or non-clustered unique index. SQL Server creates this automatically with a `PRIMARY KEY` constraint.

Columnstore indexes: For large fact tables (10M+ rows), a clustered columnstore index transforms analytical query performance — aggregations over many rows are typically 10–100x faster than with traditional row-store indexes.

The CabotTrail fact table at 16,359 rows is too small to benefit from any of this. But understanding these principles prepares you for production environments where join performance matters.

Industry Perspectives

JOINs Are the Core Skill

Data analyst job postings consistently list "strong SQL skills" as a requirement. When interviewers assess SQL proficiency, joins are the primary test — specifically:

- Can you write a correct `INNER JOIN`?
- Do you understand the difference between `INNER JOIN` and `LEFT JOIN`?
- Can you explain what happens when a join key has no match?
- Can you debug a query returning unexpected row counts?

The concepts in this chapter — join types, the ON clause, fan-out, the LEFT JOIN filter trap — are the subject of SQL interview questions at virtually every data role. Mastering them is a direct career investment.

The Star Schema and BI Tools

Power BI, Tableau, and other BI tools are designed around the star schema join pattern. When you connect Power BI to a star schema data mart, it can automatically detect the FK-to-PK relationships between the fact and dimension tables and propose join conditions. The visual report builder then translates user interactions (drag a field, apply a filter) into the equivalent `JOIN`-and-`WHERE` SQL queries.

Understanding star schema joins in SQL gives you the ability to verify and debug what BI tools are actually doing — an essential skill when a report produces unexpected results and you need to determine whether the issue is in the data or the report logic.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 3 covers joins comprehensively with clear visual examples of each join type.
2. Itzik Ben-Gan, Lubor Kollar, Dejan Sarka, & Ron Talmage. (2015). *T-SQL Querying*. Microsoft Press. — Chapters 1–2 cover joins and the relational model at depth, including physical join algorithms.
3. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — Chapter 1 explains the star schema join pattern from the designer's perspective, complementing the analyst's view in this chapter.
4. Microsoft. (2024). *FROM clause plus JOIN, APPLY, PIVOT (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/from-transact-sql
5. Microsoft. (2024). *Joins*. https://learn.microsoft.com/en-us/sql/relational-databases/performance/joins

6. Microsoft. (2024). *Understanding Merge Join Efficiency*. https://learn.microsoft.com/en-us/sql/relational-databases/query-processing-architecture-guide

Chapter 6: Aggregation — Summarising Data for Analysis

Chapter Overview

The previous two chapters gave you the tools to retrieve and connect detailed data — individual rows representing specific transactions, products, or customers. But analysis rarely lives at that level. A manager does not want to see 16,359 individual invoice lines — they want to know total revenue by territory, average margin by product category, or the number of orders per month.

Aggregation is the process of summarising many rows into fewer rows, computing measures like totals, averages, and counts along the way. In SQL, aggregation is controlled by two clauses: `GROUP BY` and `HAVING`, supported by a set of aggregate functions: `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX`.

This chapter is where querying starts to feel like analysis. By the end, you will be writing queries that answer real business questions — the kind of questions that appear in management dashboards, quarterly reports, and performance reviews.

By the end of this chapter you will be able to:

- Explain the difference between scalar functions and aggregate functions
- Use `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX` to summarise data
- Use `GROUP BY` to produce subtotals for each unique combination of grouping columns
- Use `HAVING` to filter the result of aggregation
- Explain the difference between `WHERE` and `HAVING` and apply each correctly
- Use `COUNT(DISTINCT column)` to count unique values
- Combine `JOIN` with aggregation to answer multi-table business questions
- Recognise and avoid the most common aggregation mistakes

Table of Contents

1. From Detail to Summary: Why Aggregation Matters
2. Aggregate Functions
3. COUNT: Counting Rows and Values
4. SUM: Totalling Numeric Values
5. AVG, MIN, MAX: Averages and Extremes
6. GROUP BY: Grouping Rows
7. GROUP BY with Multiple Columns
8. GROUP BY with JOINs
9. HAVING: Filtering Groups
10. WHERE vs HAVING: The Critical Distinction
11. Aggregation with CASE: Conditional Counts and Sums
12. NULL Behaviour in Aggregation
13. Putting It Together: Real Business Questions
14. Chapter Summary
15. Review Questions
16. Deeper Dive

---

1. From Detail to Summary: Why Aggregation Matters

The 16,359 rows in `fact.Sales` represent every individual invoice line CabotTrail has issued over four years. That level of detail is essential for certain questions — auditing a specific transaction, investigating a customer's order history, verifying a specific refund. But most analytical questions do not live at the invoice-line level.

Consider the difference:

A detail-level question: "What was the line total for invoice 10341, order line 14201?"
Answer: query a single row.

An analytical question: "Which product category generated the most revenue in 2024?"
Answer: sum all 2024 revenue for each category and compare.

The second question requires collapsing thousands of rows into a few summary rows — one per category — each representing the total of many individual transactions. That collapse is aggregation.

1.1 Aggregation Reduces Rows

This is the fundamental effect of aggregation: it reduces the number of rows in the result. A table with 16,359 rows can be aggregated into 13 rows (one per product category) or 50 rows (one per sales rep) or 48 rows (one per month across four years), depending on the grouping.

The aggregate functions perform the calculation during the collapse — summing the revenue, counting the orders, finding the maximum sale value — for each group.

1.2 The Relationship Between Scalar and Aggregate Functions

In Chapter 4, you used **scalar functions** — `LEN`, `UPPER`, `DATEDIFF` — that operate on each row independently and produce one output per input row. The row count of the result matches the row count of the input.

Aggregate functions are fundamentally different. They operate on a *set* of rows and return a *single value* for that entire set. `SUM(LineTotal)` does not return 16,359 values — it returns one. The distinction matters because aggregate and scalar functions cannot be mixed freely in a `SELECT` list without a `GROUP BY` clause.

---

2. Aggregate Functions

SQL Server's five core aggregate functions cover the most common summary operations:

Function	What it computes	Input	Notes
`COUNT(*)`	Number of rows	Any	Counts all rows including NULLs
`COUNT(column)`	Number of non-NULL values	Any column	Excludes NULLs
`COUNT(DISTINCT column)`	Number of unique values	Any column	Excludes NULLs and duplicates
`SUM(column)`	Total of all values	Numeric	Ignores NULLs
`AVG(column)`	Average of non-NULL values	Numeric	Ignores NULLs; uses column's data type
`MIN(column)`	Lowest value	Any comparable type	Works on numbers, dates, and strings
`MAX(column)`	Highest value	Any comparable type	Works on numbers, dates, and strings

2.1 Aggregate Functions Without GROUP BY

When used without a `GROUP BY` clause, aggregate functions treat the entire table as a single group and return one row:

```
-- Single-row summary of all sales
USE CabotTrailOutdoorsSales;

SELECT
    COUNT(*)                    AS TotalSalesLines,
    COUNT(DISTINCT InvoiceID)   AS TotalInvoices,
    SUM(LineTotal)              AS TotalRevenue,
    AVG(LineTotal)              AS AvgLineValue,
    MIN(LineTotal)              AS SmallestSale,
    MAX(LineTotal)              AS LargestSale,
    MIN(OrderDate)              AS EarliestOrder,
    MAX(OrderDate)              AS LatestOrder
FROM    fact.Sales;
```

Run this. It returns exactly one row — a summary of every row in `fact.Sales`. This is the simplest possible aggregation: no grouping, one result for the whole table.

3. COUNT: Counting Rows and Values

`COUNT` is the most frequently used aggregate function. Its three variants behave differently and the differences matter.

3.1 COUNT(*): Count All Rows

`COUNT(*)` counts every row in the group, including rows with NULL values in any column:

```
-- How many products do we have?
SELECT  COUNT(*)    AS TotalProducts
FROM    dim.Product;
-- Returns: 142
```

`COUNT(*)` is the standard way to count rows. The `*` does not mean "all columns" here — it means "count all rows regardless of what is in them."

3.2 COUNT(column): Count Non-NULL Values

`COUNT(column)` counts only rows where the specified column is not NULL:

```
-- How many products have a product code assigned?
SELECT  COUNT(*)            AS TotalProducts,
        COUNT(ProductCode)  AS ProductsWithCode,
        COUNT(*) - COUNT(ProductCode) AS ProductsWithoutCode
FROM    dim.Product;
```

If `ProductCode` is NULL for some products, `COUNT(ProductCode)` returns a smaller number than `COUNT(*)`. This is the standard way to find the count of non-NULL values in a column.

3.3 COUNT(DISTINCT column): Count Unique Values

`COUNT(DISTINCT column)` counts the number of unique, non-NULL values:

```
-- How many distinct customers have placed orders?
SELECT  COUNT(DISTINCT CustomerKey)     AS UniqueCustomers
FROM    fact.Sales;
-- Returns: up to 100 (the number of customers who appear in fact.Sales)
```

```
-- How many distinct invoices are there?
-- (Each invoice can have multiple lines — COUNT(*) counts lines, not invoices)
SELECT  COUNT(*)                AS TotalSalesLines,
        COUNT(DISTINCT InvoiceID) AS TotalInvoices
FROM    fact.Sales;
-- TotalSalesLines ≈ 16,359; TotalInvoices is smaller (multiple lines per invoice)
```

The distinction between `COUNT(*)`, `COUNT(column)`, and `COUNT(DISTINCT column)` answers three completely different questions. Choosing the wrong one produces wrong results that look plausible.

---

4. SUM: Totalling Numeric Values

`SUM(column)` adds all non-NULL values in the column across the group:

```
-- Total revenue, total gross profit, total tax
SELECT
    SUM(LineTotal)          AS TotalRevenue,
    SUM(GrossProfit)        AS TotalGrossProfit,
    SUM(TaxAmount)          AS TotalTax,
    SUM(OrderedQuantity)    AS TotalUnitsSold
FROM    fact.Sales;
```

4.1 SUM with a Calculated Expression

`SUM` can take an expression, not just a column:

```
-- Total revenue excluding tax (calculated inline)
SELECT
    SUM(LineTotal)                      AS RevenueIncTax,
    SUM(LineTotal - TaxAmount)          AS RevenueExcTax,
    SUM(LineTotalIncludingTax)          AS RevIncTaxStored,
    -- Verify: stored vs calculated
    SUM(LineTotal + TaxAmount) -
    SUM(LineTotalIncludingTax)          AS Discrepancy
FROM    fact.Sales;
-- Discrepancy should be 0.00 if the stored column is calculated correctly
```

4.2 SUM on Non-Additive Measures

Recall from the ETL for Business Intelligence textbook: some measures are **non-additive** — summing them produces meaningless results. `GrossProfitMarginPct` is one:

```
-- WRONG: summing a percentage produces nonsense
SELECT  SUM(GrossProfitMarginPct)   AS WrongTotalMargin
FROM    fact.Sales;
-- Returns a large number that has no business meaning

-- RIGHT: compute the overall margin from additive measures
SELECT
    SUM(GrossProfit)                        AS TotalGrossProfit,
    SUM(LineTotal)                          AS TotalRevenue,
    ROUND(SUM(GrossProfit)
          / NULLIF(SUM(LineTotal), 0) * 100, 2) AS OverallMarginPct
FROM    fact.Sales;
```

The non-additive principle: always derive percentage measures by summing the numerator and denominator separately, then dividing.

---

5. AVG, MIN, MAX: Averages and Extremes

5.1 AVG: The Average

`AVG(column)` computes the arithmetic mean of non-NULL values. It uses the same data type as the input column — a potential issue when the column is an integer:

```
-- Average line total
SELECT  AVG(LineTotal)  AS AvgLineTotal
FROM    fact.Sales;
-- LineTotal is DECIMAL(18,2), so the result is also DECIMAL -- safe

-- Integer division trap with AVG
SELECT  AVG(OrderedQuantity)    AS AvgQuantity
FROM    fact.Sales;
-- OrderedQuantity may be INT -- result will be an integer (truncated)
-- Fix: CAST to DECIMAL
SELECT  AVG(CAST(OrderedQuantity AS DECIMAL(10,2)))  AS AvgQuantity
FROM    fact.Sales;
```

The same integer division behaviour from Chapter 3 applies to `AVG` — if the column is `INT`, the average is an integer (truncated, not rounded). Always check the data type and cast if needed.

5.2 MIN and MAX: Extremes

`MIN` and `MAX` work on any comparable data type — numbers, dates, and strings (alphabetically):

```
-- Price range
SELECT
    MIN(UnitPrice)          AS CheapestProduct,
    MAX(UnitPrice)          AS MostExpensiveProduct,
    MAX(UnitPrice) - MIN(UnitPrice) AS PriceRange
FROM    dim.Product
WHERE   IsDiscontinued = 0;
```

```
-- Date range of orders
SELECT
    MIN(OrderDate)  AS FirstOrder,
    MAX(OrderDate)  AS LatestOrder,
    DATEDIFF(day, MIN(OrderDate), MAX(OrderDate)) AS DaySpan
FROM    fact.Sales;
```

```
-- First and last customer name alphabetically
SELECT
    MIN(CustomerName)   AS FirstAlphabetically,
    MAX(CustomerName)   AS LastAlphabetically
FROM    dim.Customer;
```

`MAX` and `MIN` on strings use alphabetical ordering — useful for finding the first/last value in a sorted order, though typically you want to pair this with `GROUP BY` to find the min/max per group.

---

6. GROUP BY: Grouping Rows

A `COUNT(*)` against the entire table gives you one total. `GROUP BY` gives you that total broken down by a category — one row per unique value in the grouping column.

6.1 What GROUP BY Does

`GROUP BY` divides rows into groups based on unique combinations of the specified columns, then applies aggregate functions within each group:

```
-- How many products are in each category?
SELECT  CategoryName,
        COUNT(*)    AS ProductCount
FROM    dim.Product
GROUP BY CategoryName
ORDER BY ProductCount DESC;
```

SQL Server processes this as:

1. Read all rows from `dim.Product`
2. Group them by `CategoryName` — rows with the same category are placed together

3. For each group, compute `COUNT(*)` — count the rows in the group
4. Return one row per group with the `CategoryName` and `COUNT(*)`

The result has 13 rows — one per unique category — instead of 142.

6.2 The GROUP BY Rule

Every column in the SELECT list must either be in the GROUP BY clause or be inside an aggregate function. This rule is absolute. Breaking it causes an error:

```
-- ERROR: ProductName is not in GROUP BY or an aggregate function
SELECT  CategoryName, ProductName, COUNT(*)
FROM    dim.Product
GROUP BY CategoryName;
-- Error: Column 'dim.Product.ProductName' is invalid in the select list
-- because it is not contained in either an aggregate function or the GROUP BY clause.
```

The reason: when rows are grouped, there are multiple product names per category group. SQL Server cannot return a single value for `ProductName` when there are many options — it has no basis to choose.

The fix: Add `ProductName` to `GROUP BY` (which changes the grain to one row per category+product combination) or remove it from the SELECT list.

6.3 GROUP BY on a Dimension Table

```
-- Revenue by product category
SELECT  prod.CategoryName,
        COUNT(*)                AS SalesLines,
        SUM(fs.LineTotal)       AS TotalRevenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
ORDER BY TotalRevenue DESC;
```

This returns 13 rows — one per category. Each row shows how many sales lines and how much total revenue that category generated. The `INNER JOIN` is applied before grouping — SQL Server joins the tables, then groups the joined result.

---

7. GROUP BY with Multiple Columns

`GROUP BY` accepts multiple columns, creating one group per unique combination of all the specified columns:

```
-- Revenue by category AND calendar year
SELECT  cal.CalendarYear,
        prod.CategoryName,
        COUNT(*)                AS SalesLines,
        SUM(fs.LineTotal)       AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal  ON cal.DateKey    = fs.OrderDateKey
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
GROUP BY cal.CalendarYear, prod.CategoryName
ORDER BY cal.CalendarYear, Revenue DESC;
```

This produces one row per (Year, Category) combination — up to 52 rows (13 categories × 4 years, minus any combinations with no sales).

7.1 The Grouping Grain

The columns in `GROUP BY` define the **grain** of the result — what one row in the output represents. In the example above, each output row represents "sales in category X during year Y." This is analogous to the grain concept in dimensional modelling.

Changing the `GROUP BY` columns changes the grain and the meaning of the result:

```
-- Grain: one row per year
GROUP BY cal.CalendarYear

-- Grain: one row per year + category combination
GROUP BY cal.CalendarYear, prod.CategoryName

-- Grain: one row per year + category + territory combination
GROUP BY cal.CalendarYear, prod.CategoryName, cust.SalesTerritory
```

Each additional grouping column subdivides the groups further, producing more rows and finer-grained results.

7.2 Ordering Grouped Results

`ORDER BY` applies after grouping and can reference grouping columns or aggregate expressions:

```
-- Year/month revenue, sorted chronologically
SELECT  cal.CalendarYear,
        cal.MonthNumber,
        cal.MonthName,
        SUM(fs.LineTotal)   AS MonthlyRevenue
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName
ORDER BY cal.CalendarYear, cal.MonthNumber;  -- Chronological order
```

Notice `MonthNumber` is in `GROUP BY` even though it does not appear in `SELECT`. This is valid — `GROUP BY` columns do not have to be in `SELECT`. `MonthNumber` is included to enable correct chronological sorting (sorting by `MonthName` alphabetically would put April before August, which is not chronological).

---

8. GROUP BY with JOINs

Combining `GROUP BY` with `JOIN` is where the most powerful analytical queries live. The pattern:

1. Join the fact table to the needed dimension tables
2. Filter with `WHERE` if needed
3. Group by dimension attributes
4. Compute aggregate measures
5. Sort the result

8.1 Revenue by Sales Representative

```
-- Which sales rep generated the most revenue in 2024?
SELECT  emp.FullName                        AS SalesRep,
        emp.Department,
        COUNT(DISTINCT fs.InvoiceID)        AS InvoiceCount,
        COUNT(*)                            AS SalesLineCount,
        SUM(fs.LineTotal)                   AS TotalRevenue,
        ROUND(AVG(fs.LineTotal), 2)         AS AvgLineValue
FROM    fact.Sales fs
INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
INNER JOIN dim.Calendar  cal ON cal.DateKey    = fs.OrderDateKey
WHERE   cal.CalendarYear = 2024
GROUP BY emp.FullName, emp.Department
ORDER BY TotalRevenue DESC;
```

8.2 Monthly Revenue Trend

```
-- Monthly revenue for all years — year-on-year comparison base
SELECT  cal.CalendarYear,
        cal.MonthNumber,
        cal.MonthName,
        COUNT(DISTINCT fs.InvoiceID)        AS Invoices,
        SUM(fs.LineTotal)                   AS Revenue,
        SUM(fs.GrossProfit)                 AS GrossProfit,
        ROUND(SUM(fs.GrossProfit)
              / NULLIF(SUM(fs.LineTotal), 0) * 100, 2) AS MarginPct
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName
ORDER BY cal.CalendarYear, cal.MonthNumber;
```

8.3 Customer Performance Summary

```
-- Top customers by revenue with territory and category breadth
SELECT  cust.CustomerName,
        cust.SalesTerritory,
        COUNT(DISTINCT fs.InvoiceID)            AS TotalInvoices,
        COUNT(DISTINCT prod.CategoryName)        AS CategoriesPurchased,
        SUM(fs.LineTotal)                        AS TotalRevenue,
        MIN(cal.FullDate)                        AS FirstOrder,
        MAX(cal.FullDate)                        AS LastOrder
FROM    fact.Sales fs
INNER JOIN dim.Customer  cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Product   prod ON prod.ProductKey  = fs.ProductKey
INNER JOIN dim.Calendar  cal  ON cal.DateKey      = fs.OrderDateKey
GROUP BY cust.CustomerName, cust.SalesTerritory
ORDER BY TotalRevenue DESC;
```

`COUNT(DISTINCT prod.CategoryName)` counts how many different product categories each customer has purchased from — a useful measure of customer breadth.

---

9. HAVING: Filtering Groups

`WHERE` filters individual rows before grouping. `HAVING` filters groups *after* grouping. It is the mechanism for conditions that involve aggregate values.

9.1 Syntax

```
SELECT  grouping_column, AGG_FUNCTION(measure)
FROM    table
WHERE   row_level_condition
GROUP BY grouping_column
HAVING  aggregate_condition
ORDER BY something;
```

9.2 A First HAVING Example

```
-- Categories with total revenue exceeding $500,000
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)   AS TotalRevenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
HAVING  SUM(fs.LineTotal) > 500000
ORDER BY TotalRevenue DESC;
```

The `HAVING` clause filters out any category whose total revenue is $500,000 or less. Only high-revenue categories appear in the result.

9.3 HAVING with COUNT

```
-- Sales reps who handled more than 500 invoice lines
SELECT  emp.FullName,
        COUNT(*)    AS SalesLineCount
FROM    fact.Sales fs
INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
GROUP BY emp.FullName
HAVING  COUNT(*) > 500
ORDER BY SalesLineCount DESC;
```

```
-- Customers who have purchased products from at least 5 different categories
SELECT  cust.CustomerName,
        COUNT(DISTINCT prod.CategoryName)   AS CategoriesOrdered
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Product  prod ON prod.ProductKey  = fs.ProductKey
GROUP BY cust.CustomerName
HAVING  COUNT(DISTINCT prod.CategoryName) >= 5
ORDER BY CategoriesOrdered DESC;
```

9.4 HAVING with a Computed Expression

`HAVING` can contain any expression that could appear in an aggregate context:

```
-- Categories where average gross profit margin is below 35%
SELECT  prod.CategoryName,
        ROUND(SUM(fs.GrossProfit)
              / NULLIF(SUM(fs.LineTotal), 0) * 100, 2)  AS AvgMarginPct
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
HAVING  SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100 < 35
ORDER BY AvgMarginPct;
```

Notice the `HAVING` clause cannot reference the alias `AvgMarginPct` — for the same reason `WHERE` cannot reference aliases (evaluation order). The expression must be repeated.

---

10. WHERE vs HAVING: The Critical Distinction

This distinction trips up analysts at every level of experience. A clear mental model prevents most errors.

10.1 The Rule

`WHERE` filters rows before they are grouped. It operates on individual row values — column values, not aggregate values.

`HAVING` filters groups after they are formed. It operates on group results — aggregate values like `SUM(column)` or `COUNT(*)`.

Clause	When it runs	What it filters	Can reference
`WHERE`	Before `GROUP BY`	Individual rows	Column values
`HAVING`	After `GROUP BY`	Groups	Aggregate values

10.2 A Side-by-Side Comparison

Question A: "What is the total revenue per category, but only for sales in 2024?"

`WHERE` is correct — you want to restrict which *rows* are included in the aggregation:

```
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)   AS Revenue2024
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
WHERE   cal.CalendarYear = 2024      -- Row filter: only 2024 rows enter the aggregation
GROUP BY prod.CategoryName
ORDER BY Revenue2024 DESC;
```

Question B: "Which categories had more than $200,000 in revenue in 2024?"

Both `WHERE` and `HAVING` needed — `WHERE` restricts to 2024 rows; `HAVING` restricts to high-revenue groups:

```
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)   AS Revenue2024
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
WHERE   cal.CalendarYear = 2024          -- Row filter first
GROUP BY prod.CategoryName
HAVING  SUM(fs.LineTotal) > 200000       -- Group filter after
ORDER BY Revenue2024 DESC;
```

10.3 The Performance Reason to Use WHERE Over HAVING

Beyond correctness, `WHERE` is more efficient than `HAVING` for row-level filters. `WHERE` reduces the number of rows before grouping — fewer rows means less work in the aggregation step. `HAVING` filters after grouping — all the aggregation work is done before any rows are discarded.

```
-- SLOW: groups all data, then discards most groups
SELECT  CategoryName, SUM(LineTotal)
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
HAVING  prod.CategoryName = 'Sleeping';  -- Should have been WHERE

-- FAST: filters rows first, then groups only 'Sleeping' rows
SELECT  prod.CategoryName, SUM(fs.LineTotal)
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
WHERE   prod.CategoryName = 'Sleeping'
GROUP BY prod.CategoryName;
```

The rule: **always filter on non-aggregate conditions with `WHERE`, not `HAVING`.**

---

11. Aggregation with CASE: Conditional Counts and Sums

One of the most powerful patterns in analytical SQL: embedding a `CASE` expression inside an aggregate function to produce conditional counts and sums in a single query.

11.1 Conditional COUNT with CASE

```
-- Count active and discontinued products by category in one query
SELECT  CategoryName,
        COUNT(*)                                              AS TotalProducts,
        COUNT(CASE WHEN IsDiscontinued = 0 THEN 1 END)     AS ActiveProducts,
        COUNT(CASE WHEN IsDiscontinued = 1 THEN 1 END)     AS DiscontinuedProducts
FROM    dim.Product
GROUP BY CategoryName
ORDER BY CategoryName;
```

How this works: `COUNT(CASE WHEN IsDiscontinued = 0 THEN 1 END)` returns `1` for active products and `NULL` for discontinued ones. Since `COUNT` ignores NULLs, it effectively counts only active products.

11.2 Conditional SUM with CASE

```
-- Revenue split by delivery method type
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)                                          AS TotalRevenue,
        SUM(CASE WHEN dlv.DeliveryMethodName = 'Road Freight'
                 THEN fs.LineTotal ELSE 0 END)                     AS RoadFreightRevenue,
        SUM(CASE WHEN dlv.DeliveryMethodName <> 'Road Freight'
                 THEN fs.LineTotal ELSE 0 END)                     AS OtherDeliveryRevenue
FROM    fact.Sales fs
INNER JOIN dim.Product       prod ON prod.ProductKey        = fs.ProductKey
INNER JOIN dim.DeliveryMethod dlv  ON dlv.DeliveryMethodKey = fs.DeliveryMethodKey
GROUP BY prod.CategoryName
ORDER BY TotalRevenue DESC;
```

11.3 Year-Over-Year Comparison in One Query

This pattern is particularly valuable for producing comparison reports without complex joins:

```
-- Compare annual revenue side by side
SELECT  prod.CategoryName,
        SUM(CASE WHEN cal.CalendarYear = 2022 THEN fs.LineTotal ELSE 0 END) AS Rev_2022,
        SUM(CASE WHEN cal.CalendarYear = 2023 THEN fs.LineTotal ELSE 0 END) AS Rev_2023,
        SUM(CASE WHEN cal.CalendarYear = 2024 THEN fs.LineTotal ELSE 0 END) AS Rev_2024,
        SUM(CASE WHEN cal.CalendarYear = 2025 THEN fs.LineTotal ELSE 0 END) AS Rev_2025,
        SUM(fs.LineTotal)                                                    AS TotalAllY
ears
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
GROUP BY prod.CategoryName
ORDER BY TotalAllYears DESC;
```

This pivots years into columns — a technique called a **pivot** — using only `GROUP BY` and `CASE`. No special SQL syntax required. The result is a summary table that is immediately useful for year-over-year analysis.

---

12. NULL Behaviour in Aggregation

Aggregate functions handle NULLs in a specific way that is worth understanding explicitly.

12.1 Aggregate Functions Ignore NULLs

All aggregate functions except `COUNT(*)` silently ignore NULL values:

```
-- Demonstrate NULL behaviour in aggregation
-- If a column has NULLs, SUM, AVG, MIN, MAX ignore those rows

SELECT
    COUNT(*)                    AS TotalRows,       -- Counts all rows including NULLs
    COUNT(ProductCode)          AS NonNullCodes,    -- Counts only non-NULL ProductCode
    AVG(UnitPrice)              AS AvgPrice         -- Average of non-NULL UnitPrice
values
FROM    dim.Product;
```

For `SUM` and `AVG`, ignoring NULLs is usually the correct behaviour — a NULL price should not contribute to a total or average. But for `COUNT`, the distinction between `COUNT(*)` and `COUNT(column)` can produce very different results when NULLs are present.

12.2 NULL Groups in GROUP BY

A NULL value in a grouping column creates its own group:

```
-- If SalesTerritory were NULL for some customers,
-- those customers would form a NULL group

SELECT  SalesTerritory,
        COUNT(*)        AS CustomerCount
FROM    dim.Customer
GROUP BY SalesTerritory
ORDER BY CustomerCount DESC;
-- A NULL SalesTerritory would appear as a separate row with NULL label
```

In CabotTrail's `dim.Customer`, `SalesTerritory` is populated for all customers — no NULL group. But in real-world data, NULL groups are common and easy to miss unless you look for them.

12.3 AVG and NULL: A Subtle Trap

Because `AVG` ignores NULLs, the average is computed over fewer rows than the total — which may not match the "total divided by count" you might expect:

```
-- Columns with NULLs: AVG ignores them but COUNT(*) does not
-- The apparent average (SUM/COUNT(*)) differs from AVG(column)

SELECT
    COUNT(*)                        AS TotalRows,
    COUNT(TypicalWeightPerUnit)     AS NonNullRows,
    AVG(TypicalWeightPerUnit)       AS AvgWeight_IgnoresNull,
    SUM(TypicalWeightPerUnit)
        / COUNT(*)                  AS ApparentAvg_IncludesNull
FROM    dim.Product
WHERE   TypicalWeightPerUnit IS NOT NULL OR 1 = 1; -- See all rows
```

`AVG` divides the sum by the count of non-NULL values. If you divide `SUM` by `COUNT(*)` (which includes NULLs), you get a different — and arguably wrong — average.

---

13. Putting It Together: Real Business Questions

This section answers five analytical questions that combine everything in this chapter — and in the course so far.

13.1 Territory Performance Report

```
-- Revenue, invoice count, average order value, and margin by territory
SELECT  cust.SalesTerritory,
        COUNT(DISTINCT fs.InvoiceID)               AS Invoices,
        COUNT(*)                                   AS SalesLines,
        SUM(fs.LineTotal)                          AS TotalRevenue,
        ROUND(SUM(fs.LineTotal)
              / NULLIF(COUNT(DISTINCT fs.InvoiceID), 0), 2) AS AvgInvoiceValue,
        ROUND(SUM(fs.GrossProfit)
              / NULLIF(SUM(fs.LineTotal), 0) * 100, 2)      AS MarginPct
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
GROUP BY cust.SalesTerritory
ORDER BY TotalRevenue DESC;
```

13.2 Product Category Trend (2022–2025)

```
-- Year-over-year revenue comparison by category
SELECT  prod.CategoryName,
        SUM(CASE WHEN cal.CalendarYear = 2022 THEN fs.LineTotal ELSE 0 END) AS Rev2022,
        SUM(CASE WHEN cal.CalendarYear = 2023 THEN fs.LineTotal ELSE 0 END) AS Rev2023,
        SUM(CASE WHEN cal.CalendarYear = 2024 THEN fs.LineTotal ELSE 0 END) AS Rev2024,
        SUM(CASE WHEN cal.CalendarYear = 2025 THEN fs.LineTotal ELSE 0 END) AS Rev2025,
        -- Growth: 2025 vs 2022
        ROUND((SUM(CASE WHEN cal.CalendarYear = 2025 THEN fs.LineTotal ELSE 0 END)
               - SUM(CASE WHEN cal.CalendarYear = 2022 THEN fs.LineTotal ELSE 0 END))
              / NULLIF(SUM(CASE WHEN cal.CalendarYear = 2022 THEN fs.LineTotal ELSE 0
END), 0)
              * 100, 1)                                                      AS GrowthPct
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
GROUP BY prod.CategoryName
ORDER BY Rev2025 DESC;
```

13.3 Delivery Method Effectiveness

```
-- Revenue, average order value, and count by delivery method
SELECT  dlv.DeliveryMethodName,
        COUNT(DISTINCT fs.InvoiceID)        AS InvoiceCount,
        SUM(fs.LineTotal)                   AS TotalRevenue,
        ROUND(AVG(fs.LineTotal), 2)         AS AvgLineValue,
        MIN(fs.LineTotal)                   AS SmallestLine,
        MAX(fs.LineTotal)                   AS LargestLine
FROM    fact.Sales fs
INNER JOIN dim.DeliveryMethod dlv ON dlv.DeliveryMethodKey = fs.DeliveryMethodKey
GROUP BY dlv.DeliveryMethodName
ORDER BY TotalRevenue DESC;
```

13.4 Monthly Revenue Filtered to High-Volume Months

```
-- Months with more than 500 sales lines (busy months)
SELECT  cal.CalendarYear,
        cal.MonthName,
        cal.MonthNumber,
        COUNT(*)            AS SalesLines,
        SUM(fs.LineTotal)   AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName
HAVING  COUNT(*) > 500
ORDER BY cal.CalendarYear, cal.MonthNumber;
```

13.5 Supplier Revenue via Purchasing Datamart

```
-- Top suppliers by total purchase spend (using the Purchasing datamart)
USE CabotTrailOutdoorsPurchasing;

SELECT  sup.SupplierName,
        COUNT(DISTINCT fp.PurchaseOrderID)  AS PurchaseOrders,
        SUM(fp.LineTotal)                   AS TotalSpend,
        AVG(fp.DaysToDelivery)              AS AvgDaysToDeliver,
        SUM(CASE WHEN fp.IsLateDelivery = 1
                 THEN 1 ELSE 0 END)         AS LateDeliveries,
        ROUND(
            SUM(CASE WHEN fp.IsLateDelivery = 1 THEN 1.0 ELSE 0 END)
            / NULLIF(COUNT(*), 0) * 100, 1) AS LateDeliveryPct
FROM    fact.Purchasing fp
INNER JOIN dim.Supplier sup ON sup.SupplierKey = fp.SupplierKey
GROUP BY sup.SupplierName
ORDER BY TotalSpend DESC;
```

---

14. Chapter Summary

- **Aggregate functions** collapse multiple rows into a single summary value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
- `COUNT(*)` counts all rows. `COUNT(column)` counts non-NULL values. `COUNT(DISTINCT column)` counts unique non-NULL values. These answer three different questions.
- **`GROUP BY`** divides rows into groups based on unique combinations of the specified columns, then applies aggregate functions within each group. Every non-aggregated column in `SELECT` must appear in `GROUP BY`.
- The `GROUP BY` columns define the **grain** of the result — what one row in the output represents.

- **`HAVING`** filters groups after aggregation. `WHERE` filters rows before aggregation. Use `WHERE` for row-level conditions; use `HAVING` only for conditions on aggregate values.
- For performance: always filter with `WHERE` when the condition does not involve an aggregate. `WHERE` reduces rows before grouping; `HAVING` filters after the aggregation work is done.
- **CASE inside aggregates** (`COUNT(CASE WHEN ... END)` , `SUM(CASE WHEN ... THEN value ELSE 0 END)`) produces conditional counts and sums — a powerful technique for pivot-style reports.
- **Aggregate functions ignore NULLs** (except `COUNT(*)`). This affects `AVG` (divides by non-NULL count), `SUM` (ignores NULL rows), and NULL groups in `GROUP BY` .
- **Non-additive measures** like percentages should never be summed. Compute them from additive components: `SUM(Numerator) / SUM(Denominator)` .

---

15. Review Questions

1. Explain the difference between `COUNT(*)` , `COUNT(ProductCode)` , and `COUNT(DISTINCT CategoryName)` . Write a single query against `dim.Product` that demonstrates all three, and explain what each value represents.
2. Write a query that returns the number of sales lines, total revenue, average line value, and gross profit margin percentage for each sales territory. Join the necessary tables and sort by total revenue descending. Compute margin correctly as `SUM(GrossProfit) / SUM(LineTotal) * 100` .
3. The following query is intended to return categories with more than $100,000 in 2024 revenue, but it returns incorrect results. Identify the error, explain what the query actually returns, and rewrite it correctly:
   ```sql SELECT prod.CategoryName, SUM(fs.LineTotal) AS Revenue FROM fact.Sales fs INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey GROUP BY prod.CategoryName HAVING cal.CalendarYear = 2024 AND SUM(fs.LineTotal) > 100000;```
4. Write a query that produces a year-over-year comparison of invoice counts (not revenue) for each sales territory, with one column per year (2022, 2023, 2024, 2025) and a total column. Use `COUNT(DISTINCT InvoiceID)` with conditional CASE expressions.
5. Explain why `HAVING CategoryName = 'Sleeping'` is less efficient than `WHERE CategoryName = 'Sleeping'` in a query with `GROUP BY CategoryName` . Under what circumstance would you *need* to use `HAVING` rather than `WHERE` ?
   ```

6. Write a query that returns each delivery method name, the total number of sales using that method, and two counts: one for orders over $500 (`HighValueCount`) and one for orders at $500 or under (`StandardValueCount`). Use conditional CASE expressions within COUNT. Sort by total count descending.
7. A developer writes `SELECT AVG(OrderedQuantity) FROM fact.Sales` and gets `2` as the result. They claim the average order quantity is exactly 2. What potential issue might make this result misleading, and how would you verify it?
8. Write a query that identifies the top 5 customers by revenue in 2024, showing their name, territory, total revenue, invoice count, and the number of distinct product categories they purchased. Include only customers who purchased from at least 3 different categories. Use appropriate filters, grouping, and `HAVING` .

Deeper Dive

Going Further with Aggregation

ROLLUP and CUBE: Multi-Level Aggregation

Chapter 8 of the companion textbook *ETL for Business Intelligence* introduced `GROUPING SETS` , `ROLLUP` , and `CUBE` — extensions to `GROUP BY` that produce multiple levels of aggregation in a single query. They are worth knowing as an analyst because BI tools often generate queries using them.

ROLLUP produces a hierarchy of aggregations from left to right in the column list:

```
-- ROLLUP: produces totals at every level of the hierarchy
SELECT  prod.CategoryName,
        cal.CalendarYear,
        SUM(fs.LineTotal)   AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
GROUP BY ROLLUP (prod.CategoryName, cal.CalendarYear)
ORDER BY prod.CategoryName, cal.CalendarYear;
```

This produces:

- One row per (CategoryName, CalendarYear) combination — the most detailed level
- One row per CategoryName with NULLs for CalendarYear — subtotals per category
- One row with NULLs for both — the grand total

The NULLs in the rollup rows represent "all values" — the subtotal or grand total. Use `ISNULL(CategoryName, 'All Categories')` to label them clearly.

CUBE produces all possible combinations of the grouping columns — like ROLLUP but bidirectional:

```
GROUP BY CUBE (CategoryName, CalendarYear)
-- Produces: per (Category, Year), per Category, per Year, and grand total
```

GROUPING SETS lets you specify exactly which combination levels you want:

```
GROUP BY GROUPING SETS (
    (CategoryName, CalendarYear),   -- Detail level
    (CategoryName),                  -- Category subtotals
    ()                               -- Grand total
)
```

Microsoft documentation:
GROUP BY ROLLUP, CUBE, GROUPING SETS

The PIVOT Operator

The year-over-year comparison pattern in section 11.3 manually pivots years into columns using CASE expressions. SQL Server has a formal `PIVOT` operator that does the same thing:

```
-- PIVOT: years become columns automatically
SELECT  CategoryName,
        [2022], [2023], [2024], [2025]
FROM (
    SELECT  prod.CategoryName,
            cal.CalendarYear,
            fs.LineTotal
    FROM    fact.Sales fs
    INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
    INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
) AS SourceData
PIVOT (
    SUM(LineTotal)
    FOR CalendarYear IN ([2022], [2023], [2024], [2025])
) AS PivotTable
ORDER BY CategoryName;
```

`PIVOT` is more concise when there are many columns to pivot, but it requires knowing the pivot values at query-write time. The CASE approach is more flexible — it works with dynamic values and is easier for most analysts to read and maintain.

Microsoft documentation:
Using PIVOT and UNPIVOT

Running Totals and Moving Averages

Chapter 8 covers window functions in depth, but it is worth previewing the running total — a common analytical calculation that `GROUP BY` alone cannot produce.

A running total adds each row's value to all previous rows' values:

```
-- Monthly revenue with running total (window function preview)
SELECT
    cal.CalendarYear,
    cal.MonthNumber,
    cal.MonthName,
    SUM(fs.LineTotal)                   AS MonthlyRevenue,
    -- Running total: sum of all months up to and including this one
    SUM(SUM(fs.LineTotal)) OVER (
        PARTITION BY cal.CalendarYear
        ORDER BY cal.MonthNumber
        ROWS UNBOUNDED PRECEDING)       AS YTDRevenue
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName
ORDER BY cal.CalendarYear, cal.MonthNumber;
```

The `SUM(...) OVER (...)` clause is a **window function** — it computes the running total across an ordered window of rows. Chapter 8 explains the full syntax. For now, recognise that `GROUP BY` collapses rows while window functions preserve them — a fundamental architectural difference.

The COUNT(*) vs COUNT(1) Debate

You will occasionally see `COUNT(1)` in SQL code instead of `COUNT(*)`. Some developers believe `COUNT(1)` is faster. In SQL Server (and all modern databases), this is a myth — the query optimizer treats `COUNT(*)` and `COUNT(1)` identically. Both count all rows including NULLs. Use `COUNT(*)` — it is more idiomatic and communicates intent clearly.

Aggregation in BI Tools

Power BI, Tableau, and Excel Power Pivot all perform aggregation through their own interfaces — dragging fields to rows and columns, setting measures to Sum/Count/Average. What they generate internally is SQL (or DAX/MDX) with `GROUP BY` and aggregate functions. Every pivot table in Excel Power Pivot is a `GROUP BY` query.

Understanding SQL aggregation gives you the ability to:
- Debug why a BI tool report shows unexpected totals (usually a `GROUP BY` grain mismatch)
- Predict what a calculated measure will do when filters are applied (a `WHERE` or `HAVING` effect)
- Build the same report in SQL when the BI tool cannot express the required logic

The `CASE` pivot pattern from section 11.3 is especially useful here — it produces results that match what a BI tool pivot would show, making it easy to verify BI tool outputs against raw SQL queries.

Industry Perspectives

Aggregation Is the Core of Business Reporting

Virtually every business report, dashboard, and KPI involves aggregation: total revenue by region, average ticket size, customer count by cohort, monthly growth rate. The analytical questions that matter to organisations are aggregate questions — the detail rows are just the raw material.

Mastering `GROUP BY` and its interaction with `WHERE`, `HAVING`, `JOIN`, and aggregate functions is the single most impactful step in becoming a productive SQL analyst. An analyst who can write correct GROUP BY queries with CASE pivots and proper margin calculations is immediately useful in a business setting.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 2 covers aggregate functions; Chapter 7 covers GROUPING SETS, ROLLUP, and CUBE.
2. Microsoft. (2024). *Aggregate Functions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/aggregate-functions-transact-sql
3. Microsoft. (2024). *GROUP BY (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/select-group-by-transact-sql
4. Microsoft. (2024). *HAVING (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/select-having-transact-sql
5. Microsoft. (2024). *Using PIVOT and UNPIVOT*. https://learn.microsoft.com/en-us/sql/t-sql/queries/from-using-pivot-and-unpivot
6. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — Chapter 1 explains additive, semi-additive, and non-additive measures — the conceptual foundation for correct aggregation design.

Chapter 7: Scripting — Variables, Subqueries, and CTEs

Chapter Overview

The first six chapters built the core vocabulary of SQL: selecting, filtering, joining, and aggregating. This chapter introduces the tools that turn individual queries into reusable, readable analytical programs.

A single query answers a single question. A **script** is a sequence of SQL statements saved in a file that can be run repeatedly, shared with colleagues, and adapted to new questions by changing a parameter. **Variables** make scripts flexible. **Subqueries** allow one query to use the result of another. **Common Table Expressions (CTEs)** give complex queries readable structure. Together, these three tools are the bridge between writing SQL queries and writing SQL programs.

By the end of this chapter you will be able to:

- Declare and use variables to parameterise queries
- Write subqueries in WHERE, FROM, and SELECT positions
- Explain the difference between correlated and non-correlated subqueries
- Write Common Table Expressions (CTEs) and explain why they improve readability
- Chain multiple CTEs to break a complex problem into named steps
- Understand when to use a subquery versus a CTE versus a JOIN
- Write queries that reference the same CTE more than once

Table of Contents

1. From Queries to Scripts

2. Variables: DECLARE and SET
3. Using Variables in Queries
4. Subqueries: Queries Inside Queries
5. Subqueries in the WHERE Clause
6. Subqueries in the FROM Clause: Derived Tables
7. Subqueries in the SELECT Clause
8. Correlated Subqueries
9. EXISTS and NOT EXISTS
10. Common Table Expressions (CTEs)
11. Chaining Multiple CTEs
12. CTE vs Subquery vs JOIN: Choosing the Right Tool
13. Chapter Summary
14. Review Questions
15. Deeper Dive

---

1. From Queries to Scripts

A **query** is a single SQL statement that retrieves or modifies data. A **script** is one or more SQL statements saved together as a `.sql` file, intended to be run as a unit.

1.1 Why Scripts Matter

Consider the difference between these two approaches to answering "What was total revenue by category for the Atlantic region in 2024?":

Ad-hoc query approach: Open SSMS, type the query, adjust the year manually, adjust the territory manually, run it. To answer the same question for 2023, retype or edit the query. To share the analysis, paste the query into an email.

Script approach: Save the query as a `.sql` file with variables at the top for year and territory. To answer the same question for 2023, change `@Year = 2024` to `@Year = 2023` and run the script again. To share it, share the file.

Scripts are reproducible, shareable, and maintainable. They form the foundation of analytical work that needs to be repeated — monthly revenue reports, quarterly performance summaries, year-end analyses.

1.2 The Structure of a SQL Script

A well-structured SQL script has a standard layout:

```
-- ============================================================
-- Script: Revenue by Category and Territory
-- Purpose: Monthly category revenue for a specified year
-- Author: [Your Name]
-- Date: YYYY-MM-DD
-- ============================================================

USE CabotTrailOutdoorsSales;
GO

-- Parameters: change these to adjust the report
DECLARE @Year           INT             = 2024;
DECLARE @Territory      NVARCHAR(60)    = 'Atlantic — Nova Scotia';

-- Main query
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)   AS TotalRevenue
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Calendar cal  ON cal.DateKey      = fs.OrderDateKey
WHERE   cal.CalendarYear = @Year
AND     cust.SalesTerritory = @Territory
GROUP BY prod.CategoryName
ORDER BY TotalRevenue DESC;
```

The parameters section at the top is the "dial" that lets users adjust the script without touching the query logic.

---

2. Variables: DECLARE and SET

A **variable** is a named value that can be assigned and referenced within a SQL script. In SQL Server T-SQL, variables are declared with `DECLARE` and assigned with `SET`.

2.1 DECLARE: Creating a Variable

```
DECLARE @variable_name  data_type;
```

Variable names must start with `@`. The data type is any valid SQL Server type:

```
DECLARE @Year           INT;
DECLARE @Territory      NVARCHAR(60);
DECLARE @MinRevenue     DECIMAL(18,2);
DECLARE @StartDate      DATE;
DECLARE @EndDate        DATE;
```

2.2 SET: Assigning a Value

```
SET @variable_name = value;
```

```
SET @Year       = 2024;
SET @Territory  = 'Atlantic — Nova Scotia';
SET @MinRevenue = 10000.00;
SET @StartDate  = '2024-01-01';
SET @EndDate    = '2024-12-31';
```

2.3 Declaring and Setting in One Step

Variables can be initialised at declaration:

```
DECLARE @Year       INT             = 2024;
DECLARE @Territory  NVARCHAR(60)    = 'Atlantic — Nova Scotia';
DECLARE @MinRevenue DECIMAL(18,2)   = 10000.00;
```

This is the most common pattern in scripts — cleaner and more readable than separate `DECLARE` and `SET` statements.

2.4 Setting a Variable from a Query

`SET` can assign the result of a query to a variable, as long as the query returns exactly one value:

```
-- Set @MaxRevenue to the maximum single-line total in the fact table
DECLARE @MaxRevenue DECIMAL(18,2);

SET @MaxRevenue = (
    SELECT MAX(LineTotal)
    FROM   fact.Sales
);

SELECT @MaxRevenue AS MaxLineTotal;
-- Now @MaxRevenue holds the maximum value for use later in the script
```

If the subquery returns more than one row, SQL Server raises an error. This restriction enforces the scalar (single-value) nature of variable assignment.

2.5 Variables Are Session-Scoped

Variables exist only for the duration of the current session and are not visible to other sessions. They are reset when you disconnect and reconnect. This makes them safe for parameterising scripts — one analyst's `@Year = 2024` does not interfere with another analyst's `@Year = 2023` in a different session.

---

3. Using Variables in Queries

Once declared and set, variables are used exactly like literals in queries:

```
USE CabotTrailOutdoorsSales;

DECLARE @Year           INT             = 2024;
DECLARE @Territory      NVARCHAR(60)    = 'Atlantic — Nova Scotia';
DECLARE @MinRevenue     DECIMAL(18,2)   = 5000.00;

-- Use variables in WHERE, HAVING, and expressions
SELECT  cust.CustomerName,
        cust.SalesTerritory,
        SUM(fs.LineTotal)               AS TotalRevenue,
        COUNT(DISTINCT fs.InvoiceID)    AS InvoiceCount
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Calendar  cal ON cal.DateKey      = fs.OrderDateKey
WHERE   cal.CalendarYear    = @Year
AND     cust.SalesTerritory = @Territory
GROUP BY cust.CustomerName, cust.SalesTerritory
HAVING  SUM(fs.LineTotal)   > @MinRevenue
ORDER BY TotalRevenue DESC;
```

To run this for a different year or territory, change only the `DECLARE` lines at the top. The query body is unchanged.

3.1 Variables in Calculations

Variables participate in arithmetic like any value:

```
DECLARE @TaxRate    DECIMAL(5,4) = 0.15;   -- 15% HST
DECLARE @Discount   DECIMAL(5,4) = 0.10;   -- 10% discount

SELECT  ProductName,
        UnitPrice,
        ROUND(UnitPrice * (1 - @Discount), 2)              AS DiscountedPrice,
        ROUND(UnitPrice * (1 - @Discount) * (1 + @TaxRate), 2) AS FinalPrice
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY UnitPrice DESC;
```

3.2 Variables vs Hardcoded Literals: A Practical Demonstration

Run the same query twice — once with `@Year = 2023` and once with `@Year = 2024` — and compare the results. The variable makes the year a configuration parameter rather than logic embedded in the query. A colleague reading the script immediately knows which values to change.

---

4. Subqueries: Queries Inside Queries

A **subquery** is a complete `SELECT` statement nested inside another SQL statement. The outer query uses the result of the inner query as a data source, a filter value, or a calculated expression.

4.1 The Basic Structure

```
-- Outer query uses the result of the inner query
SELECT  column1
FROM    TableA
WHERE   column2 = (SELECT single_value FROM TableB WHERE condition);
```

The inner query (the subquery) is always enclosed in parentheses. SQL Server evaluates it first, then uses its result in the outer query.

4.2 When Subqueries Are Useful

Subqueries answer questions that require the result of one query to answer another:

- "Show me products whose price is above the average price." (Average must be computed first)
- "Show me customers who have placed orders this year." (Order existence must be checked per customer)
- "Show me the top 3 categories, then show me all products in those categories." (Categories must be identified first)

These questions have a natural two-step structure: compute something, then use that result to filter or join. Subqueries make that two-step structure explicit in SQL.

---

5. Subqueries in the WHERE Clause

The most common subquery position is in the `WHERE` clause, providing a filter value or set of values.

5.1 Single-Value Subquery

When the subquery returns exactly one value, use `=`, `>`, `<`, `>=`, or `<=`:

```
-- Products priced above the average price of all products
SELECT  ProductName,
        CategoryName,
        UnitPrice
FROM    dim.Product
WHERE   UnitPrice > (
    SELECT AVG(UnitPrice)
    FROM   dim.Product
    WHERE  IsDiscontinued = 0
)
ORDER BY UnitPrice DESC;
```

The subquery `SELECT AVG(UnitPrice) FROM dim.Product WHERE IsDiscontinued = 0` returns one value — the average price of active products. The outer query uses that value as the filter threshold.

5.2 Multi-Value Subquery with IN

When the subquery returns multiple values, use `IN`:

```
-- Customers who placed orders in 2024
-- (appear in fact.Sales with an order in 2024)
SELECT  CustomerName,
        SalesTerritory,
        CreditLimit
FROM    dim.Customer
WHERE   CustomerKey IN (
    SELECT DISTINCT fs.CustomerKey
    FROM   fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    WHERE  cal.CalendarYear = 2024
)
ORDER BY CustomerName;
```

The subquery returns the set of CustomerKeys for customers who appear in 2024 sales. The outer query then returns all `dim.Customer` rows whose `CustomerKey` is in that set.

> ***Note on NOT IN and NULLs:*** *As Chapter 2 warned,* `NOT IN` *behaves unexpectedly when the subquery returns NULLs. If any value in the subquery result is NULL,* `NOT IN` *returns no rows. For exclusion queries,* `NOT EXISTS` *(section 9) is safer.*

5.3 Comparing to Category Average

```
-- Products priced above their own category's average
SELECT  p1.ProductName,
        p1.CategoryName,
        p1.UnitPrice,
        (
            SELECT AVG(p2.UnitPrice)
            FROM   dim.Product p2
            WHERE  p2.CategoryName = p1.CategoryName
        )                       AS CategoryAvgPrice
FROM    dim.Product p1
WHERE   p1.UnitPrice > (
    SELECT AVG(p2.UnitPrice)
    FROM   dim.Product p2
    WHERE  p2.CategoryName = p1.CategoryName
)
ORDER BY p1.CategoryName, p1.UnitPrice DESC;
```

This query references `p1.CategoryName` from the outer query inside the subquery — making it a **correlated subquery** (covered in section 8). It computes the category average for each product's own category.

---

6. Subqueries in the FROM Clause: Derived Tables

A subquery in the `FROM` clause creates a **derived table** — a temporary result set that the outer query can treat like a regular table. The derived table must be given an alias.

6.1 Basic Derived Table

```
-- Derived table: aggregate first, then query the aggregated result
SELECT  Territory,
        TotalRevenue,
        RANK() OVER (ORDER BY TotalRevenue DESC)    AS RevenueRank
FROM (
    -- Subquery: revenue by territory
    SELECT  cust.SalesTerritory         AS Territory,
            SUM(fs.LineTotal)           AS TotalRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cust.SalesTerritory
) AS TerritoryRevenue
ORDER BY RevenueRank;
```

The inner query produces a summary of revenue by territory. The outer query applies `RANK()` to that summary — something that cannot be done in a single `SELECT` without a subquery or CTE, because `RANK()` cannot be applied to a `GROUP BY` result in the same query.

6.2 When Derived Tables Are Necessary

Derived tables are necessary when you need to:

- **Filter on an aggregate result** but cannot use `HAVING` (e.g., `WHERE Revenue > Avg_Revenue` — the average itself comes from aggregation)
- **Apply a function to an aggregated result** (e.g., `RANK()` over the grouped totals)
- **Join an aggregated result** to another table

```
-- Customers who spent more than the average customer
-- Step 1 (derived table): calculate each customer's total
-- Step 2 (outer query): find those above average

DECLARE @AvgCustomerRevenue DECIMAL(18,2);
SET @AvgCustomerRevenue = (
    SELECT AVG(CustomerTotal)
    FROM (
        SELECT CustomerKey, SUM(LineTotal) AS CustomerTotal
        FROM   fact.Sales
        GROUP BY CustomerKey
    ) AS Totals
);

SELECT  cust.CustomerName,
        cust.SalesTerritory,
        SUM(fs.LineTotal)   AS TotalRevenue
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
GROUP BY cust.CustomerName, cust.SalesTerritory
HAVING  SUM(fs.LineTotal) > @AvgCustomerRevenue
ORDER BY TotalRevenue DESC;
```

---

7. Subqueries in the SELECT Clause

A subquery in the `SELECT` list returns a single value for each row in the outer query — it is called a **scalar subquery**. It runs once per row, which means it can be slow on large tables.

7.1 Basic Scalar Subquery

```
-- For each product, show how its price compares to the overall average
SELECT  ProductName,
        CategoryName,
        UnitPrice,
        (SELECT AVG(UnitPrice) FROM dim.Product WHERE IsDiscontinued = 0)
                                AS OverallAvgPrice,
        UnitPrice - (SELECT AVG(UnitPrice) FROM dim.Product WHERE IsDiscontinued = 0)
                                AS DifferenceFromAvg
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY DifferenceFromAvg DESC;
```

The subquery `(SELECT AVG(UnitPrice) FROM dim.Product WHERE IsDiscontinued = 0)` is non-correlated — it returns the same value regardless of which outer row is being processed. SQL Server is smart enough to compute it once and reuse the result.

7.2 When to Use Scalar Subqueries vs Variables

The scalar subquery in section 7.1 can be rewritten using a variable:

```
DECLARE @AvgPrice DECIMAL(18,2) = (
    SELECT AVG(UnitPrice) FROM dim.Product WHERE IsDiscontinued = 0
);

SELECT  ProductName, UnitPrice,
        @AvgPrice           AS OverallAvgPrice,
        UnitPrice - @AvgPrice AS DifferenceFromAvg
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY DifferenceFromAvg DESC;
```

The variable version is often cleaner — the computation is done once, named clearly, and easy to inspect by printing `SELECT @AvgPrice`. Use scalar subqueries in SELECT when the value depends on each row (a correlated subquery); use variables when the value is fixed for the whole query.

---

8. Correlated Subqueries

A **correlated subquery** references a column from the outer query. It re-executes for each row of the outer query, using the current row's value as part of its condition.

8.1 The Mechanics

```
-- For each product, find the most expensive product in its category
SELECT  p_outer.ProductName,
        p_outer.CategoryName,
        p_outer.UnitPrice,
        (
            -- Correlated subquery: references p_outer.CategoryName
            SELECT MAX(p_inner.UnitPrice)
            FROM   dim.Product p_inner
            WHERE  p_inner.CategoryName = p_outer.CategoryName  -- correlation
        )                   AS MaxPriceInCategory
FROM    dim.Product p_outer
ORDER BY p_outer.CategoryName, p_outer.UnitPrice DESC;
```

For each row in the outer query (each product), the subquery finds the maximum price in *that product's* category. `p_outer.CategoryName` changes with each outer row — the subquery result changes accordingly.

8.2 Correlated Subquery Performance

Correlated subqueries run once per row of the outer query. For a dimension table with 142 rows, this is 142 executions of the inner query — fast. For a fact table with 16,359 rows, this is 16,359 executions — potentially slow.

Most correlated subqueries can be rewritten as JOINs with aggregation, which is more efficient:

```
-- Same result as the correlated subquery, using a JOIN instead
SELECT  p.ProductName,
        p.CategoryName,
        p.UnitPrice,
        cat_max.MaxPrice    AS MaxPriceInCategory
FROM    dim.Product p
INNER JOIN (
    SELECT  CategoryName, MAX(UnitPrice) AS MaxPrice
    FROM    dim.Product
    GROUP BY CategoryName
) AS cat_max ON cat_max.CategoryName = p.CategoryName
ORDER BY p.CategoryName, p.UnitPrice DESC;
```

The JOIN version computes category maxima once (in the subquery), then joins — much more efficient than recomputing per row.

Rule of thumb: Correlated subqueries in `SELECT` are often fine for small tables. In `WHERE` clauses with `EXISTS` (section 9), they are efficient even for large tables. Avoid correlated subqueries in `SELECT` on large fact tables — use CTEs or JOINs instead.

---

9. EXISTS and NOT EXISTS

`EXISTS` is a special predicate that tests whether a subquery returns any rows. It does not return values — it returns TRUE if the subquery returns at least one row, FALSE if it returns no rows.

9.1 EXISTS

```
-- Customers who have placed at least one order in 2024
SELECT  cust.CustomerName,
        cust.SalesTerritory
FROM    dim.Customer cust
WHERE   EXISTS (
    SELECT 1    -- The value returned does not matter -- only row existence is checked
    FROM   fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    WHERE  fs.CustomerKey   = cust.CustomerKey  -- Correlation
    AND    cal.CalendarYear = 2024
)
ORDER BY cust.CustomerName;
```

The `SELECT 1` inside the `EXISTS` is conventional — `EXISTS` only cares whether any row was returned, not what the row contains. SQL Server stops as soon as it finds the first matching row, making `EXISTS` very efficient.

9.2 NOT EXISTS

`NOT EXISTS` returns TRUE when the subquery returns no rows — the complement of `EXISTS`. It is the preferred tool for "find rows in A that have no corresponding rows in B":

```
-- Products that have never been sold
SELECT  prod.ProductName,
        prod.CategoryName,
        prod.UnitPrice
FROM    dim.Product prod
WHERE   NOT EXISTS (
    SELECT 1
    FROM   fact.Sales fs
    WHERE  fs.ProductKey = prod.ProductKey  -- Correlation
)
ORDER BY prod.CategoryName, prod.ProductName;
```

Compare this to the `LEFT JOIN ... WHERE IS NULL` approach from Chapter 5:

```
-- Equivalent using LEFT JOIN
SELECT  prod.ProductName, prod.CategoryName, prod.UnitPrice
FROM    dim.Product prod
LEFT JOIN fact.Sales fs ON fs.ProductKey = prod.ProductKey
WHERE   fs.SalesKey IS NULL
ORDER BY prod.CategoryName, prod.ProductName;
```

Both produce identical results. `NOT EXISTS` is generally preferred because:

- It is unambiguous in intent ("no matching row" is explicit)

- It avoids the `NOT IN` NULL trap (NULLs in `NOT IN` subqueries produce no rows)
- The query optimizer often produces the same or better execution plans

9.3 EXISTS vs IN: A Performance Note

`EXISTS` stops searching as soon as a match is found. `IN` collects all matching values, then checks membership. For large sets, `EXISTS` is typically faster because of early termination.

```
-- EXISTS (efficient for large sets):
WHERE EXISTS (SELECT 1 FROM fact.Sales fs WHERE fs.CustomerKey = cust.CustomerKey)

-- IN (less efficient for large sets):
WHERE CustomerKey IN (SELECT DISTINCT CustomerKey FROM fact.Sales)
```

For the CabotTrail data mart's modest sizes, the performance difference is negligible. In production with millions of rows, `EXISTS` is the preferred pattern for existence checks.

---

10. Common Table Expressions (CTEs)

A **Common Table Expression (CTE)** is a named temporary result set defined at the beginning of a query, before the main SELECT. It can be referenced by name in the query that follows — just like a table.

10.1 Why CTEs Exist

Consider a complex query that requires two levels of aggregation:

```
-- WITHOUT CTE: hard to read nested subquery
SELECT  Territory, TotalRevenue, TotalRevenue / SUM(TotalRevenue) OVER () AS RevShare
FROM (
    SELECT  cust.SalesTerritory AS Territory, SUM(fs.LineTotal) AS TotalRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cust.SalesTerritory
) AS TerritoryTotals
ORDER BY RevShare DESC;
```

The nested subquery is correct but hard to read. A CTE names the inner result, making it self-documenting:

```
-- WITH CTE: readable and self-documenting
WITH TerritoryRevenue AS (
    SELECT  cust.SalesTerritory     AS Territory,
            SUM(fs.LineTotal)       AS TotalRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cust.SalesTerritory
)
SELECT  Territory,
        TotalRevenue,
        ROUND(TotalRevenue * 100.0
              / SUM(TotalRevenue) OVER (), 2)   AS RevSharePct
FROM    TerritoryRevenue
ORDER BY RevSharePct DESC;
```

The CTE `TerritoryRevenue` has a name that describes what it contains. A reader immediately understands the query structure without parsing nested parentheses.

10.2 CTE Syntax

```
WITH CTE_Name AS (
    SELECT  ...     -- The CTE definition: any valid SELECT
    FROM    ...
    WHERE   ...
)
-- The main query: references CTE_Name like a table
SELECT  ...
FROM    CTE_Name
WHERE   ...;
```

The `WITH` keyword begins the CTE. The CTE name and its definition appear before the main `SELECT`. The CTE exists only for the duration of the query — it is not stored in the database.

10.3 A CTE for Monthly Revenue

```
-- Monthly revenue CTE used to calculate month-over-month change
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear,
        cal.MonthNumber,
        cal.YearMonthName,
        SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    LAG(Revenue) OVER (ORDER BY CalendarYear, MonthNumber)  AS PrevMonthRevenue,
    Revenue -
        LAG(Revenue) OVER (ORDER BY CalendarYear, MonthNumber) AS MoMChange
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

The CTE computes monthly totals. The outer query applies `LAG()` (a window function — covered in Chapter 8) to compare each month to the previous one. Without the CTE, this would require a complex nested subquery.

10.4 CTEs Can Be Referenced Multiple Times

Unlike a derived table subquery (which is defined where it is used), a CTE is defined once and can be referenced multiple times in the main query:

```
WITH CategoryRevenue AS (
    SELECT  prod.CategoryName,
            SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
    GROUP BY prod.CategoryName
)
-- Reference CategoryRevenue twice in the same main query
SELECT  cr.CategoryName,
        cr.Revenue,
        -- Compare to the average (second reference to the same CTE)
        cr.Revenue - (SELECT AVG(Revenue) FROM CategoryRevenue) AS DiffFromAvg,
        ROUND(cr.Revenue * 100.0
              / (SELECT SUM(Revenue) FROM CategoryRevenue), 2)  AS RevSharePct
FROM    CategoryRevenue cr
ORDER BY cr.Revenue DESC;
```

The CTE is computed once — both references use the same cached result.

11. Chaining Multiple CTEs

Multiple CTEs can be defined in sequence by separating them with commas. Each subsequent CTE can reference all previously defined CTEs.

11.1 Syntax

```
WITH FirstCTE AS (
    -- First CTE definition
    SELECT ...
),
SecondCTE AS (
    -- Second CTE: can reference FirstCTE
    SELECT ... FROM FirstCTE WHERE ...
),
ThirdCTE AS (
    -- Third CTE: can reference FirstCTE and SecondCTE
    SELECT ... FROM FirstCTE JOIN SecondCTE ON ...
)
-- Main query: can reference any of the CTEs
SELECT ... FROM ThirdCTE;
```

11.2 A Three-CTE Pipeline

This example answers a complex question: "For each sales territory, what percentage of customers are 'high value' (total revenue > $5,000), and what is the average revenue for the high-value vs standard segments?"

```
WITH
-- Step 1: Total revenue per customer
CustomerRevenue AS (
    SELECT
        fs.CustomerKey,
        SUM(fs.LineTotal)   AS TotalRevenue
    FROM    fact.Sales fs
    GROUP BY fs.CustomerKey
),
-- Step 2: Label each customer as high-value or standard
CustomerSegment AS (
    SELECT
        cr.CustomerKey,
        cr.TotalRevenue,
        CASE
            WHEN cr.TotalRevenue > 5000 THEN 'High Value'
            ELSE 'Standard'
        END                 AS Segment
    FROM    CustomerRevenue cr
),
-- Step 3: Join to dim.Customer for territory, then summarise by segment
TerritorySegmentSummary AS (
    SELECT
        cust.SalesTerritory,
        cs.Segment,
        COUNT(*)                        AS CustomerCount,
        ROUND(AVG(cs.TotalRevenue), 2)  AS AvgRevenue,
        SUM(cs.TotalRevenue)            AS SegmentRevenue
    FROM    CustomerSegment cs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = cs.CustomerKey
    GROUP BY cust.SalesTerritory, cs.Segment
)
-- Main query: final output
SELECT
    SalesTerritory,
    Segment,
    CustomerCount,
    AvgRevenue,
    SegmentRevenue
FROM    TerritorySegmentSummary
ORDER BY SalesTerritory, Segment;
```

Each CTE is a named, understandable step. Reading the query is like reading a recipe: first compute customer revenues, then classify them, then summarise by territory and segment. Compare this to a single deeply-nested query performing the same logic — the CTE version is dramatically more readable and debuggable.

11.3 Debugging Multi-CTE Queries

A key advantage of CTEs: you can test each step independently by running just the CTE definition as a standalone SELECT:

```
-- Test Step 1 alone:
SELECT CustomerKey, SUM(LineTotal) AS TotalRevenue
FROM   fact.Sales
GROUP BY CustomerKey
ORDER BY TotalRevenue DESC;

-- Test Step 2 alone:
WITH CustomerRevenue AS (
    SELECT CustomerKey, SUM(LineTotal) AS TotalRevenue
    FROM   fact.Sales GROUP BY CustomerKey
)
SELECT *, CASE WHEN TotalRevenue > 5000 THEN 'High Value' ELSE 'Standard' END AS Segment
FROM   CustomerRevenue;
```

Build and verify each step before adding the next. When a multi-step query produces unexpected results, this incremental debugging approach isolates the problem immediately.

---

12. CTE vs Subquery vs JOIN: Choosing the Right Tool

All three approaches — CTEs, subqueries, and JOINs — can often produce the same result. The choice is about readability, maintainability, and sometimes performance.

12.1 Decision Guide

Situation	Preferred tool	Reason
Simple key lookup or existence check	JOIN or EXISTS subquery	Direct, single-query
Filter based on an aggregate	Subquery in WHERE or HAVING	Clean and targeted
Complex multi-step calculation	CTE	Readable named steps
Result needed in multiple places in the same query	CTE	Defined once, referenced many times
Large set for IN comparison	EXISTS or JOIN	Better performance than IN
Intermediate result that needs to be joined	Derived table subquery or CTE	Both work; CTE is more readable

12.2 The Same Question, Three Ways

Question: "Which product categories have above-average revenue?"

Approach 1 — Subquery in HAVING:

```
SELECT  prod.CategoryName, SUM(fs.LineTotal) AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
HAVING  SUM(fs.LineTotal) > (
    SELECT AVG(CategoryTotal)
    FROM (
        SELECT SUM(LineTotal) AS CategoryTotal
        FROM   fact.Sales fs2
        INNER JOIN dim.Product p2 ON p2.ProductKey = fs2.ProductKey
        GROUP BY p2.CategoryName
    ) AS Totals
);
```

Approach 2 — Variable:

```
DECLARE @AvgCategoryRevenue DECIMAL(18,2);
SET @AvgCategoryRevenue = (
    SELECT AVG(CatRev)
    FROM (
        SELECT SUM(fs.LineTotal) AS CatRev
        FROM   fact.Sales fs
        INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
        GROUP BY prod.CategoryName
    ) AS t
);

SELECT  prod.CategoryName, SUM(fs.LineTotal) AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
HAVING  SUM(fs.LineTotal) > @AvgCategoryRevenue
ORDER BY Revenue DESC;
```

Approach 3 — CTE (clearest):

```
WITH CategoryRevenue AS (
    SELECT  prod.CategoryName,
            SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
    GROUP BY prod.CategoryName
),
AvgRevenue AS (
    SELECT AVG(Revenue) AS AvgCategoryRevenue
    FROM   CategoryRevenue
)
SELECT  cr.CategoryName, cr.Revenue
FROM    CategoryRevenue cr
CROSS JOIN AvgRevenue avg
WHERE   cr.Revenue > avg.AvgCategoryRevenue
ORDER BY cr.Revenue DESC;
```

All three are correct. For this specific question, Approach 2 (variable) is probably the most practical. Approach 3 (CTE) is most readable when this is part of a larger analysis. Approach 1 (nested subquery) is the most compact but hardest to read.

The professional habit: choose the approach that will be clearest to the next person who reads it — usually yourself, six months later.

---

13. Chapter Summary

- A **SQL script** is a `.sql` file containing one or more SQL statements. Variables, comments, and a clear structure make scripts reusable and shareable.
- **Variables** (`DECLARE @name type = value`) store values that can be referenced throughout a script. They parameterise queries — change the variable, not the query logic.
- A **subquery** is a complete SELECT nested inside another SQL statement. It can appear in `WHERE` (to filter), `FROM` (as a derived table), or `SELECT` (as a scalar value).
- **Single-value subqueries** use comparison operators (`=`, `>`). **Multi-value subqueries** use `IN`. Correlated subqueries reference the outer query's columns and re-execute per row.
- **EXISTS** tests whether a subquery returns any rows — efficient, stops at the first match. **NOT EXISTS** finds rows with no match in another table — preferred over `NOT IN` when NULLs may be present.
- A **CTE** (`WITH name AS (...)`) names a result set before the main query. CTEs improve readability, can be referenced multiple times, and allow complex queries to be decomposed into named steps.

- **Multiple CTEs** chain together, each able to reference its predecessors. Build and test each step independently, then combine.
- **Choose the right tool:** JOINs for direct relationships, subqueries for targeted filters, CTEs for multi-step calculations and readability.

---

14. Review Questions

1. Write a parameterised script (using variables) that returns the top N products by revenue for a specified year and category. Declare variables for `@Year`, `@CategoryName`, and `@TopN`. The query should join `fact.Sales` to `dim.Product` and `dim.Calendar`, filter by year and category, and use `TOP @TopN` to limit results.
2. Write a subquery in the WHERE clause that returns all customers whose `CreditLimit` is higher than the average credit limit across all customers. Include `CustomerName`, `SalesTerritory`, and `CreditLimit`. Sort by `CreditLimit` descending.
3. Using a derived table (subquery in FROM), write a query that:
   - First calculates total revenue per customer (inner query)
   - Then returns only customers whose total revenue is above the average customer revenue (outer query)
   Include `CustomerKey` and total revenue in the result. Sort by revenue descending.
4. Rewrite the following correlated subquery as a JOIN with aggregation, and explain why the JOIN approach is generally more efficient:
   `sql SELECT p.ProductName, p.CategoryName, p.UnitPrice, (SELECT MAX(p2.UnitPrice) FROM dim.Product p2 WHERE p2.CategoryName = p.CategoryName) AS CategoryMax FROM dim.Product p;`
5. Write a query using `NOT EXISTS` that returns all products in `dim.Product` that have no sales in `fact.Sales` (never sold). Include `ProductName`, `CategoryName`, and `UnitPrice`. Sort by category then product name. Explain why `NOT EXISTS` is preferred over `NOT IN` for this type of query.
6. Write a CTE called `MonthlySales` that computes total revenue and sales line count for each calendar year and month. Then use the CTE in the main query to show only months where revenue exceeded $50,000. Include year, month name, revenue, and line count. Sort chronologically.
7. Write a three-CTE query:
   - CTE 1 (`ProductRevenue`): total revenue per product
   - CTE 2 (`ProductRanked`): add a revenue rank within each category using `RANK() OVER (PARTITION BY CategoryName ORDER BY Revenue DESC)`

- Main query: return only the top 3 products per category (where rank ≤ 3)
Include product name, category, revenue, and rank. Sort by category then rank.

8. A colleague claims that replacing a subquery with a CTE always makes queries faster. Evaluate this claim. Is it true? Under what circumstances does a CTE provide a performance benefit, and when does it not?

Deeper Dive

Going Further with Scripting, Subqueries, and CTEs

Recursive CTEs

A **recursive CTE** references itself, enabling queries over hierarchical data — org charts, bill-of-materials trees, category hierarchies — without knowing the depth of the hierarchy in advance.

The structure has two parts: an **anchor member** (the starting point) and a **recursive member** (the repeating step):

```
-- Employee hierarchy: who reports to whom (all levels)
WITH EmployeeHierarchy AS (
    -- Anchor: start with the top-level employees (no manager)
    SELECT  e.EmployeeID,
            e.FullName,
            e.ManagerID,
            0               AS HierarchyLevel,
            CAST(e.FullName AS NVARCHAR(500)) AS HierarchyPath
    FROM    Application.Employees e
    WHERE   e.ManagerID IS NULL

    UNION ALL

    -- Recursive: find each employee's direct reports
    SELECT  e.EmployeeID,
            e.FullName,
            e.ManagerID,
            eh.HierarchyLevel + 1,
            CAST(eh.HierarchyPath + ' > ' + e.FullName AS NVARCHAR(500))
    FROM    Application.Employees e
    INNER JOIN EmployeeHierarchy eh ON eh.EmployeeID = e.ManagerID
)
SELECT  HierarchyLevel,
        REPLICATE('  ', HierarchyLevel) + FullName  AS IndentedName,
        HierarchyPath
FROM    EmployeeHierarchy
ORDER BY HierarchyPath;
```

`MAXRECURSION` controls the maximum depth (default 100). Set it to 0 for unlimited depth:

```
OPTION (MAXRECURSION 0)  -- Add at the end of the query
```

Recursive CTEs are covered in depth in DBAS 2103. For now, understanding that CTEs can be recursive opens the door to a powerful class of queries.

Microsoft documentation:
WITH common_table_expression (Transact-SQL)

The Difference Between a CTE and a Temporary Table

CTEs and temporary tables both name an intermediate result. They behave differently:

Feature	CTE	Temporary table (`#temp`)
Storage	In memory (or re-evaluated)	`tempdb` on disk
Scope	Current query only	Current session (until dropped or session ends)
Indexable	No	Yes — can add indexes
Referenceable	Within the same statement only	By any subsequent statement in the session
Statistics	No	Yes — optimizer can estimate sizes

Use a CTE when:
- The result is used only in the immediately following query
- Readability and structure are the goal
- The intermediate result set is small

Use a temporary table when:
- The result is used by multiple subsequent queries in the same session
- The result set is large and benefits from indexing
- You need to join to the intermediate result many times

```
-- Temporary table example: materialise an expensive result for reuse
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)   AS Revenue
INTO    #CategoryRevenue    -- Creates a temp table
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName;

-- Query #CategoryRevenue multiple times without recomputing
SELECT * FROM #CategoryRevenue ORDER BY Revenue DESC;
SELECT AVG(Revenue) AS AvgRevenue FROM #CategoryRevenue;

DROP TABLE #CategoryRevenue;  -- Clean up when done
```

APPLY: A Powerful Alternative to Correlated Subqueries

SQL Server's `APPLY` operator (`CROSS APPLY` and `OUTER APPLY`) is a powerful alternative to correlated subqueries that offers better performance and more flexibility:

```
-- CROSS APPLY: find the top 2 products per category
SELECT  prod.CategoryName,
        top_prods.ProductName,
        top_prods.UnitPrice
FROM    (SELECT DISTINCT CategoryName FROM dim.Product) prod
CROSS APPLY (
    SELECT TOP 2 ProductName, UnitPrice
    FROM   dim.Product p2
    WHERE  p2.CategoryName = prod.CategoryName
    ORDER BY UnitPrice DESC
) top_prods
ORDER BY prod.CategoryName, top_prods.UnitPrice DESC;
```

`CROSS APPLY` is like an `INNER JOIN` with a correlated subquery — only rows where the inner query returns results are kept. `OUTER APPLY` is like a `LEFT JOIN` — all left rows are kept, with NULLs where the inner query returns nothing.

`APPLY` is particularly powerful for applying table-valued functions per row, which cannot be done with JOINs alone.

Microsoft documentation:
Using APPLY

Query Plan Caching and Variable Sniffing

When SQL Server compiles a parameterised query (one using variables), it creates an execution plan based on the variable values at compilation time — a process called **parameter sniffing**. This plan is cached and reused for subsequent executions.

If the first execution uses unusual variable values (e.g., `@Year = 2020` when most data is in 2024), the cached plan may be suboptimal for typical executions. Solutions:

```
-- Option 1: Use OPTION (RECOMPILE) to force a new plan each time
SELECT ...
FROM   ...
WHERE  CalendarYear = @Year
OPTION (RECOMPILE);

-- Option 2: Use local variables inside a stored procedure (defends against sniffing)
DECLARE @LocalYear INT = @Year;
SELECT ... WHERE CalendarYear = @LocalYear;
```

For the CabotTrail data mart, parameter sniffing is not a concern — the data volumes are small. In production environments with uneven data distributions (a very busy year vs quiet years), this is an important tuning consideration.

Industry Perspectives

CTEs as Documentation

Well-named CTEs serve as inline documentation — they communicate analytical intent in a way that raw SQL cannot. Compare:

```
-- Without CTEs: what does this do?
SELECT t2.Region, t2.Seg, t2.Cnt, t2.Rev
FROM (SELECT c.SalesTerritory AS Region,
             CASE WHEN r.TotalRev > 5000 THEN 'HV' ELSE 'Std' END AS Seg,
             COUNT(*) AS Cnt, SUM(r.TotalRev) AS Rev
      FROM (SELECT CustomerKey, SUM(LineTotal) AS TotalRev
            FROM fact.Sales GROUP BY CustomerKey) r
      INNER JOIN dim.Customer c ON c.CustomerKey = r.CustomerKey
      GROUP BY c.SalesTerritory,
               CASE WHEN r.TotalRev > 5000 THEN 'HV' ELSE 'Std' END) t2
ORDER BY t2.Region, t2.Seg;

-- With CTEs: immediately clear
WITH CustomerRevenue AS (...),
     CustomerSegment AS (...),
     TerritorySegmentSummary AS (...)
SELECT SalesTerritory, Segment, CustomerCount, AvgRevenue, SegmentRevenue
FROM   TerritorySegmentSummary
ORDER BY SalesTerritory, Segment;
```

The CTE version communicates *what* the query computes — it is self-documenting. When you return to this query in three months (or hand it to a colleague), the CTEs are the documentation.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 5 covers subqueries; Chapter 9 covers CTEs including recursive CTEs.
2. Ben-Gan, I., Kollar, L., Sarka, D., & Talmage, R. (2015). *T-SQL Querying*. Microsoft Press. — Chapters 4–5 cover CTEs, derived tables, and advanced subquery patterns with performance analysis.
3. Microsoft. (2024). *WITH common_table_expression (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/with-common-table-expression-transact-sql
4. Microsoft. (2024). *Subqueries (SQL Server)*. https://learn.microsoft.com/en-us/sql/relational-databases/performance/subqueries
5. Microsoft. (2024). *DECLARE @local_variable (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/declare-local-variable-transact-sql
6. Microsoft. (2024). *Using APPLY*. https://learn.microsoft.com/en-us/sql/t-sql/queries/from-transact-sql#using-apply
7. Fritchey, G. (2018). *SQL Server Query Performance Tuning* (5th ed.). Apress. — Chapter 4 covers parameter sniffing, plan caching, and variable impact on query performance.

Chapter 8: Advanced Queries — Window Functions, UDFs, and Stored Procedures

Chapter Overview

Chapter 6 showed how aggregate functions collapse rows into groups. This chapter introduces a family of functions that do something subtler and more powerful: they perform calculations *across a set of rows* while keeping every individual row intact. These are **window functions** — and they are the most analytically expressive tools in the SQL analyst's toolkit.

This chapter also introduces **User-Defined Functions (UDFs)** and **stored procedures** — the mechanisms for packaging reusable SQL logic into named objects that live in the database, available to any query or application that needs them.

By the end of this chapter you will be able to:

- Explain what a window function is and how it differs from an aggregate function
- Write ranking functions: `RANK`, `DENSE_RANK`, and `ROW_NUMBER`
- Write offset functions: `LAG` and `LEAD` for period-over-period comparisons
- Write running totals and moving averages using `SUM` and `AVG` with `OVER`
- Use `PARTITION BY` to scope window calculations to sub-groups
- Use `ROWS` and `RANGE` frames to control which rows the window covers
- Create and call scalar User-Defined Functions
- Create and execute stored procedures with parameters
- Use `CAST` and `FORMAT` for type conversion and display formatting

Table of Contents

1. What Window Functions Are
2. The OVER Clause: Defining the Window
3. PARTITION BY: Scoping the Window
4. Ranking Functions: RANK, DENSE_RANK, ROW_NUMBER
5. Offset Functions: LAG and LEAD
6. Running Totals and Moving Aggregates
7. ROWS vs RANGE Frames
8. Practical Window Function Patterns
9. Type Conversion: CAST and FORMAT Revisited
10. User-Defined Functions: Scalar UDFs
11. Stored Procedures
12. Chapter Summary
13. Review Questions
14. Deeper Dive

---

1. What Window Functions Are

To understand window functions, contrast them with two things you already know.

1.1 Scalar vs Aggregate vs Window

Scalar functions (Chapter 4) operate on one row at a time, returning one value per input row. The row count of the result equals the row count of the input:

```
Input: 16,359 rows → LEN(ProductName) → Output: 16,359 values, one per row
```

Aggregate functions (Chapter 6) with `GROUP BY` collapse many rows into one per group. The row count of the result is the number of groups:

```
Input: 16,359 rows → SUM(LineTotal) GROUP BY CategoryName → Output: 13 rows (one per
category)
```

Window functions compute across a set of rows but return one value *per input row* — the rows are not collapsed:

```
Input: 16,359 rows → RANK() OVER (ORDER BY LineTotal DESC) → Output: 16,359 rows, each
with a rank
```

This is the key insight: window functions add new information to each row (a rank, a running total, the previous row's value) without removing any rows. The full detail is preserved.

1.2 Why This Matters

The inability to combine aggregate functions and row-level details in the same `SELECT` is a fundamental limitation of `GROUP BY`. Window functions break that limitation:

```
-- IMPOSSIBLE with GROUP BY alone:
-- "Show each sale line with its own total AND the running total AND the category share"
-- GROUP BY collapses the rows — you cannot see individual lines AND group totals
simultaneously

-- POSSIBLE with window functions:
SELECT
    InvoiceID,
    LineTotal,
    SUM(LineTotal) OVER ()                      AS GrandTotal,      -- Total of all rows
    SUM(LineTotal) OVER (ORDER BY OrderDate     -- Running total
                         ROWS UNBOUNDED PRECEDING) AS RunningTotal,
    ROUND(LineTotal * 100.0
          / SUM(LineTotal) OVER (), 2)          AS ShareOfTotal     -- Percentage of
grand total
FROM    fact.Sales
ORDER BY OrderDate;
```

Each row retains its own `LineTotal` while also gaining access to the grand total and its running position in time. This is the analytical power that window functions unlock.

---

2. The OVER Clause: Defining the Window

Every window function requires an `OVER` clause. The `OVER` clause defines the **window** — the set of rows that the function operates over for each output row.

2.1 OVER with No Arguments: The Entire Result Set

```
FUNCTION() OVER ()
```

An empty `OVER ()` means the window is the entire result set — every row in the query result is in the window for every output row:

```
-- Total revenue for the whole company (same value on every row)
SELECT  InvoiceID,
        LineTotal,
        SUM(LineTotal) OVER ()  AS CompanyTotalRevenue
FROM    fact.Sales
ORDER BY InvoiceID;
```

Every row has the same `CompanyTotalRevenue` — the sum of all rows. This is useful for computing percentage shares without a separate subquery.

2.2 OVER with ORDER BY: Ordered Window

```
FUNCTION() OVER (ORDER BY column)
```

Adding `ORDER BY` inside `OVER` creates an ordered window — rows are processed in order, and the function accumulates across that order:

```
-- Running count of sales lines ordered by date
SELECT  OrderDate,
        InvoiceID,
        LineTotal,
        ROW_NUMBER() OVER (ORDER BY OrderDate, InvoiceID) AS RowNum,
        SUM(LineTotal) OVER (ORDER BY OrderDate, InvoiceID
                             ROWS UNBOUNDED PRECEDING)    AS RunningRevenue
FROM    fact.Sales
ORDER BY OrderDate, InvoiceID;
```

The `ROWS UNBOUNDED PRECEDING` frame is covered in section 7. For now, understand that `ORDER BY` inside `OVER` establishes the sequence within the window.

2.3 The Window Is Per Output Row

For every row in the output, SQL Server looks at that row's position within its window and computes the function over the rows in that window. The window can overlap between rows — most window calculations use overlapping windows.

---

3. PARTITION BY: Scoping the Window

`PARTITION BY` divides the rows into sub-groups before the window function is applied — like `GROUP BY` for window functions, except the rows are not collapsed.

3.1 Syntax

```
FUNCTION() OVER (PARTITION BY column ORDER BY column)
```

3.2 PARTITION BY in Practice

```
-- Rank products by price within each category
SELECT  ProductName,
        CategoryName,
        UnitPrice,
        RANK() OVER (
            PARTITION BY CategoryName   -- Reset the window for each category
            ORDER BY UnitPrice DESC     -- Rank by price within the category
        )                               AS PriceRankInCategory
FROM    dim.Product
ORDER BY CategoryName, PriceRankInCategory;
```

Without `PARTITION BY`, `RANK()` would rank across all 142 products. With `PARTITION BY CategoryName`, it ranks within each category independently — the most expensive product in each category gets rank 1.

3.3 Running Total per Category

```
-- Running revenue total, reset for each product category
SELECT
    prod.CategoryName,
    cal.FullDate,
    fs.LineTotal,
    SUM(fs.LineTotal) OVER (
        PARTITION BY prod.CategoryName
        ORDER BY cal.FullDate
        ROWS UNBOUNDED PRECEDING
    )                               AS CategoryRunningRevenue
FROM    fact.Sales fs
INNER JOIN dim.Product  prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal  ON cal.DateKey     = fs.OrderDateKey
ORDER BY prod.CategoryName, cal.FullDate;
```

The running total resets to zero at the start of each new category — `PARTITION BY` creates independent windows for each.

---

4. Ranking Functions: RANK, DENSE_RANK, ROW_NUMBER

Three functions assign ordinal positions to rows within a window. They differ in how they handle tied values.

4.1 ROW_NUMBER: Unique Sequential Numbers

`ROW_NUMBER()` assigns a unique integer to every row, with no ties — even rows with identical values get different numbers. The assignment is arbitrary when values are equal (the order among equals is not defined unless there is a tiebreaker column):

```
-- Unique row number for each sale line
SELECT  InvoiceID,
        LineTotal,
        ROW_NUMBER() OVER (ORDER BY LineTotal DESC) AS RowNum
FROM    fact.Sales
ORDER BY RowNum;
```

Use `ROW_NUMBER()` when you need a unique identifier for every row — for pagination, for deduplication, or to pick exactly N rows per group.

4.2 RANK: Gaps After Ties

`RANK()` assigns the same rank to tied rows and skips the next rank(s):

```
-- Rank sales reps by revenue: tied reps share a rank, the next rank is skipped
WITH SalesRepRevenue AS (
    SELECT  emp.FullName,
            SUM(fs.LineTotal)   AS TotalRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
    GROUP BY emp.FullName
)
SELECT  FullName,
        TotalRevenue,
        RANK() OVER (ORDER BY TotalRevenue DESC) AS RevenueRank
FROM    SalesRepRevenue
ORDER BY RevenueRank;
-- If two reps tie for rank 3, the next rep gets rank 5 (rank 4 is skipped)
```

4.3 DENSE_RANK: No Gaps After Ties

`DENSE_RANK()` also assigns the same rank to tied rows, but does not skip subsequent ranks:

```
WITH SalesRepRevenue AS (
    SELECT  emp.FullName,
            SUM(fs.LineTotal)   AS TotalRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
    GROUP BY emp.FullName
)
SELECT  FullName,
        TotalRevenue,
        RANK()        OVER (ORDER BY TotalRevenue DESC) AS RankWithGaps,
        DENSE_RANK()  OVER (ORDER BY TotalRevenue DESC) AS RankNoGaps,
        ROW_NUMBER()  OVER (ORDER BY TotalRevenue DESC) AS UniqueRowNum
FROM    SalesRepRevenue
ORDER BY RankWithGaps;
```

Running this query side by side shows the three functions' different behaviours clearly. If no ties exist, all three produce identical output.

4.4 Choosing Between Ranking Functions

Use case	Function
Unique sequential identifier for every row	`ROW_NUMBER()`
Top N per group (deduplication)	`ROW_NUMBER()` — unique, predictable
Sports-style ranking ("shared 3rd place")	`RANK()`
Ordinal ranking with no gaps	`DENSE_RANK()`
Percentile-style ranking	`NTILE(n)` (covered in Deeper Dive)

4.5 Top N Per Group Using ROW_NUMBER

This is one of the most practical uses of `ROW_NUMBER()` — selecting the top N rows within each partition:

```
-- Top 2 products by revenue in each category
WITH ProductRevenue AS (
    SELECT  prod.ProductName,
            prod.CategoryName,
            SUM(fs.LineTotal)       AS Revenue,
            ROW_NUMBER() OVER (
                PARTITION BY prod.CategoryName
                ORDER BY SUM(fs.LineTotal) DESC
            )                       AS RankInCategory
    FROM    fact.Sales fs
    INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
    GROUP BY prod.ProductName, prod.CategoryName
)
SELECT  CategoryName, ProductName, Revenue, RankInCategory
FROM    ProductRevenue
WHERE   RankInCategory <= 2
ORDER BY CategoryName, RankInCategory;
```

This pattern — window function in a CTE, filter in the outer query — is the standard approach for top-N-per-group queries. It cannot be done with `TOP` alone (which applies to the entire result) or with `HAVING` (which filters groups, not ranked rows).

---

5. Offset Functions: LAG and LEAD

`LAG` and `LEAD` access the value of a column from a previous or subsequent row within the window — without a self-join.

5.1 LAG: Look Backwards

`LAG(column, offset, default)` returns the value of `column` from `offset` rows before the current row. If no prior row exists, returns `default` (or NULL if default is not specified):

```
-- Monthly revenue with previous month for comparison
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear,
        cal.MonthNumber,
        cal.YearMonthName,
        SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    LAG(Revenue, 1, 0)  OVER (ORDER BY CalendarYear, MonthNumber) AS PrevMonthRevenue,
    Revenue - LAG(Revenue, 1, 0) OVER (ORDER BY CalendarYear, MonthNumber) AS MoMChange,
    ROUND(
        (Revenue - LAG(Revenue, 1, 0) OVER (ORDER BY CalendarYear, MonthNumber))
        / NULLIF(LAG(Revenue, 1, 0) OVER (ORDER BY CalendarYear, MonthNumber), 0)
        * 100, 1
    )                   AS MoMChangePct
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

`LAG(Revenue, 1, 0)` returns the previous month's revenue. For the first month in the dataset, it returns 0 (the default).

5.2 Same Period Last Year with LAG

By setting the offset to 12 (12 months back), `LAG` looks at the same month in the prior year:

```
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear, cal.MonthNumber, cal.YearMonthName,
        SUM(fs.LineTotal) AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    LAG(Revenue, 12, NULL) OVER (ORDER BY CalendarYear, MonthNumber) AS SamePeriodLastYea
r,
    ROUND(
        (Revenue - LAG(Revenue, 12, NULL) OVER (ORDER BY CalendarYear, MonthNumber))
        / NULLIF(LAG(Revenue, 12, NULL) OVER (ORDER BY CalendarYear, MonthNumber), 0)
        * 100, 1
    )                   AS YoYChangePct
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

Year-over-year comparison — one of the most common financial reporting requirements — in a single query.

5.3 LEAD: Look Forwards

`LEAD(column, offset, default)` works like `LAG` but looks ahead — it returns the value from `offset` rows *after* the current row:

```
-- Show next month's revenue for each row (useful for projections or previewing trends)
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear, cal.MonthNumber, cal.YearMonthName,
        SUM(fs.LineTotal) AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    LEAD(Revenue, 1, NULL) OVER (ORDER BY CalendarYear, MonthNumber) AS NextMonthRevenue
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

6. Running Totals and Moving Aggregates

Window functions applied to aggregate functions (`SUM` , `AVG` , `MIN` , `MAX` , `COUNT`) with `ORDER BY` produce running calculations.

6.1 Running Total

```
-- Cumulative revenue over time
WITH DailyRevenue AS (
    SELECT
        cal.FullDate,
        SUM(fs.LineTotal)   AS DailyRevenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    WHERE   cal.CalendarYear = 2024
    GROUP BY cal.FullDate
)
SELECT
    FullDate,
    DailyRevenue,
    SUM(DailyRevenue) OVER (
        ORDER BY FullDate
        ROWS UNBOUNDED PRECEDING    -- All rows from the start to the current row
    )                               AS CumulativeRevenue
FROM    DailyRevenue
ORDER BY FullDate;
```

6.2 Year-to-Date (YTD) Total

With `PARTITION BY` , the running total resets at the start of each year:

```
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear, cal.MonthNumber, cal.YearMonthName,
        SUM(fs.LineTotal) AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    SUM(Revenue) OVER (
        PARTITION BY CalendarYear       -- Reset for each year
        ORDER BY MonthNumber
        ROWS UNBOUNDED PRECEDING
    )                           AS YTD_Revenue
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

6.3 Moving Average

A moving average smooths out short-term fluctuations to reveal underlying trends. A 3-month moving average averages the current month with the previous two:

```
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear, cal.MonthNumber, cal.YearMonthName,
        SUM(fs.LineTotal) AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName,
    Revenue,
    ROUND(
        AVG(Revenue) OVER (
            ORDER BY CalendarYear, MonthNumber
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- Current + 2 prior months
        ), 2
    )                       AS MovingAvg3Month
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

The first two months will have a moving average based on fewer than 3 months (only the data available), which is the correct behaviour.

---

7. ROWS vs RANGE Frames

The frame clause (`ROWS BETWEEN ... AND ...`) precisely defines which rows are included in the window for each row.

7.1 Frame Boundaries

Boundary keyword	Meaning
`UNBOUNDED PRECEDING`	From the first row of the partition
`n PRECEDING`	n rows before the current row
`CURRENT ROW`	The current row itself
`n FOLLOWING`	n rows after the current row
`UNBOUNDED FOLLOWING`	To the last row of the partition

7.2 Common Frame Patterns

```
-- Running total: from start to current row
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

-- 3-month moving average: current row plus 2 before
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

-- Centred moving average: 1 before, current, 1 after
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

-- Entire partition (all rows): useful for percent of total
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
```

7.3 ROWS vs RANGE

`ROWS` counts physical rows. `RANGE` counts logical values — rows with the same `ORDER BY` value are treated as a single logical unit.

```
-- ROWS: counts each row individually
SUM(Revenue) OVER (ORDER BY MonthNumber ROWS UNBOUNDED PRECEDING)

-- RANGE: rows with equal MonthNumber values are treated together
SUM(Revenue) OVER (ORDER BY MonthNumber RANGE UNBOUNDED PRECEDING)
```

For most analytical use cases, `ROWS` is the correct choice — it produces predictable, deterministic results. `RANGE` is appropriate when you specifically want to group identical values.

> ***Default behaviour warning:*** *When you write* `OVER (ORDER BY column)` *without a frame clause, SQL Server defaults to* `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`*. This is usually what you want for running totals, but if there are tied values in the ORDER BY column,* `RANGE` *may include more rows than expected. Explicitly specify* `ROWS UNBOUNDED PRECEDING` *for deterministic results.*

---

8. Practical Window Function Patterns

8.1 Percentage of Total

```
-- Each category's share of total revenue
SELECT  prod.CategoryName,
        SUM(fs.LineTotal)                   AS CategoryRevenue,
        SUM(SUM(fs.LineTotal)) OVER ()      AS TotalRevenue,
        ROUND(
            SUM(fs.LineTotal) * 100.0
            / SUM(SUM(fs.LineTotal)) OVER (),
        2)                                  AS RevSharePct
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
ORDER BY CategoryRevenue DESC;
```

Notice `SUM(SUM(fs.LineTotal)) OVER ()` — a nested aggregate. The inner `SUM(fs.LineTotal)` is the GROUP BY aggregate per category; the outer `SUM(...) OVER ()` sums those category totals to get the grand total. This is the standard pattern for computing shares over grouped results.

8.2 Ranking Within Groups

```
-- Each customer ranked by revenue within their territory
WITH CustomerRevenue AS (
    SELECT
        cust.CustomerName,
        cust.SalesTerritory,
        SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cust.CustomerName, cust.SalesTerritory
)
SELECT
    SalesTerritory,
    CustomerName,
    Revenue,
    RANK() OVER (
        PARTITION BY SalesTerritory
        ORDER BY Revenue DESC
    )              AS TerritoryRank
FROM    CustomerRevenue
ORDER BY SalesTerritory, TerritoryRank;
```

8.3 Detecting First and Last Occurrence

```
-- For each customer: date of first and last purchase, days between them
SELECT
    cust.CustomerName,
    MIN(cal.FullDate)                               AS FirstPurchase,
    MAX(cal.FullDate)                               AS LastPurchase,
    DATEDIFF(day, MIN(cal.FullDate), MAX(cal.FullDate)) AS DaysBetween,
    COUNT(DISTINCT fs.InvoiceID)                    AS TotalInvoices
FROM    fact.Sales fs
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
INNER JOIN dim.Calendar  cal ON cal.DateKey      = fs.OrderDateKey
GROUP BY cust.CustomerName
ORDER BY FirstPurchase;
```

---

9. Type Conversion: CAST and FORMAT Revisited

Chapter 3 introduced `CAST` and Chapter 4 introduced `FORMAT`. Advanced queries often require explicit type conversion to ensure correct results in calculations and string outputs.

9.1 CAST for Calculation Control

Conversion is most critical in division and when combining types in string operations:

```
-- Integer to decimal for safe division
CAST(OrderedQuantity AS DECIMAL(10,2)) / CAST(InvoiceCount AS DECIMAL(10,2))

-- Integer date key to readable string
CAST(OrderDateKey AS NVARCHAR(8))   -- '20240315'

-- NVARCHAR to INT for a stored numeric string
CAST('42' AS INT)

-- Date to NVARCHAR for concatenation
CAST(GETDATE() AS DATE)             -- Strips time component
CAST(CAST(GETDATE() AS DATE) AS NVARCHAR(10))  -- '2026-09-10'
```

9.2 FORMAT for Display Output

`FORMAT` is ideal for the final output layer of a report — when human-readable formatting matters more than performance:

```
-- Revenue report with formatted output
SELECT
    prod.CategoryName,
    SUM(fs.LineTotal)                           AS RawRevenue,
    FORMAT(SUM(fs.LineTotal), 'C', 'en-CA')     AS FormattedRevenue,    -- $1,234,567.89
    FORMAT(SUM(fs.GrossProfit)
           / NULLIF(SUM(fs.LineTotal), 0), 'P1')  AS MarginFormatted    -- 33.5%
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
ORDER BY SUM(fs.LineTotal) DESC;
```

9.3 Converting Integer Date Keys for Display

The YYYYMMDD integer date key used throughout the CabotTrail DW requires conversion for human-readable output:

```
-- Convert integer date key to a readable date label
SELECT
    fs.OrderDateKey,
    -- Method 1: Join to dim.Calendar (preferred — uses calendar dimension attributes)
    cal.FullDate,
    cal.MonthName + ' ' + CAST(cal.CalendarYear AS NVARCHAR(4)) AS MonthYearLabel,
    -- Method 2: Parse the integer directly (when calendar join is not available)
    DATEFROMPARTS(
        fs.OrderDateKey / 10000,            -- Year: first 4 digits
        (fs.OrderDateKey / 100) % 100,      -- Month: digits 5-6
        fs.OrderDateKey % 100               -- Day: last 2 digits
    )                                       AS ParsedDate
FROM    fact.Sales fs
INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
ORDER BY fs.OrderDateKey;
```

`DATEFROMPARTS(year, month, day)` constructs a DATE from its components — useful when working with YYYYMMDD integer keys without a calendar dimension.

---

10. User-Defined Functions: Scalar UDFs

A **User-Defined Function (UDF)** is a named, reusable piece of SQL logic stored in the database. A scalar UDF takes input parameters, performs a calculation, and returns a single value.

10.1 Why UDFs Matter

Consider a margin percentage calculation that appears in a dozen different queries:

```
ROUND(GrossProfit / NULLIF(LineTotal, 0) * 100, 2)
```

If the business changes the definition of margin — perhaps now rounding to 3 decimal places, or using a different base — every query containing this expression must be found and updated. A UDF encapsulates the logic once:

```
-- Define once: dbo.fn_MarginPct(LineTotal, GrossProfit)
-- Use everywhere: dbo.fn_MarginPct(fs.LineTotal, fs.GrossProfit)
-- Change once: update the function, all queries update automatically
```

10.2 Creating a Scalar UDF

```
-- Create a function that calculates gross profit margin percentage
USE CabotTrailOutdoorsSales;
GO

CREATE FUNCTION dbo.fn_MarginPct
(
    @LineTotal      DECIMAL(18,2),
    @GrossProfit    DECIMAL(18,2)
)
RETURNS DECIMAL(8,2)
AS
BEGIN
    RETURN
        CASE
            WHEN @LineTotal = 0 OR @LineTotal IS NULL THEN 0
            ELSE ROUND(@GrossProfit / @LineTotal * 100, 2)
        END
END;
GO
```

The function:

- Takes two DECIMAL parameters: `@LineTotal` and `@GrossProfit`
- Returns a DECIMAL(8,2) value
- Handles the divide-by-zero case (NULL or zero LineTotal → returns 0)
- The `BEGIN ... END` block contains the function body
- `RETURN` specifies the value returned

10.3 Calling a Scalar UDF

UDFs are called by prefixing with the schema name (`dbo.`):

```
-- Use the UDF in a query
SELECT  TOP 10
        InvoiceID,
        LineTotal,
        GrossProfit,
        -- Stored column (for comparison)
        GrossProfitMarginPct                        AS StoredMarginPct,
        -- UDF calculation
        dbo.fn_MarginPct(LineTotal, GrossProfit)    AS UDF_MarginPct,
        -- Verify they match
        GrossProfitMarginPct
        - dbo.fn_MarginPct(LineTotal, GrossProfit)  AS Discrepancy
FROM    fact.Sales
WHERE   LineTotal > 0
ORDER BY LineTotal DESC;
```

10.4 A More Complex UDF: Sales Territory Lookup

```
-- Function that returns the sales territory for a given province code
CREATE FUNCTION dbo.fn_GetSalesTerritory
(
    @ProvinceCode   NCHAR(2)
)
RETURNS NVARCHAR(60)
AS
BEGIN
    RETURN
        CASE @ProvinceCode
            WHEN 'NS' THEN 'Atlantic — Nova Scotia'
            WHEN 'NB' THEN 'Atlantic — New Brunswick'
            WHEN 'PE' THEN 'Atlantic — PEI'
            WHEN 'NL' THEN 'Atlantic — Newfoundland'
            WHEN 'QC' THEN 'Quebec'
            WHEN 'ON' THEN 'Ontario'
            WHEN 'MB' THEN 'Prairies'
            WHEN 'SK' THEN 'Prairies'
            WHEN 'AB' THEN 'Alberta'
            WHEN 'BC' THEN 'British Columbia'
            ELSE           'Other'
        END
END;
GO

-- Use the territory function
SELECT  CustomerName,
        ProvinceCode,
        dbo.fn_GetSalesTerritory(ProvinceCode)  AS DerivedTerritory,
        SalesTerritory                          AS StoredTerritory
FROM    dim.Customer
ORDER BY ProvinceCode;
```

10.5 Updating and Dropping UDFs

```
-- Update an existing UDF
ALTER FUNCTION dbo.fn_MarginPct
(
    @LineTotal      DECIMAL(18,2),
    @GrossProfit    DECIMAL(18,2)
)
RETURNS DECIMAL(8,3)    -- Changed to 3 decimal places
AS
BEGIN
    RETURN CASE
        WHEN @LineTotal = 0 OR @LineTotal IS NULL THEN 0
        ELSE ROUND(@GrossProfit / @LineTotal * 100, 3)
    END
END;
GO

-- Drop a UDF when it is no longer needed
DROP FUNCTION IF EXISTS dbo.fn_MarginPct;
```

10.6 UDF Performance Considerations

Scalar UDFs in SQL Server have historically been a performance concern. Each UDF call executes as a separate, non-inlineable operation — calling a scalar UDF on 16,359 rows means 16,359 separate function invocations.

SQL Server 2019 introduced **scalar UDF inlining** — automatically transforming scalar UDFs into equivalent inline expressions during query compilation. Most simple scalar UDFs (like `fn_MarginPct`) are inlined automatically, eliminating the performance penalty.

If you are on SQL Server 2017 or earlier, consider replacing scalar UDFs in large-table queries with inline expressions or inline table-valued functions (covered in the Deeper Dive).

---

11. Stored Procedures

A **stored procedure** is a named, reusable collection of SQL statements stored in the database. Unlike UDFs, stored procedures can execute DML statements, contain control flow logic, return multiple result sets, and accept both input and output parameters.

11.1 Why Stored Procedures

Stored procedures serve two related purposes for analysts:

Encapsulation: Complex analytical queries can be saved as named procedures and called with simple `EXEC` statements — useful for reports that run regularly.

Parameterisation: Parameters make a procedure flexible — the same procedure answers "revenue by category in 2024" and "revenue by category in 2023" by changing one parameter value.

11.2 Creating a Stored Procedure

```
-- Create a procedure: sales summary for a given year and optional territory
CREATE PROCEDURE dbo.usp_SalesSummary
    @Year           INT,
    @Territory      NVARCHAR(60) = NULL     -- Optional: NULL = all territories
AS
BEGIN
    -- SET NOCOUNT ON suppresses the "rows affected" message
    SET NOCOUNT ON;

    SELECT  cust.SalesTerritory,
            prod.CategoryName,
            cal.CalendarYear,
            cal.QuarterName,
            COUNT(DISTINCT fs.InvoiceID)    AS Invoices,
            SUM(fs.LineTotal)               AS Revenue,
            ROUND(
                SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100,
            2)                              AS MarginPct
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    INNER JOIN dim.Product  prod ON prod.ProductKey  = fs.ProductKey
    INNER JOIN dim.Calendar cal  ON cal.DateKey      = fs.OrderDateKey
    WHERE   cal.CalendarYear        = @Year
    AND     (
                @Territory IS NULL
                OR cust.SalesTerritory = @Territory
            )
    GROUP BY cust.SalesTerritory, prod.CategoryName,
             cal.CalendarYear, cal.QuarterNumber, cal.QuarterName
    ORDER BY cust.SalesTerritory, cal.QuarterNumber, Revenue DESC;
END;
GO
```

11.3 Executing Stored Procedures

```
-- Execute with both parameters
EXEC dbo.usp_SalesSummary
    @Year       = 2024,
    @Territory  = 'Atlantic — Nova Scotia';

-- Execute with only the required parameter (territory defaults to NULL = all)
EXEC dbo.usp_SalesSummary @Year = 2024;

-- Execute using positional arguments (no parameter names)
EXEC dbo.usp_SalesSummary 2024, 'Atlantic — Nova Scotia';
```

11.4 Modifying and Dropping Procedures

```
-- Modify a procedure
ALTER PROCEDURE dbo.usp_SalesSummary
    @Year       INT,
    @Territory  NVARCHAR(60) = NULL,
    @MinRevenue DECIMAL(18,2) = 0   -- New optional parameter
AS
BEGIN
    -- Updated procedure body
END;
GO

-- Drop a procedure
DROP PROCEDURE IF EXISTS dbo.usp_SalesSummary;
```

11.5 Procedure vs UDF: When to Use Each

Scenario	Use
Single calculated value used in a SELECT expression	Scalar UDF
Complex business logic returning a set of rows	Stored procedure
Report that runs on a schedule	Stored procedure
Calculation encapsulated for reuse across many queries	Scalar UDF or inline TVF
DML operations (INSERT, UPDATE, DELETE)	Stored procedure (UDFs cannot execute DML)
Called from a BI tool or application	Stored procedure

12. Chapter Summary

- **Window functions** compute across a set of rows (the window) while preserving all individual rows — unlike aggregate functions, which collapse rows.
- The **OVER clause** defines the window. An empty `OVER ()` uses the entire result set. `ORDER BY` inside `OVER` creates an ordered window for running calculations.
- **PARTITION BY** divides rows into sub-groups — like `GROUP BY` for window functions, but without collapsing rows.
- **Ranking functions:** `ROW_NUMBER()` — unique sequential numbers; `RANK()` — gaps after ties; `DENSE_RANK()` — no gaps. Use `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` for top-N-per-group.
- **Offset functions:** `LAG(col, offset, default)` looks backward; `LEAD(col, offset, default)` looks forward. Used for period-over-period comparisons — month-over-month, year-over-year.
- **Running totals** use `SUM() OVER (ORDER BY col ROWS UNBOUNDED PRECEDING)`. **Moving averages** use `AVG() OVER (ORDER BY col ROWS BETWEEN n PRECEDING AND CURRENT ROW)`.
- The **frame clause** (`ROWS BETWEEN ... AND ...`) controls which rows are in the window. Use `ROWS` over `RANGE` for predictable, deterministic results.
- **Scalar UDFs** encapsulate a reusable calculation. Create with `CREATE FUNCTION`, call with `schema.fn_name(arguments)`. SQL Server 2019+ inlines most scalar UDFs automatically.
- **Stored procedures** encapsulate complex query logic with optional parameters. Create with `CREATE PROCEDURE`, execute with `EXEC procedure_name @param = value`.

---

13. Review Questions

1. Explain the fundamental difference between `GROUP BY` with `SUM` and a window function `SUM() OVER ()`. Write two queries against `fact.Sales` that demonstrate this difference — one using GROUP BY (returning one row per category) and one using the window function (returning all rows with the category total on each).
2. Write a query that returns every row from `fact.Sales` with three ranking columns:
   - `OverallRank` : ranked by `LineTotal` descending across all rows using `RANK()`
   - `CategoryRank` : ranked by `LineTotal` descending within each product category using `RANK()` with `PARTITION BY`
   - `UniqueRowNum` : a unique row number using `ROW_NUMBER()`

Join to `dim.Product` for the category name. Show only the top 5 rows per category (`CategoryRank <= 5`).

3. Using a CTE for monthly revenue (`CalendarYear`, `MonthNumber`, `YearMonthName`, `Revenue`), write a query that adds:
   - `PrevMonthRevenue`: using `LAG`
   - `MoMChangePct`: month-over-month change as a percentage, handling divide-by-zero with `NULLIF`
   - `YTD_Revenue`: year-to-date running total, resetting each year using `PARTITION BY CalendarYear`
   Sort chronologically.
4. Write a query that shows each product category's revenue AND its percentage of total revenue, using `SUM(SUM(LineTotal)) OVER ()` to get the grand total. Include `CategoryName`, `CategoryRevenue`, and `RevSharePct`. Round to 2 decimal places. Sort by revenue descending.
5. Create a scalar UDF called `dbo.fn_DayLabel` that accepts a `DATE` parameter and returns `'Weekend'` if the day is Saturday or Sunday, and `'Weekday'` otherwise. Use `DATEPART(weekday, @date)` — weekday values are 1=Sunday, 7=Saturday. Then write a query against `dim.Calendar` that uses the function to label each date, filtering to January 2024.
6. Write a 3-month centred moving average query (1 row before, current row, 1 row after) for monthly revenue. Use the frame `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`. Show `YearMonthName`, `Revenue`, and `CentredMovingAvg`. Explain what happens to the first and last row in the result.
7. Create a stored procedure called `dbo.usp_CustomerReport` that accepts `@Territory NVARCHAR(60)` (required) and `@Year INT = NULL` (optional, NULL = all years). The procedure should return: `CustomerName`, `TotalRevenue`, `InvoiceCount`, `AvgOrderValue`, and `MarginPct`. Show the EXEC statement that calls it for Atlantic — Nova Scotia in 2024.
8. A colleague says: "I can always replace a window function with a self-join or subquery, so window functions are just syntactic sugar." Evaluate this claim. Are there cases where window functions cannot be replicated by other means? Give a specific example.

Deeper Dive

Going Further with Window Functions and Reusable Code

NTILE: Percentile Ranking

`NTILE(n)` divides the rows in the window into `n` equal-sized buckets and assigns each row a bucket number from 1 to n. It is used for percentile and quartile analysis:

```
-- Classify customers into revenue quartiles
WITH CustomerRevenue AS (
    SELECT  cust.CustomerName,
            SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cust.CustomerName
)
SELECT  CustomerName,
        Revenue,
        NTILE(4) OVER (ORDER BY Revenue DESC) AS RevenueQuartile
        -- 1 = top 25%, 2 = next 25%, 3 = next 25%, 4 = bottom 25%
FROM    CustomerRevenue
ORDER BY RevenueQuartile, Revenue DESC;
```

Combined with a CASE expression, `NTILE` creates customer tier labels:

```
CASE NTILE(4) OVER (ORDER BY Revenue DESC)
    WHEN 1 THEN 'Platinum'
    WHEN 2 THEN 'Gold'
    WHEN 3 THEN 'Silver'
    ELSE        'Bronze'
END AS CustomerTier
```

FIRST_VALUE and LAST_VALUE

`FIRST_VALUE(column)` and `LAST_VALUE(column)` return the first and last values in the window — useful for comparing each row to the beginning or end of a group:

```
-- Each month's revenue compared to the first and last month of the year
WITH MonthlyRevenue AS (
    SELECT
        cal.CalendarYear, cal.MonthNumber, cal.YearMonthName,
        SUM(fs.LineTotal) AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cal.YearMonthName
)
SELECT
    YearMonthName, Revenue,
    FIRST_VALUE(Revenue) OVER (
        PARTITION BY CalendarYear
        ORDER BY MonthNumber
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    )                           AS JanuaryRevenue,
    LAST_VALUE(Revenue) OVER (
        PARTITION BY CalendarYear
        ORDER BY MonthNumber
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    )                           AS DecemberRevenue
FROM    MonthlyRevenue
ORDER BY CalendarYear, MonthNumber;
```

Note: `LAST_VALUE` requires the full frame `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` — without it, the default frame stops at the current row, making `LAST_VALUE` return the current row's value.

Inline Table-Valued Functions

A **scalar UDF** returns a single value. An **inline table-valued function (iTVF)** returns a table — a set of rows. iTVFs are more powerful and (on SQL Server 2014+) are always inlined by the optimizer, avoiding the performance concerns of scalar UDFs:

```
-- Inline table-valued function: returns customer summary as a table
CREATE FUNCTION dbo.fn_CustomerSummary
(
    @Year   INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT  cust.CustomerName,
            cust.SalesTerritory,
            COUNT(DISTINCT fs.InvoiceID)    AS InvoiceCount,
            SUM(fs.LineTotal)               AS TotalRevenue,
            ROUND(SUM(fs.GrossProfit)
                  / NULLIF(SUM(fs.LineTotal), 0) * 100, 2) AS MarginPct
    FROM    fact.Sales fs
    INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
    INNER JOIN dim.Calendar  cal ON cal.DateKey      = fs.OrderDateKey
    WHERE   cal.CalendarYear = @Year
    GROUP BY cust.CustomerName, cust.SalesTerritory
);
GO

-- Call like a table in a FROM clause
SELECT  *
FROM    dbo.fn_CustomerSummary(2024)
ORDER BY TotalRevenue DESC;

-- Join to other tables
SELECT  cs.CustomerName, cs.TotalRevenue, cust.CreditLimit
FROM    dbo.fn_CustomerSummary(2024) cs
INNER JOIN dim.Customer cust ON cust.CustomerName = cs.CustomerName
ORDER BY cs.TotalRevenue DESC;
```

iTVFs combine the reusability of UDFs with the flexibility of a table — they can be joined, filtered, and grouped just like any other table.

Microsoft documentation:
User-Defined Functions (SQL Server)

Window Functions in Other Databases

Window functions are part of the SQL standard (ISO/IEC 9075) and supported by all major databases, with minor syntax differences:

Feature	SQL Server	PostgreSQL	BigQuery/Snowflake
`ROW_NUMBER()`	✓	✓	✓
`RANK()`	✓	✓	✓
`LAG` / `LEAD`	✓	✓	✓
`ROWS BETWEEN`	✓	✓	✓
`RANGE BETWEEN`	✓	✓	✓

The window function syntax in this chapter transfers directly to PostgreSQL, MySQL 8+, Snowflake, BigQuery, and DuckDB. This is one of the most portable parts of advanced SQL.

Industry Perspectives

Window Functions and Modern Analytics

Window functions were introduced to the SQL standard in SQL:2003 and have become one of the most widely used features in analytical SQL. In the Stack Overflow Developer Survey and data analytics tool usage reports, window functions consistently appear among the most valuable SQL features for working analysts.

The year-over-year comparison (`LAG` offset 12), the running YTD total (`SUM OVER PARTITION BY year ORDER BY month`), and the top-N-per-group pattern (`ROW_NUMBER OVER PARTITION BY category`) appear in virtually every financial dashboard and performance report. Learning these patterns is not an advanced specialisation — it is foundational to the analyst role.

When Stored Procedures and UDFs Are Part of the Architecture

In many organisations, analytical results are delivered not by BI tools querying tables directly but by stored procedures whose outputs are consumed by applications. An analyst who understands stored procedures can:

- Call them from SSMS to verify results
- Read their code to understand what data processing they perform
- Propose optimisations when performance degrades
- Write new procedures when BI tools cannot express the required logic

Even if you never write a stored procedure from scratch in your job, reading and understanding them is a core production competency.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapter 7 covers window functions comprehensively with performance analysis and frame behaviour.
2. Ben-Gan, I. (2012). *Microsoft SQL Server 2012 High-Performance T-SQL Using Window Functions*. Microsoft Press. — Dedicated entirely to window functions; the most thorough treatment available.
3. Microsoft. (2024). *Window functions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql
4. Microsoft. (2024). *Ranking Functions (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/ranking-functions-transact-sql
5. Microsoft. (2024). *LAG (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/functions/lag-transact-sql
6. Microsoft. (2024). *CREATE FUNCTION (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-function-transact-sql
7. Microsoft. (2024). *CREATE PROCEDURE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql
8. Microsoft. (2024). *Scalar UDF Inlining*. https://learn.microsoft.com/en-us/sql/relational-databases/user-defined-functions/scalar-udf-inlining

Chapter 9: Moving Data — DDL, DML, and Source-to-Target Scripts

Chapter Overview

Every chapter up to this point has been about reading data — retrieving, filtering, joining, aggregating, and analysing rows that already exist. This final chapter crosses to the other side: creating the structures that hold data and moving data from one place to another.

This is where SQL for analysts meets SQL for data engineers. The skills here — designing a target table, writing a source-to-target INSERT, verifying the result — are the foundation of the ETL work you will do in DBAS 2103. They are also the skills you need as an analyst when you need to save a complex query's result for reuse, build a summary table for a report, or prototype a new analytical structure.

By the end of this chapter you will be able to:

- Write `CREATE TABLE` statements with appropriate data types and constraints
- Explain primary keys, foreign keys, and the constraints that enforce data integrity
- Write `ALTER TABLE` statements to modify existing structures
- Write `INSERT INTO ... VALUES` to add individual rows
- Write `INSERT INTO ... SELECT` to populate a table from a query
- Write `SELECT INTO` for rapid prototyping
- Write `UPDATE` and `DELETE` statements with the SELECT-first discipline
- Write a complete source-to-target script that creates a target, populates it, and verifies the result
- Explain the relationship between the T-SQL skills in this book and the SSIS ETL work in DBAS 2103

Table of Contents

1. DDL: Creating Database Structures
2. CREATE TABLE: Defining a Target
3. Constraints: Enforcing Business Rules
4. ALTER TABLE: Modifying Structures
5. DROP TABLE: Removing Structures
6. DML Revisited: INSERT, UPDATE, DELETE
7. INSERT INTO … SELECT: Populating from a Query
8. SELECT INTO: Rapid Prototyping
9. Writing a Source-to-Target Script
10. Verifying Your Work: Reconciliation Queries
11. From T-SQL Scripts to SSIS Packages
12. Looking Back, Looking Forward
13. Chapter Summary
14. Review Questions
15. Deeper Dive

1. DDL: Creating Database Structures

In Chapter 3 you read data types from the system catalog. In this chapter you use them to define your own tables. **Data Definition Language (DDL)** is the SQL category that creates, modifies, and removes database objects.

1.1 The DDL Statement Family

Statement	What it does
`CREATE TABLE`	Defines a new table with its columns and constraints
`ALTER TABLE`	Modifies an existing table (add/remove/change columns or constraints)
`DROP TABLE`	Permanently removes a table and all its data
`CREATE INDEX`	Creates an index to speed up queries
`DROP INDEX`	Removes an index
`TRUNCATE TABLE`	Removes all rows from a table (faster than DELETE, minimal logging)

DDL changes the **structure** of the database. DML changes the **data** in that structure. The distinction matters for permissions — DBAs often restrict who can run DDL while allowing analysts broader DML access.

1.2 The Design-Before-Build Principle

Chapter 6 of the companion textbook *ETL for Business Intelligence* established the principle: **design before you build**. Define the target structure — column names, data types, constraints — before writing any INSERT statements. The reason: changing a table structure after data exists is possible but disruptive. Choosing the right data type at creation costs five minutes; migrating data to a different type after millions of rows exist costs days.

For this chapter's exercises, the design step is as important as the SQL step.

---

2. CREATE TABLE: Defining a Target

2.1 Basic Syntax

```
CREATE TABLE schema_name.table_name
(
    column1_name    data_type       [NULL | NOT NULL] [DEFAULT value],
    column2_name    data_type       [NULL | NOT NULL] [DEFAULT value],
    ...
    [CONSTRAINT constraint_name constraint_type (...)]
);
```

2.2 A First Table

```
-- Create a monthly sales summary table
USE CabotTrailOutdoorsSales;
GO

CREATE TABLE dbo.MonthlySalesSummary
(
    SummaryID       INT             NOT NULL IDENTITY(1,1),
    CalendarYear    SMALLINT        NOT NULL,
    MonthNumber     TINYINT         NOT NULL,
    MonthName       NVARCHAR(10)    NOT NULL,
    SalesTerritory  NVARCHAR(60)    NOT NULL,
    InvoiceCount    INT             NOT NULL    DEFAULT 0,
    SalesLineCount  INT             NOT NULL    DEFAULT 0,
    TotalRevenue    DECIMAL(18,2)   NOT NULL    DEFAULT 0,
    TotalGrossProfit DECIMAL(18,2)  NOT NULL    DEFAULT 0,
    AvgMarginPct    DECIMAL(8,2)    NULL,
    CreatedDate     DATE            NOT NULL    DEFAULT GETDATE()
);
```

Walk through each column:

- `SummaryID INT NOT NULL IDENTITY(1,1)` — auto-incrementing integer primary key candidate. `IDENTITY(1,1)` means start at 1, increment by 1.
- `CalendarYear SMALLINT NOT NULL` — year number. `SMALLINT` is sufficient (range ±32,767) and uses 2 bytes instead of 4.
- `MonthNumber TINYINT NOT NULL` — 1–12. `TINYINT` (0–255, 1 byte) is the right size.
- `SalesTerritory NVARCHAR(60) NOT NULL` — variable-length Unicode text.
- `TotalRevenue DECIMAL(18,2) NOT NULL DEFAULT 0` — exact decimal for financial amounts; default of 0 prevents NULL in a mandatory column.
- `AvgMarginPct DECIMAL(8,2) NULL` — nullable because it might not be computable (if revenue is 0).
- `CreatedDate DATE NOT NULL DEFAULT GETDATE()` — automatically populated with today's date when a row is inserted.

2.3 Choosing Data Types: The Design Decisions

Every column in the CREATE TABLE statement above reflects a design decision:

Question	Answer in this table
Can this column be empty?	`AvgMarginPct` may be NULL (no sales); all others NOT NULL
What is the largest possible value?	SMALLINT for year, TINYINT for month number
Is Unicode needed?	Yes for territory name (`NVARCHAR`)
What precision is needed for financial values?	`DECIMAL(18,2)` — 2 decimal places, large total range
Should a default be provided?	Revenue/count columns default to 0; CreatedDate defaults to today

These decisions are not arbitrary. Choosing `INT` instead of `TINYINT` for `MonthNumber` wastes 3 bytes per row — small at 100 rows, noticeable at 100 million. Choosing `FLOAT` instead of `DECIMAL` for revenue invites rounding errors. Taking 10 minutes to think through these choices before writing the DDL is time well spent.

---

3. Constraints: Enforcing Business Rules

Constraints are rules applied to a table that the database engine enforces automatically. Every row inserted or updated must satisfy all constraints — violations are rejected with an error before any data is changed.

3.1 Primary Key Constraint

The primary key uniquely identifies each row. No two rows can have the same primary key value, and the column cannot be NULL:

```
CREATE TABLE dbo.MonthlySalesSummary
(
    SummaryID       INT             NOT NULL IDENTITY(1,1),
    CalendarYear    SMALLINT        NOT NULL,
    ...
    CONSTRAINT PK_MonthlySalesSummary
        PRIMARY KEY (SummaryID)
);
```

Naming constraints explicitly (`PK_MonthlySalesSummary`) makes them identifiable in error messages and system catalog queries. SQL Server generates names for unnamed constraints, but they are cryptic and hard to reference.

3.2 Unique Constraint

A unique constraint ensures no two rows have the same value(s) in the specified column(s):

```
-- No two rows can have the same year + month + territory combination
CONSTRAINT UQ_MonthlySalesSummary_Period
    UNIQUE (CalendarYear, MonthNumber, SalesTerritory)
```

This prevents duplicate summary rows — a critical data quality rule for summary tables. Attempting to insert a second row for 'Atlantic — Nova Scotia', year 2024, month 3 will fail.

3.3 CHECK Constraint

A check constraint validates that column values satisfy a logical condition:

```
-- Month must be between 1 and 12
CONSTRAINT CK_MonthlySalesSummary_Month
    CHECK (MonthNumber BETWEEN 1 AND 12),

-- Revenue cannot be negative
CONSTRAINT CK_MonthlySalesSummary_Revenue
    CHECK (TotalRevenue >= 0),

-- Margin percentage must be between 0 and 100 (or NULL)
CONSTRAINT CK_MonthlySalesSummary_Margin
    CHECK (AvgMarginPct IS NULL OR AvgMarginPct BETWEEN 0 AND 100)
```

3.4 Foreign Key Constraint

A foreign key links a column in one table to the primary key of another, enforcing referential integrity:

```
-- If SummaryID references another table's FK, a foreign key constraint enforces the link
CONSTRAINT FK_SomeTable_Summary
    FOREIGN KEY (SummaryID) REFERENCES dbo.MonthlySalesSummary (SummaryID)
```

For standalone summary tables like `MonthlySalesSummary`, foreign keys to the source dimension tables are optional — they are useful for enforcing that `CalendarYear` and `MonthNumber` values are valid, but they create a coupling between the summary table and the dimension tables that may not always be desirable.

3.5 The Complete CREATE TABLE with All Constraints

```
DROP TABLE IF EXISTS dbo.MonthlySalesSummary;
GO

CREATE TABLE dbo.MonthlySalesSummary
(
    SummaryID           INT             NOT NULL IDENTITY(1,1),
    CalendarYear        SMALLINT        NOT NULL,
    MonthNumber         TINYINT         NOT NULL,
    MonthName           NVARCHAR(10)    NOT NULL,
    SalesTerritory      NVARCHAR(60)    NOT NULL,
    InvoiceCount        INT             NOT NULL    DEFAULT 0,
    SalesLineCount      INT             NOT NULL    DEFAULT 0,
    TotalRevenue        DECIMAL(18,2)   NOT NULL    DEFAULT 0,
    TotalGrossProfit    DECIMAL(18,2)   NOT NULL    DEFAULT 0,
    AvgMarginPct        DECIMAL(8,2)    NULL,
    CreatedDate         DATE            NOT NULL    DEFAULT GETDATE(),

    CONSTRAINT PK_MonthlySalesSummary
        PRIMARY KEY (SummaryID),
    CONSTRAINT UQ_MonthlySalesSummary_Period
        UNIQUE (CalendarYear, MonthNumber, SalesTerritory),
    CONSTRAINT CK_MonthlySalesSummary_Month
        CHECK (MonthNumber BETWEEN 1 AND 12),
    CONSTRAINT CK_MonthlySalesSummary_Revenue
        CHECK (TotalRevenue >= 0),
    CONSTRAINT CK_MonthlySalesSummary_Margin
        CHECK (AvgMarginPct IS NULL OR AvgMarginPct BETWEEN 0 AND 100)
);
GO
```

The `DROP TABLE IF EXISTS` at the top makes the script safe to re-run — if the table already exists, it is dropped first. Without this, re-running the script would fail with "table already exists."

---

4. ALTER TABLE: Modifying Structures

`ALTER TABLE` modifies an existing table without rebuilding it from scratch.

4.1 Adding a Column

```
-- Add a column after the table exists
ALTER TABLE dbo.MonthlySalesSummary
ADD AvgInvoiceValue DECIMAL(18,2) NULL;
-- New columns must be NULL or have a DEFAULT when the table already has rows
-- (otherwise the database cannot populate the column for existing rows)
```

4.2 Changing a Column's Data Type

```
-- Increase precision of an existing column
ALTER TABLE dbo.MonthlySalesSummary
ALTER COLUMN AvgMarginPct DECIMAL(10,4) NULL;
-- Changing to higher precision is safe
-- Changing to lower precision may truncate existing data
```

4.3 Adding and Dropping Constraints

```
-- Add a constraint after creation
ALTER TABLE dbo.MonthlySalesSummary
ADD CONSTRAINT CK_MonthlySalesSummary_Year
    CHECK (CalendarYear BETWEEN 2000 AND 2100);

-- Drop a constraint by name
ALTER TABLE dbo.MonthlySalesSummary
DROP CONSTRAINT CK_MonthlySalesSummary_Year;
```

4.4 Renaming a Column

SQL Server uses a system procedure to rename columns:

```
-- Rename a column
EXEC sp_rename
    'dbo.MonthlySalesSummary.AvgInvoiceValue',
    'AvgLineValue',
    'COLUMN';
```

> ***Renaming carries risk:*** *any query, view, or stored procedure that references the old column name will break immediately. Always search for references before renaming a column in a production database.*

---

5. DROP TABLE: Removing Structures

`DROP TABLE` permanently removes the table and all its data:

```
-- Safe version: only drops if the table exists
DROP TABLE IF EXISTS dbo.MonthlySalesSummary;

-- Unsafe version: raises an error if the table does not exist
DROP TABLE dbo.MonthlySalesSummary;
```

DROP TABLE is permanent. There is no undo. Always verify you are dropping the right table in the right database before executing.

`TRUNCATE TABLE` removes all rows but keeps the table structure:

```
-- Remove all data, keep the table
TRUNCATE TABLE dbo.MonthlySalesSummary;

-- Compare: DROP removes both data and structure
-- TRUNCATE removes data only
```

`TRUNCATE` is faster than `DELETE` for clearing large tables — it deallocates data pages without logging individual row deletions. It cannot be used when FK constraints reference the table from other tables with data.

---

6. DML Revisited: INSERT, UPDATE, DELETE

Chapter 3 introduced DML concepts. This section extends them with the patterns you need for source-to-target work.

6.1 INSERT with Multiple Rows

`INSERT INTO ... VALUES` can insert multiple rows in a single statement:

```
INSERT INTO dbo.MonthlySalesSummary
    (CalendarYear, MonthNumber, MonthName, SalesTerritory,
     InvoiceCount, SalesLineCount, TotalRevenue, TotalGrossProfit, AvgMarginPct)
VALUES
    (2024, 1, 'January', 'Atlantic — Nova Scotia', 12, 87, 45230.50, 14925.07, 33.00),
    (2024, 1, 'January', 'Ontario',                 8, 54, 29100.00, 9603.00,  33.00),
    (2024, 2, 'February', 'Atlantic — Nova Scotia', 15, 103, 51420.75, 16968.85, 33.00);
-- Three rows inserted in one statement
```

6.2 UPDATE with the SELECT-First Discipline

Always verify the target rows with a `SELECT` before running `UPDATE`:

```
-- Step 1: See what you are about to update
SELECT  SummaryID, TotalRevenue, AvgMarginPct
FROM    dbo.MonthlySalesSummary
WHERE   CalendarYear  = 2024
AND     MonthNumber   = 1
AND     SalesTerritory = 'Atlantic — Nova Scotia';

-- Step 2: Run the UPDATE with the identical WHERE clause
UPDATE  dbo.MonthlySalesSummary
SET     AvgMarginPct = 32.95
WHERE   CalendarYear  = 2024
AND     MonthNumber   = 1
AND     SalesTerritory = 'Atlantic — Nova Scotia';

-- Step 3: Verify the change
SELECT  SummaryID, TotalRevenue, AvgMarginPct
FROM    dbo.MonthlySalesSummary
WHERE   CalendarYear  = 2024
AND     MonthNumber   = 1
AND     SalesTerritory = 'Atlantic — Nova Scotia';
```

6.3 UPDATE from a JOIN

`UPDATE` can reference another table using a JOIN — useful for updating a target table from source data:

```
-- Update TotalRevenue in the summary table from actual fact table data
UPDATE  mss
SET     mss.TotalRevenue = src.Revenue
FROM    dbo.MonthlySalesSummary mss
INNER JOIN (
    SELECT  cal.CalendarYear,
            cal.MonthNumber,
            cust.SalesTerritory,
            SUM(fs.LineTotal)   AS Revenue
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar  cal  ON cal.DateKey      = fs.OrderDateKey
    INNER JOIN dim.Customer  cust ON cust.CustomerKey = fs.CustomerKey
    GROUP BY cal.CalendarYear, cal.MonthNumber, cust.SalesTerritory
) src
    ON  src.CalendarYear   = mss.CalendarYear
    AND src.MonthNumber    = mss.MonthNumber
    AND src.SalesTerritory = mss.SalesTerritory;
```

This pattern — `UPDATE target FROM target JOIN (source query)` — is the T-SQL equivalent of an SSIS `OLE DB Command` transformation. It updates the target table in place rather than rebuilding it from scratch.

---

7. INSERT INTO … SELECT: Populating from a Query

`INSERT INTO ... SELECT` is the most important DML pattern for analytical SQL. It populates a table from any SELECT query — the entire ETL source-to-target process in a single statement.

7.1 Syntax

```
INSERT INTO target_table (col1, col2, col3, ...)
SELECT  expression1, expression2, expression3, ...
FROM    source_table
WHERE   condition;
```

The columns in the INSERT list and the expressions in the SELECT list must match in number and compatible data types. The SELECT can be any valid query — including joins, aggregations, subqueries, CTEs, and window functions.

7.2 Populating MonthlySalesSummary

```
-- Clear the table first (safe to re-run pattern)
TRUNCATE TABLE dbo.MonthlySalesSummary;

-- Populate from the star schema
INSERT INTO dbo.MonthlySalesSummary
(
    CalendarYear, MonthNumber, MonthName, SalesTerritory,
    InvoiceCount, SalesLineCount, TotalRevenue, TotalGrossProfit, AvgMarginPct
)
SELECT
    cal.CalendarYear,
    cal.MonthNumber,
    cal.MonthName,
    cust.SalesTerritory,
    COUNT(DISTINCT fs.InvoiceID)                                    AS InvoiceCount,
    COUNT(*)                                                        AS SalesLineCount,
    SUM(fs.LineTotal)                                               AS TotalRevenue,
    SUM(fs.GrossProfit)                                             AS TotalGrossProfit,
    ROUND(
        SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100,
    2)                                                              AS AvgMarginPct
FROM    fact.Sales fs
INNER JOIN dim.Calendar  cal  ON cal.DateKey      = fs.OrderDateKey
INNER JOIN dim.Customer  cust ON cust.CustomerKey = fs.CustomerKey
GROUP BY
    cal.CalendarYear,
    cal.MonthNumber,
    cal.MonthName,
    cust.SalesTerritory;
```

Note: `SummaryID` and `CreatedDate` are not in the INSERT column list — they have `IDENTITY` and `DEFAULT GETDATE()` respectively, so SQL Server populates them automatically.

7.3 Verifying the Insert

```
-- Check row count
SELECT COUNT(*) AS RowsInserted FROM dbo.MonthlySalesSummary;

-- Sample the results
SELECT TOP 10 *
FROM   dbo.MonthlySalesSummary
ORDER BY CalendarYear, MonthNumber, SalesTerritory;

-- Cross-check: total revenue in summary should match total revenue in fact table
SELECT
    'Source (fact.Sales)'              AS Layer,
    ROUND(SUM(LineTotal), 2)           AS TotalRevenue
FROM    fact.Sales
UNION ALL
SELECT
    'Target (MonthlySalesSummary)',
    ROUND(SUM(TotalRevenue), 2)
FROM    dbo.MonthlySalesSummary;
-- Both values must be equal
```

The reconciliation query — comparing source and target aggregate values — is the fundamental quality check for any source-to-target operation. If the totals match, the data moved correctly. If they differ, there is a defect to find.

---

8. SELECT INTO: Rapid Prototyping

`SELECT INTO` creates a new table and inserts rows in a single statement — faster to write than the two-step `CREATE TABLE` + `INSERT INTO ... SELECT`:

```
-- Create and populate a new table in one step
SELECT
    cal.CalendarYear,
    cal.MonthNumber,
    cal.MonthName,
    cust.SalesTerritory,
    COUNT(DISTINCT fs.InvoiceID)    AS InvoiceCount,
    SUM(fs.LineTotal)               AS TotalRevenue
INTO    dbo.MonthlySalesSummaryQuick    -- New table created here
FROM    fact.Sales fs
INNER JOIN dim.Calendar  cal  ON cal.DateKey      = fs.OrderDateKey
INNER JOIN dim.Customer  cust ON cust.CustomerKey = fs.CustomerKey
GROUP BY cal.CalendarYear, cal.MonthNumber, cal.MonthName, cust.SalesTerritory;

-- Verify
SELECT TOP 5 * FROM dbo.MonthlySalesSummaryQuick ORDER BY CalendarYear, MonthNumber;
```

8.1 What SELECT INTO Does Not Create

`SELECT INTO` creates the table with:

- The column names and data types derived from the SELECT list
- No primary key
- No indexes
- No constraints
- No defaults (except those inherited from the source if using `IDENTITY` columns)

For prototyping and one-off analysis, this is fine. For production tables that must enforce data integrity and perform well under query load, always use `CREATE TABLE` with explicit constraints followed by `INSERT INTO ... SELECT`.

8.2 SELECT INTO a Temporary Table

`SELECT INTO` with a `#` prefix creates a **temporary table** in `tempdb`:

```
-- Create a temporary table for intermediate calculations
SELECT
    fs.CustomerKey,
    SUM(fs.LineTotal)   AS TotalRevenue
INTO    #CustomerTotals     -- # prefix = temporary table in tempdb
FROM    fact.Sales fs
GROUP BY fs.CustomerKey;

-- Use the temp table in subsequent queries
SELECT  cust.CustomerName,
        ct.TotalRevenue
FROM    #CustomerTotals ct
INNER JOIN dim.Customer cust ON cust.CustomerKey = ct.CustomerKey
ORDER BY ct.TotalRevenue DESC;

-- Clean up
DROP TABLE IF EXISTS #CustomerTotals;
```

Temporary tables exist only for the current session and are automatically dropped when the session ends. They are ideal for breaking a complex multi-step query into manageable pieces when CTEs are not sufficient — for example, when the intermediate result needs to be referenced by multiple subsequent queries or benefits from an index.

---

9. Writing a Source-to-Target Script

A **source-to-target script** is a complete, self-contained SQL script that:

1. Creates the target structure (DDL)
2. Clears any existing data
3. Populates the target from the source
4. Verifies the result

This is the T-SQL equivalent of what SSIS packages do — the same logical steps, expressed as SQL statements rather than graphical components.

9.1 A Complete Source-to-Target Script

```
-- ============================================================
-- Script: Monthly Sales Rep Performance Summary
-- Source:  CabotTrailOutdoorsSales (fact.Sales + dimensions)
-- Target:  dbo.SalesRepMonthlySummary
-- Author:  [Your Name]
-- Date:    [Date]
-- Purpose: Creates and populates a monthly performance table
--          for use in sales rep dashboards
-- ============================================================

USE CabotTrailOutdoorsSales;
GO

-- ── Step 1: Create the target table ──────────────────────────
DROP TABLE IF EXISTS dbo.SalesRepMonthlySummary;
GO

CREATE TABLE dbo.SalesRepMonthlySummary
(
    SummaryKey          INT             NOT NULL IDENTITY(1,1),
    SalesRepName        NVARCHAR(100)   NOT NULL,
    Department          NVARCHAR(60)    NULL,
    CalendarYear        SMALLINT        NOT NULL,
    MonthNumber         TINYINT         NOT NULL,
    MonthName           NVARCHAR(10)    NOT NULL,
    YearMonthLabel      NVARCHAR(15)    NOT NULL,
    InvoiceCount        INT             NOT NULL    DEFAULT 0,
    SalesLineCount      INT             NOT NULL    DEFAULT 0,
    TotalRevenue        DECIMAL(18,2)   NOT NULL    DEFAULT 0,
    TotalGrossProfit    DECIMAL(18,2)   NOT NULL    DEFAULT 0,
    AvgMarginPct        DECIMAL(8,2)    NULL,
    AvgDaysToInvoice    DECIMAL(6,2)    NULL,

    CONSTRAINT PK_SalesRepMonthlySummary
        PRIMARY KEY (SummaryKey),
    CONSTRAINT UQ_SalesRepMonthlySummary_Period
        UNIQUE (SalesRepName, CalendarYear, MonthNumber),
    CONSTRAINT CK_SalesRepMonthlySummary_Month
        CHECK (MonthNumber BETWEEN 1 AND 12),
    CONSTRAINT CK_SalesRepMonthlySummary_Revenue
        CHECK (TotalRevenue >= 0)
);
GO

-- ── Step 2: Populate from source ─────────────────────────────
INSERT INTO dbo.SalesRepMonthlySummary
(
    SalesRepName, Department,
    CalendarYear, MonthNumber, MonthName, YearMonthLabel,
    InvoiceCount, SalesLineCount,
    TotalRevenue, TotalGrossProfit, AvgMarginPct, AvgDaysToInvoice
)
SELECT
```

```
    emp.FullName                                AS SalesRepName,
    emp.Department,
    cal.CalendarYear,
    cal.MonthNumber,
    cal.MonthName,
    -- Build a readable period label: 'Jan 2024'
    LEFT(cal.MonthName, 3) + ' '
        + CAST(cal.CalendarYear AS NVARCHAR(4))  AS YearMonthLabel,
    COUNT(DISTINCT fs.InvoiceID)                AS InvoiceCount,
    COUNT(*)                                    AS SalesLineCount,
    SUM(fs.LineTotal)                           AS TotalRevenue,
    SUM(fs.GrossProfit)                         AS TotalGrossProfit,
    ROUND(
        SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100,
    2)                                          AS AvgMarginPct,
    ROUND(AVG(CAST(fs.DaysUntilInvoice AS DECIMAL(6,2))), 2)
                                                AS AvgDaysToInvoice
FROM    fact.Sales fs
INNER JOIN dim.Employee emp  ON emp.EmployeeKey  = fs.EmployeeKey
INNER JOIN dim.Calendar  cal ON cal.DateKey      = fs.OrderDateKey
GROUP BY
    emp.FullName, emp.Department,
    cal.CalendarYear, cal.MonthNumber, cal.MonthName;
GO

-- ── Step 3: Verify ──────────────────────────────────────

-- 3a: Row count
SELECT  COUNT(*)    AS RowsLoaded FROM dbo.SalesRepMonthlySummary;

-- 3b: Revenue reconciliation (source vs target totals must match)
SELECT
    'Source (fact.Sales)'           AS Layer,
    ROUND(SUM(LineTotal), 2)        AS TotalRevenue,
    COUNT(DISTINCT InvoiceID)       AS TotalInvoices
FROM    fact.Sales
UNION ALL
SELECT
    'Target (SalesRepMonthlySummary)',
    ROUND(SUM(TotalRevenue), 2),
    SUM(InvoiceCount)
FROM    dbo.SalesRepMonthlySummary;

-- 3c: Sample output
SELECT TOP 15 *
FROM   dbo.SalesRepMonthlySummary
ORDER BY CalendarYear, MonthNumber, SalesRepName;
GO
```

9.2 The Anatomy of a Good Source-to-Target Script

Every production-quality source-to-target script has the same anatomy:

Section	Purpose
Comment block	Documents purpose, source, target, author, date
`USE` statement	Ensures the right database is active
Step 1: DDL	`DROP IF EXISTS` + `CREATE TABLE` with all constraints named
Step 2: Load	`TRUNCATE` or `DELETE`, then `INSERT INTO ... SELECT`
Step 3: Verify	Row counts, aggregate reconciliation, sample rows

The `DROP IF EXISTS` + `CREATE TABLE` pattern makes scripts **idempotent** — safe to run multiple times without error. Each run starts fresh.

---

10. Verifying Your Work: Reconciliation Queries

Reconciliation is the practice of verifying that data moved correctly from source to target. Three levels of verification, in increasing depth:

10.1 Level 1: Row Count

```
-- Does the target have the expected number of rows?
SELECT COUNT(*) AS TargetRows FROM dbo.SalesRepMonthlySummary;

-- Compare to source: how many (SalesRep, Year, Month) combinations exist?
SELECT COUNT(*) AS SourceCombinations
FROM (
    SELECT DISTINCT
        fs.EmployeeKey,
        cal.CalendarYear,
        cal.MonthNumber
    FROM    fact.Sales fs
    INNER JOIN dim.Calendar cal ON cal.DateKey = fs.OrderDateKey
) AS Combinations;
-- TargetRows should equal SourceCombinations
```

10.2 Level 2: Aggregate Reconciliation

```
-- Does total revenue in the target match total revenue in the source?
SELECT
    'Source' AS Layer, ROUND(SUM(LineTotal), 2) AS Revenue
FROM    fact.Sales
UNION ALL
SELECT
    'Target', ROUND(SUM(TotalRevenue), 2)
FROM    dbo.SalesRepMonthlySummary;
-- Both must return the same value
```

10.3 Level 3: Spot Check

```
-- Pick a specific (SalesRep, Year, Month) and verify each measure independently
DECLARE @Rep   NVARCHAR(100) = 'Alice MacLean';
DECLARE @Year  SMALLINT      = 2024;
DECLARE @Month TINYINT       = 3;

-- What the source says
SELECT
    COUNT(DISTINCT InvoiceID)              AS SourceInvoices,
    COUNT(*)                               AS SourceLines,
    ROUND(SUM(LineTotal), 2)               AS SourceRevenue,
    ROUND(SUM(GrossProfit)
          / NULLIF(SUM(LineTotal),0)*100,2) AS SourceMargin
FROM    fact.Sales fs
INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
INNER JOIN dim.Calendar  cal ON cal.DateKey     = fs.OrderDateKey
WHERE   emp.FullName      = @Rep
AND     cal.CalendarYear  = @Year
AND     cal.MonthNumber   = @Month;

-- What the target says
SELECT  InvoiceCount, SalesLineCount, TotalRevenue, AvgMarginPct
FROM    dbo.SalesRepMonthlySummary
WHERE   SalesRepName  = @Rep
AND     CalendarYear  = @Year
AND     MonthNumber   = @Month;
```

If all three levels pass — row counts match, aggregates match, spot checks agree — the source-to-target script is verified correct.

---

11. From T-SQL Scripts to SSIS Packages

Everything in this chapter has been implemented as T-SQL. In DBAS 2103 (Data Provisioning with ETL), you will build the same pipelines as SSIS packages. Understanding the relationship between the two approaches is the conceptual bridge between this course and the next.

11.1 The Same Steps, Different Tools

T-SQL step	SSIS equivalent
`USE DatabaseName`	OLE DB Connection Manager pointing to the database
`TRUNCATE TABLE target`	Execute SQL Task in Control Flow
`INSERT INTO ... SELECT`	Data Flow Task: OLE DB Source → OLE DB Destination
Complex joins in SELECT	Multiple tables in OLE DB Source SQL or Lookup transformations
`NULLIF` for divide-by-zero	Derived Column transformation with SSIS expression
Verification `SELECT COUNT(*)`	Execute SQL Task with result stored in variable
Mismatch alert	SSIS Failure path + Send Mail Task

The SSIS package adds **monitoring** (every execution is logged in the SSIS Catalog), **error handling** (individual rows can be redirected to error outputs), and **scheduling** (SQL Server Agent runs the package automatically). The T-SQL script adds none of these — it runs once when you press F5.

11.2 Why Learn Both

T-SQL source-to-target scripts are:
- Faster to write for ad-hoc or one-time loads
- Easier to understand, debug, and modify
- Ideal for prototyping before building the SSIS equivalent
- Version-controllable as plain text files
- Runnable by anyone with SSMS access

SSIS packages are:
- More appropriate for scheduled production loads
- Better for large-volume loads (streaming architecture)
- Superior for error handling at the row level
- Integrated with the monitoring and alerting infrastructure

The T-SQL script from section 9.1 and an SSIS package doing the same work are functionally equivalent. Many ETL pipelines in production use both — T-SQL for simple lookups and small loads, SSIS for the main fact table loads.

11.3 The Source-to-Target Mapping Connects Both

Whether you implement a data movement in T-SQL or SSIS, the specification comes from the same place: the **source-to-target mapping** document introduced in DBAS 2010. The mapping defines what each target column contains and how it is derived from the source. The T-SQL SELECT clause implements the mapping in SQL; the SSIS Data Flow implements the same mapping visually. The mapping is the constant; the implementation tool varies.

---

12. Looking Back, Looking Forward

12.1 What This Course Covered

Over nine chapters, this book has taken you from "what is a database?" to writing complete source-to-target pipelines. The journey:

Chapter	Core skill
1	Connecting to a database and writing SELECT
2	Filtering rows precisely with WHERE
3	Data types, arithmetic, and introductory DML
4	String, date, and NULL functions; CASE expressions
5	Joining tables; the star schema pattern
6	Aggregation: GROUP BY, HAVING, aggregate functions
7	Scripting: variables, subqueries, CTEs
8	Window functions, UDFs, and stored procedures
9	DDL, DML, and source-to-target scripts

12.2 The Analytical Foundation

The DQL skills — from basic SELECT through window functions — are the tools of the practising data analyst. They answer questions, support decisions, and surface insights from data that would otherwise remain invisible in rows and columns.

The DDL and DML skills — from CREATE TABLE through INSERT INTO ... SELECT — are the tools of the data engineer and the analyst who needs to move and transform data. They are the foundation of ETL work.

Together, they constitute the SQL literacy that makes a data professional effective in any environment with a relational database.

12.3 Where to Go Next

The three-course sequence continues:

DBAS 2010 — Database Design II applies the data type and constraint knowledge from this course to designing relational databases from business requirements. You will move from *querying* the CabotTrail databases to *designing* databases like them — entity-relationship diagrams, normalization, physical implementation.

DBAS 2103 — Data Provisioning with ETL builds the SSIS pipelines that implement the source-to-target patterns introduced in this chapter at production scale — with proper error handling, scheduling, monitoring, and governance.

The companion textbook *ETL for Business Intelligence* (the same author, same CabotTrail environment) is the reference for DBAS 2103 and covers all of these topics in depth.

---

13. Chapter Summary

- **DDL** creates and modifies database structures: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, `TRUNCATE TABLE`.
- `CREATE TABLE` defines column names, data types, nullability, and defaults. Choose the smallest data type that can represent all valid values; choose `DECIMAL` for financial amounts; use `NVARCHAR` for text.
- **Constraints** enforce business rules: `PRIMARY KEY` (unique, not null identifier), `UNIQUE` (no duplicate combinations), `CHECK` (value must satisfy a condition), `FOREIGN KEY` (references another table's PK). Always name constraints explicitly.
- `DROP TABLE IF EXISTS` before `CREATE TABLE` makes scripts idempotent — safe to re-run.

- `INSERT INTO ... SELECT` is the core source-to-target pattern: defines target columns, queries the source, and populates the target in one statement. Always `TRUNCATE` or `DELETE` before re-populating.
- `SELECT INTO` creates a table and populates it in one step — convenient for prototyping. It does not create indexes, constraints, or primary keys.
- A complete **source-to-target script** has four sections: comment block, DDL (create target), DML (populate from source), verification (row counts and aggregate reconciliation).
- **Reconciliation** verifies the load at three levels: row count, aggregate total, and spot check. All three must pass before the load is considered correct.
- T-SQL source-to-target scripts and SSIS packages implement the same logical steps with different tools. T-SQL is faster for prototyping; SSIS adds monitoring, error handling, and scheduling for production.

---

14. Review Questions

1. Write a `CREATE TABLE` statement for a table called `dbo.ProductPerformanceSummary` that stores total revenue, total units sold, and return rate by product and year. Include appropriate data types, a primary key, a unique constraint preventing duplicate (product, year) rows, and at least two CHECK constraints. Name all constraints explicitly.
2. Explain the difference between `DROP TABLE`, `TRUNCATE TABLE`, and `DELETE FROM table_name` (with no WHERE clause). When would you use each, and what are the key distinctions in terms of speed, recoverability, and FK constraint behaviour?
3. Write a complete source-to-target script (all four sections: comment block, DDL, populate, verify) that creates and populates `dbo.TerritoryQuarterlySummary` from `CabotTrailOutdoorsSales`. The target should have one row per (SalesTerritory, CalendarYear, QuarterNumber) with total revenue, invoice count, and average margin. Include the full reconciliation check.
4. A developer uses `SELECT INTO` to create a reporting table and then complains that queries against it are slow. Identify two specific things `SELECT INTO` does not create that would help query performance, and show the SQL to add them after the fact.
5. Write an `UPDATE` statement that corrects `AvgMarginPct` in `dbo.SalesRepMonthlySummary` by recomputing it from `fact.Sales`. Use the SELECT-first discipline — show the SELECT step that identifies the rows to be updated before running the UPDATE.
6. A reconciliation check shows that the target table has 50 rows but the source query (counting distinct SalesRep + Year + Month combinations) returns 52. What are the two most likely causes of this discrepancy, and how would you diagnose each?

7. Explain why `INSERT INTO dbo.SalesRepMonthlySummary (...) SELECT ...` requires the column list `(...)` to be specified explicitly, even though the SELECT list has the same columns in the same order. When is the column list safe to omit, and when is it risky?
8. The companion textbook *ETL for Business Intelligence* describes a T-SQL source-to-target script and an SSIS package as "functionally equivalent." Describe two specific scenarios where the SSIS approach is clearly preferable to a T-SQL script, and one scenario where the T-SQL script is preferable.

Deeper Dive

Going Further with DDL, DML, and Data Movement

Schema Design Principles

This chapter introduced enough DDL to build functional tables. Designing *good* tables — ones that support the queries you need, perform well, and remain maintainable — requires applying the normalization principles covered in DBAS 2010.

For summary tables like `SalesRepMonthlySummary`, the key design questions are:

Grain: What does one row represent? (One sales rep + one month = one row.) The grain determines the primary key candidate.

Measures: Are all stored values truly additive? (`TotalRevenue` is additive; `AvgMarginPct` is not — it should be derived at query time from the additive components, or at minimum documented as non-additive.)

Freshness: How often is the table repopulated? Does a TRUNCATE + full reload make sense, or is an incremental MERGE approach needed?

These questions connect directly to the dimensional modelling concepts in the ETL book's Chapters 2 and 3.

MERGE: The Upsert Pattern

Chapter 7 of *ETL for Business Intelligence* covered the `MERGE` statement in depth. For source-to-target scripts that need to **update existing rows and insert new ones** rather than truncate and reload, `MERGE` is the standard tool:

```
-- MERGE: update existing rows, insert new ones
MERGE INTO dbo.SalesRepMonthlySummary AS tgt
USING (
    SELECT
        emp.FullName                            AS SalesRepName,
        emp.Department,
        cal.CalendarYear, cal.MonthNumber, cal.MonthName,
        LEFT(cal.MonthName,3) + ' ' + CAST(cal.CalendarYear AS NVARCHAR(4)) AS YearMonthL
abel,
        COUNT(DISTINCT fs.InvoiceID)            AS InvoiceCount,
        COUNT(*)                                AS SalesLineCount,
        SUM(fs.LineTotal)                       AS TotalRevenue,
        SUM(fs.GrossProfit)                     AS TotalGrossProfit,
        ROUND(SUM(fs.GrossProfit)
              / NULLIF(SUM(fs.LineTotal),0)*100, 2) AS AvgMarginPct,
        ROUND(AVG(CAST(fs.DaysUntilInvoice AS DECIMAL(6,2))),2) AS AvgDaysToInvoice
    FROM    fact.Sales fs
    INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
    INNER JOIN dim.Calendar  cal ON cal.DateKey    = fs.OrderDateKey
    GROUP BY emp.FullName, emp.Department,
             cal.CalendarYear, cal.MonthNumber, cal.MonthName
) AS src
ON (    tgt.SalesRepName  = src.SalesRepName
    AND tgt.CalendarYear  = src.CalendarYear
    AND tgt.MonthNumber   = src.MonthNumber)
-- Row exists: update if measures have changed
WHEN MATCHED AND tgt.TotalRevenue <> src.TotalRevenue THEN
    UPDATE SET
        tgt.InvoiceCount     = src.InvoiceCount,
        tgt.SalesLineCount   = src.SalesLineCount,
        tgt.TotalRevenue     = src.TotalRevenue,
        tgt.TotalGrossProfit = src.TotalGrossProfit,
        tgt.AvgMarginPct     = src.AvgMarginPct
-- Row does not exist: insert new
WHEN NOT MATCHED BY TARGET THEN
    INSERT (SalesRepName, Department, CalendarYear, MonthNumber, MonthName,
            YearMonthLabel, InvoiceCount, SalesLineCount,
            TotalRevenue, TotalGrossProfit, AvgMarginPct, AvgDaysToInvoice)
    VALUES (src.SalesRepName, src.Department, src.CalendarYear, src.MonthNumber,
            src.MonthName, src.YearMonthLabel, src.InvoiceCount, src.SalesLineCount,
            src.TotalRevenue, src.TotalGrossProfit, src.AvgMarginPct, src.AvgDaysToInvoic
e);
```

`MERGE` is the bridge between the simple TRUNCATE + INSERT approach (appropriate for small, fast-loading tables) and the incremental update approach (appropriate for large tables or when preserving history matters). You will use it extensively in DBAS 2103.

Microsoft documentation:
MERGE (Transact-SQL)

Transactions for Multi-Step Scripts

When a source-to-target script involves multiple DML statements that must all succeed or all fail together, wrap them in a transaction:

```
BEGIN TRANSACTION;

    -- Step 1: Clear target
    TRUNCATE TABLE dbo.SalesRepMonthlySummary;

    -- Step 2: Populate
    INSERT INTO dbo.SalesRepMonthlySummary (...) SELECT ...;

    -- Step 3: Verify before committing
    DECLARE @SourceRev DECIMAL(18,2);
    DECLARE @TargetRev DECIMAL(18,2);

    SELECT @SourceRev = ROUND(SUM(LineTotal), 2) FROM fact.Sales;
    SELECT @TargetRev = ROUND(SUM(TotalRevenue), 2) FROM dbo.SalesRepMonthlySummary;

    IF @SourceRev = @TargetRev
    BEGIN
        PRINT 'Reconciliation passed. Committing.';
        COMMIT;
    END
    ELSE
    BEGIN
        PRINT 'Reconciliation FAILED. Rolling back. Source: '
              + CAST(@SourceRev AS NVARCHAR)
              + ' Target: ' + CAST(@TargetRev AS NVARCHAR);
        ROLLBACK;
    END
```

This pattern makes the entire load atomic — the target is either fully updated with verified data, or exactly as it was before (the truncate is also rolled back). It is the T-SQL equivalent of SSIS's transactional package execution.

Microsoft documentation:
Transactions (Transact-SQL)

Indexes on Summary Tables

`CREATE TABLE` creates the structure; `CREATE INDEX` makes it fast. For a summary table queried by analysts and BI tools:

```
-- Primary key creates a clustered index automatically (data sorted by SummaryKey)
-- Add non-clustered indexes for common filter/join columns

CREATE NONCLUSTERED INDEX IX_SalesRepMonthlySummary_Rep
    ON dbo.SalesRepMonthlySummary (SalesRepName);

CREATE NONCLUSTERED INDEX IX_SalesRepMonthlySummary_Period
    ON dbo.SalesRepMonthlySummary (CalendarYear, MonthNumber);

-- Covering index: includes measure columns for a common query pattern
CREATE NONCLUSTERED INDEX IX_SalesRepMonthlySummary_Rep_Period_Rev
    ON dbo.SalesRepMonthlySummary (SalesRepName, CalendarYear, MonthNumber)
    INCLUDE (TotalRevenue, InvoiceCount);
```

The last index is a **covering index** — it includes the measure columns that a common query needs, so SQL Server can answer the query entirely from the index without reading the main table data. For analytical queries that consistently access the same set of columns, covering indexes dramatically improve performance.

Industry Perspectives

Source-to-Target Scripts in the Real World

In production data environments, T-SQL source-to-target scripts serve three common purposes:

Prototyping: A data engineer writes a T-SQL script to verify logic before investing in a full SSIS package. The script proves the joins are correct, the aggregations produce the right numbers, and the target schema supports the required queries. Only then is the SSIS package built.

One-time loads: Historical data migrations, backfills, and one-off reporting tables that run once and never need scheduling. A T-SQL script is appropriate when the operational overhead of maintaining an SSIS package is not justified.

Data quality corrections: When a production ETL has loaded incorrect data, a T-SQL script may be used to correct it in-place — an UPDATE from a corrected source — rather than rerunning the full ETL.

Understanding when T-SQL is the right tool and when SSIS is the right tool is a professional judgment that comes with experience. This chapter gives you the T-SQL skills to make that judgment informed.

References and Further Reading

1. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapters 10–11 cover DDL and DML in depth, including constraints, identity columns, and sequences.

2. Microsoft. (2024). *CREATE TABLE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql

3. Microsoft. (2024). *ALTER TABLE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-table-transact-sql

4. Microsoft. (2024). *INSERT (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/insert-transact-sql

5. Microsoft. (2024). *MERGE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/merge-transact-sql

6. Microsoft. (2024). *SELECT INTO (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/queries/select-into-clause-transact-sql

7. Microsoft. (2024). *CREATE INDEX (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-index-transact-sql

8. Dolinger, P. (2027). *ETL for Business Intelligence: A practical guide to data provisioning, dimensional modelling, and pipeline design*. NSCC Institute of Technology. CC BY 4.0. — The companion textbook covering SSIS implementation of the concepts introduced in this chapter.